

SINDHU Trading Bot

Poori System Report -- Features, Database, Strategies aur Results

Real, live database aur actual codebase se banayi gayi report

Generated: 2026-09-08

Table of Contents

Table of Contents	2
SECTION 1 -- Poori Feature List (A to Z)	3
ai_integration / knowledge_compiler / knowledge_engine / self_learning_engine / sindhu_strategy	3
backtest_engine / strategies	15
data_engine / external_signals	26
paper_trading (live trading engine + Telegram)	37
sindhu_web (dashboard + API layer)	50
evolution_engine / automation_pipeline / dashboard / cloud_runtime / scripts	63
SECTION 2 -- Data Storage / Database Schema	71
SECTION 3 -- Strategy Inventory	74
SECTION 4 -- Backtest Results	81
SECTION 5 -- Paper Trading Summary	87

SECTION 1 -- Poori Feature List (A to Z)

Ye section poori SINDHU codebase ke har module ka exhaustive breakdown hai -- har function, class, endpoint, check, gate ya guard jo koi real kaam karta hai, wo yahan list hai, saath mein 1-2 line ka Hinglish brief. Ye list source code se seedha nikali gayi hai (koi assumption nahi).

ai_integration / knowledge_compiler / knowledge_engine / self_learning_engine / sindhu_strategy

ai_integration

__init__.py

- Package docstring / architectural boundary: Ye sirf documentation hai — batata hai ke AI trading engine ka hissa nahi hai, sirf optional assistant hai jo import/parsing mein madad karta hai. Koi code logic nahi hai is file mein.

chunking.py

- estimate_tokens(text): Text ki length ko char/4 se divide karke rough token count nikalta hai, real tokenizer nahi hai, sirf estimate hai.
- needs_chunking(text): Check karta hai ke text bohot bara hai (6000 tokens se zyada) to chunk karna parega ya nahi.
- split_into_chunks(text): Bare document ko paragraph boundaries par split karta hai chote chunks mein, har chunk ke end mein thora overlap (400 chars) rakhta hai taake koi rule paragraph break ke beech mein cut na ho. Agar ek hi paragraph bohot bara ho to usay hard-split kar deta hai.
- merge_chunks(cleaned_chunks): AI se clean hue chunks ko wapis ek document mein jorta hai blank lines ke saath.

youtube_import.py

- transcript_api_available(): Check karta hai ke youtube-transcript-api package installed hai ya nahi, taake frontend button disable/enable ho sake.
- extract_video_id(url_or_id): YouTube URL (watch/shorts/embed/youtu.be) ya bare video ID se 11-character video ID nikalta hai regex patterns se.
- clean_transcript(raw_text): Transcript se noise hataata hai — [Music]/[Applause] tags, music note symbols, filler words (um, uh, you know), aur extra whitespace collapse karta hai.
- fetch_transcript(video_id): YouTube API se transcript fetch karta hai (dono old aur new library version ke saath compatible), kabhi crash nahi karta — har error string ke roop mein return hota hai.
- import_from_url(url): Poora YouTube import flow — URL se video ID nikalta hai, transcript fetch karta hai, clean karta hai, aur final cleaned text return karta hai.

quality_score.py

- _strategy_completeness(strategy_result): Strategy ke config mein kitne zaroori fields (entry, exit, stop loss, take profit, risk, timeframe) filled hain uska percentage nikalta hai.
- compute_knowledge_score(doc): Compiled document se completeness%, confidence%, automation_ready, backtesting_ready, aur overall knowledge_score calculate karta hai. AI-assisted documents ke liye unresolved notes ko blocking nahi maanta (sirf transparency note hai), lekin rule-based path mein genuinely blocking hai. Hidden rules ka count aur unka average confidence bhi nikalta hai.

import_queue.py

- enqueue(...): Ek naya import item queue mein daalta hai (pending/processing/completed/failed state machine), aur background worker start karta hai agar chal nahi raha.
- enqueue_batch(items): Multiple import items ko ek saath queue mein daalta hai.

- `retry(item_id)`: Failed (ya completed) item ko wapis pending status mein reset karta hai taake worker usay dobara process kare. Currently processing item ko retry nahi hone deta.
- `retry_all_failed()`: Saare failed items ko ek saath retry karta hai.
- `_process_one(item)`: Ek queue item ko actual `import_document()` function se process karta hai aur result (completed/failed) ko database mein save karta hai.
- `_worker_loop()`: Background thread jo continuously pending items ko queue se claim karke process karta rehta hai, ek waqt mein sirf ek item.
- `ensure_worker_running()`: Ensure karta hai ke background worker thread sirf ek dafa start ho (thread-safe lock ke saath).

web_research.py

- `is_trusted_source(url)`: URL ka domain check karta hai ke wo trusted allow-list (babypips, investopedia, tradingview, binance, cmegroup) mein hai ya nahi — sirf inhi sources se content accept hota hai.
- `_TextExtractor` (class): HTML page se sirf readable text nikalta hai, script/style/nav/footer tags ko ignore karta hai.
- `_resolve_ddg_redirect(href)`: DuckDuckGo ke redirect links (`//?uddg=...`) ko unwrap karke asli article URL nikalta hai.
- `search_web(query, max_results)`: DuckDuckGo ke free HTML search endpoint se web search karta hai (koi paid API key nahi chahiye), scrape fail ho to empty list return karta hai crash ki jagah.
- `fetch_article_text(url)`: Kisi bhi URL se page fetch karke usay plain text mein convert karta hai.
- `queue_from_url(url, title)`: Ek specific article URL ko trusted-source check ke baad fetch karke standard import pipeline mein queue karta hai.
- `research_and_queue(query, max_results)`: Dashboard se call hone wala main function — web search karta hai, trusted sources filter karta hai, articles fetch karke import queue mein daalta hai. Sirf manually trigger hota hai, koi automatic/scheduled run nahi hai.

extraction_lock.py

- `_lock_from_report(report)`: Kisi strategy ke fidelity report se lock status nikalta hai — agar rules missing hain aur override nahi hua to strategy locked hai.
- `check_strategy_lock(strategy_id)`: Ek strategy ka lock status check karta hai (locked/overridden/missing rules count). Report na ho to kabhi lock nahi hota.
- `check_strategy_locks_bulk(strategy_ids)`: Multiple strategies ka lock status ek hi query mein check karta hai (list views ke liye performance optimization).
- `lock_message(lock_status, lang)`: Locked strategy ke liye plain-language (Roman Urdu ya English) explanation banata hai ke kyun test nahi ho sakti.
- `raise_if_locked(strategy_id, lang)`: Backtest/optimize/paper-trading shuru karne se pehle check karta hai — agar locked hai to error raise karta hai.
- `set_override(strategy_id, overridden)`: CEO ke override decision ko save karta hai taake locked strategy phir bhi test ho sake.
- `verification_summary(strategy_id, lang)`: Poora side-by-side comparison data banata hai — kaunse rules samajh aaye, kaunse nahi, plain-language summary ke saath.

fragment_merge.py

- `merge_list(*lists)`: Multiple lists ko duplicate hataate hue ek list mein combine karta hai.
- `_merge_entry_rule_groups(existing_groups, new_groups)`: Entry rule groups ko unke label ke hisaab se merge karta hai — same label wale group ki conditions union hoti hain, naya label nayi entry banti hai.
- `merge_fragment_additive(existing, fragment)`: Do partial strategy extraction fragments ko additive tareeke se merge karta hai — koi bhi field jo pehle se resolve ho chuki hai usay overwrite nahi karta, sirf naye info ko add karta hai.

dictionary_builder.py

- `_tighten_expansion(phrase, acronym)`: Acronym expansion phrase se sirf wo trailing words rakhta hai jinke initials acronym se match karte hain (jaise "Smart Money Technique" + "SMT").
- `_is_known(term)`: Check karta hai ke koi term pehle se static ya AI-discovered dictionary mein maujood hai.
- `discover_from_raw_text(text)`: Text mein se parenthetical acronym patterns (jaise "Break of Structure (BOS)") dhoondh kar naye dictionary terms banata hai, bina AI ke.
- `resolve_alias(name)`: Agar koi term pehle CEO ne clarify kiya tha to uska canonical naam wapis deta hai.
- `save_learned_alias(unknown_term, canonical_name, doc_id)`: User ki clarification (jaise "smc_ob" = "order_block") ko permanently save karta hai taake future imports mein dobara na poocha jaye.
- `save_discovered_terms(entries, doc_id)`: Naye discovered terms (AI se ya deterministic scan se) ko dictionary table mein save karta hai, already-known terms skip kar deta hai.

claim_extraction.py

- `extract_claimed_win_rate(raw_text)`: Document mein se performance claims (jaise "wins 60% of the time") ko regex se dhoondh kar extract karta hai, taake baad mein real backtest result se compare kiya ja sake. Koi AI call nahi, sirf pure regex.

correction_learning.py

- `record_resolution(field, value, strategy_id)`: Jab CEO koi missing-field question resolve karta hai to uska record save karta hai (sirf entry_timeframe, risk_pct, risk_reward jaise safe fields ke liye — stop_loss kabhi nahi).
- `learned_suggestion(field)`: Agar last 3 resolutions (kam se kam 2 alag strategies se) same answer dete hain to wo consistent pattern suggest karta hai, warna None deta hai.
- `apply_learned_corrections(cfg)`: Config mein missing fields ko automatically fill karta hai agar unambiguous learned pattern maujood hai, aur kya apply hua uski list return karta hai (transparency ke liye).

importer.py

- `_content_hash(text)`: Raw input text ka normalized SHA256 fingerprint banata hai, taake identical document dobara AI ko na bheja jaye (dedup cache).
- `_categorize_notes(notes)`: Compiler ke unresolved notes ko categories mein baantata hai (missing_rules, unknown_indicators, unknown_pattern, unknown_session, unknown_risk_rule, other) Import Report ke liye.
- `_maybe_trigger_pipeline(doc_dict)`: Har saved strategy ke liye automatically backtest pipeline ko queue mein daalta hai (automation_pipeline.submission_queue ke through), taake multiple imports ek saath process ho sakein.
- `_empty_input_result(error_text)`: Jab koi text hi nahi diya gaya to standard empty error result banata hai.
- `_offline_result(raw_text, title, source_hint, ai_error, content_type)`: Purana rule-based (non-AI) pipeline chalata hai jab AI disabled ho ya fail ho jaye.
- `import_document(raw_text, title, source_hint, use_ai, input_kind, content_type)`: Poora AI Knowledge Learning Engine ka main orchestrator — YouTube URL ho to transcript fetch karta hai, dedup cache check karta hai, sentence-level ya deep-understanding extraction chalata hai, self-correction apply karta hai, dictionary terms save karta hai, final document compile/save karta hai. Ye ek hi function hai jo har import type (paste, upload, YouTube, queue) handle karta hai.

self_correction.py

- `record_outcome(level, strategy_name, repairs, remaining)`: Har self-correction ka outcome telemetry file mein log karta hai (level 0-3), taake Level 2 ki rate track ki ja sake.
- `get_telemetry()`: Self-correction ke stats (kitni baar clean, auto-fixed, targeted AI, flagged) aur Level 2 ki rate percentage return karta hai.
- `repair_is_acceptable(original, candidate)`: Check karta hai ke koi repair genuinely strategy ko fix kar raha hai bina kuch aur torhe (jaise position kholne/band karne ki ability na khoye).
- `_repair_duplicate_exit_clauses(config)`: Agar exit condition entry condition jaisi hi hai (jo trade ko turant band kar degi) to usay hataata hai.
- `_repair_unreachable_exit_gate(config)`: Agar exit rules sirf slow-moving price levels (support/resistance) par based hain aur real SL/TP maujood hai to unhe hataata hai kyunke ye trade ko turant band kar dete.

- `_repair_contradictory_entry_gate(config)`: Agar entry bucket mein bullish aur bearish dono conditions hain (jo kabhi ek saath true nahi ho saktin), to usay do alag direction-tagged entry groups mein split karta hai.
- `_repair_dead_entry_buckets(config)`: Agar `entry_rule_groups` populated hai to purani flat `entry_conditions/long/short` lists ko clear karta hai kyunke wo kabhi evaluate hi nahi hongy (dead code).
- `_repair_known_term_aliases(config)`: Agar koi indicator/concept naam pehle CEO ne clarify kiya tha to usay automatically known naam se replace karta hai.
- `_repair_learned_field_corrections(config)`: `correction_learning` module se learned patterns apply karta hai missing fields ke liye.
- `auto_repair(config)`: Level 1 ke saare deterministic (AI-free) repairs ko ek saath chalata hai aur description list return karta hai.
- `_build_level2_prompt(config, issues)`: Level 2 ke liye chota targeted AI prompt banata hai — sirf failing rule aur uske relevant source lines, poora document nahi.
- `_apply_level2_patch(config, patch)`: AI ke Level 2 response se sirf jo keys return hui unhe apply karta hai, raw dicts ko real Condition objects mein convert karta hai.
- `targeted_ai_fix(config, issues, use_ai)`: Level 2 — ek chota, specific AI call jo sirf detected problem ko fix karne ki koshish karta hai, agar fix na kare to discard kar deta hai.
- `partial_reextract(config, field, use_ai)`: User-triggered re-extraction — kisi ek field (`entry/exit conditions`, `stop_loss`, `take_profit`) ko dobara raw text se nikalta hai bina poori strategy dobara import kiye.
- `plain_language_summary(issues)`: Technical validator errors ko non-trader ke samajhne layak plain language mein translate karta hai (Level 3).
- `self_correct(config, use_ai, allow_level2)`: Poora 3-level self-correction flow ka orchestrator — pehle safety check, phir Level 1 auto-repair, phir Level 2 targeted AI fix, aakhir mein Level 3 user ko flag karna.

file_extractors.py

- `extract_text_from_pdf(file_bytes)`: PDF file se text nikalta hai pypdf library se.
- `extract_text_from_docx(file_bytes)`: Word document se paragraphs aur tables ka text nikalta hai (tables '|' se separated).
- `extract_text_from_csv(file_bytes)`: CSV file ki har row ko '|' se joined line mein convert karta hai.
- `extract_text_from_xlsx(file_bytes)`: Excel file ki har sheet ke har row se text nikalta hai openpyxl se.
- `extract_text(filename, file_bytes)`: File extension ke hisaab se sahi extractor function ko call karta hai (pdf/docx/txt/md/csv/xlsx). Image OCR support nahi hai (deliberately, kyunke test nahi ho sakta).

sentence_level_extraction.py

- `_context_prefix(model_label, section_label)`: Ek isolated statement ke saath context line banata hai (kis model/section ka hissa hai) taake AI behtar samajh sake.
- `_call_one_statement(...)`: Ek single statement ko AI ko bhejta hai extraction ke liye, document background context ke saath.
- `_route_labeled_fragment(fragment, model_label)`: Agar statement kisi specific model/setup se belong karta hai to uski conditions ko us model ke apne `entry_rule_groups` mein route karta hai (multi-model documents ke liye).
- `run_sentence_level_extraction(raw_text, source_hint, content_type, max_retries)`: Main function — document ko individual statements mein split karta hai, har ek ko alag AI call se extract karta hai (bara document ke bajaye chote pieces), phir jo statements resolve nahi hue unhe retry karta hai (Retry Until Reconciled). Isse pehle Batch 3 ke muqable kaafi zyada rules capture hote hain.
- `_describe_fragment(fragment)`: Ek extracted fragment ka one-line plain-English summary banata hai (verification view ke liye).
- `_fmt_condition(c)`: Ek condition dict ko readable string mein format karta hai.

deterministic_rules.py

- `split_into_statements(text)`: Document ko individual statements/sentences mein deterministic tareeke se split karta hai (headings aur table separators ko ignore karke).

- `_signals_in(statement)`: Ek statement mein kaunse signal groups (conditional, entry_exit, comparison, session, numeric) maujood hain check karta hai.
- `is_rule_candidate(statement)`: Deterministic (no-AI) check ke koi statement trading rule ho sakta hai ya nahi, based on keyword signals.
- `extract_document_preambles(text)`: Document ke opening/setup statements ka short summary nikalta hai (per model agar multi-model document hai) taake baad ke statements backward references samajh sakein.
- `count_candidate_rules(text)`: Poora deterministic ground-truth checklist banata hai — har candidate rule ko id, text, signals, model_label, section_label ke saath list karta hai. Ye AI extraction ko measure karne ka ground truth hai, kabhi vary nahi hota.
- `_clean_heading_line(raw_line)`: Ek raw line se markdown/bold formatting hataakar clean heading text nikalta hai.
- `detect_model_sections(text)`: Document mein "Model N:"/"Setup N:" jaisi headings dhoondh kar multi-model documents detect karta hai, aur agar ambiguous mentions hain (bina clear heading ke) to flag karta hai.
- `_is_bold_only_heading_line(stripped_line)`: Check karta hai ke line sirf bold-wrapped section label hai (jaise "Placement:"), rule content nahi.
- `_clean_section_heading_line(raw_line)`: Kisi bhi structural heading (markdown ya bold-only) ko clean karke uska readable text nikalta hai.
- `split_into_statements_with_labels(text)`: Har statement ko uske model_label aur section_label ke saath pair karta hai — routing decisions ke liye model_label, context hints ke liye section_label.

strategy_builder.py

- `_is_degenerate_raw(cond)`: Check karta hai ke koi raw condition khaali hai ya sirf SL/TP type naam repeat kar rahi hai (koi real rule nahi).
- `_describe_raw(cond_dict)`: Kisi condition ke liye human-readable fallback description banata hai jab wo raw type mein demote ho.
- `build_condition(cond_dict)`: AI ke output se ek real Condition object banata hai, agar indicator/concept/session backtest engine ki known vocabulary mein nahi hai to usay "raw" type mein demote kar deta hai (kabhi fake executable rule nahi banata).
- `build_stop_loss_take_profit(sltf_dict)`: SL/TP spec ko sanitize karta hai — agar type semantically incomplete hai (jaise "rr" bina value ke) to "unknown" bana deta hai, siwaye "signal_candle" ke jo bina buffer number ke bhi meaningful hai.
- `build_strategy_config(ai_strategy, name, raw_text)`: AI ke structured output se poora real StrategyConfig object banata hai — conditions, timeframes, indicators sab kuch build karta hai aur zaroori repairs (structural stop, concepts_used sync) apply karta hai.
- `sync_concepts_used(config)`: Agar koi condition kisi concept ko reference karti hai lekin concepts_used list mein wo naam nahi hai to usay add kar deta hai, taake engine wo concept actually compute kare.
- `_repair_structural_stop(config)`: Agar stop_loss/take_profit "structure" type hai lekin koi structural concept (support/resistance/fvg/order_block) concepts_used mein nahi hai to fallback (support+resistance) add kar deta hai.
- `build_lesson(ai_lesson)`: AI ke lesson dict se seedha ek real Lesson object banata hai, bina text re-parsing ke (AI ne already structured data diya hai).

schema.py

- `build_structured_extraction_prompt(source_hint, content_type)`: Poora AI system prompt banata hai jo AI ko batata hai kis vocabulary aur JSON schema ka use karke strategy extract karni hai — bohot detailed instructions (entry/exit logic, branching, indicator-vs-indicator, self-verification checklist, etc.) is mein hain.
- `_extract_json_object(text)`: AI response se JSON object nikalta hai (code fences hatakar, pehle { se last } tak).
- `_escape_literal_control_chars_in_strings(raw)`: AI ka output mein agar real newlines/tabs JSON strings ke andar hain (invalid JSON) to unhe escape karta hai taake parsing fail na ho.
- `_parse_json_object(raw_text)`: Raw AI text se safely JSON dict parse karta hai, fail hone par None deta hai.
- `_clean_str_list, _clean_condition, _clean_sltp, _clean_strategy, _clean_lesson, _clean_dictionary_term, _clean_inferred_field`: In sab functions ka kaam hai AI ke raw output ko sanitize/validate karna — har field ka type check, invalid values ko safe defaults mein convert karna.

- `parse_structured_response(raw_text)`: Poora AI response ko parse karke ek fully-sanitized dict banata hai jo `strategy_builder` use karta hai.
- `build_rule_inventory_prompt()`: Ek narrow prompt banata hai jo AI ko sirf rules count/list karne ko kehta hai, extraction nahi (Batch 3 ka checklist step).
- `parse_rule_inventory_response(raw_text)`: Rule inventory response ko parse karta hai.
- `build_scoped_extraction_prompt(scope, source_hint, content_type)`: Ek scope-restricted prompt banata hai (entry/exit/filters mein se ek) jo sirf us scope ke fields populate karta hai.
- `build_comparison_prompt(rule_inventory, captured_summary)`: AI ko checklist aur captured summary dono deta hai taake wo judge kare kaunse rules genuinely capture hue.
- `build_retry_prompt(missing_rules, source_hint, content_type)`: Sirf missing rules ko target karke ek retry prompt banata hai.
- `parse_comparison_response(raw_text)`: Comparison AI response ko parse karta hai.
- `build_statement_extraction_prompt(source_hint, content_type)`: Sentence-level extraction ke liye ek condensed (short) prompt banata hai jo har single statement call ke saath bheja jata hai — bara prompt bhejne se cost bohot barh jati thi.
- `parse_statement_response(raw_text)`: Single-statement AI response ko parse karta hai (is_rule flag aur strategy fragment).

deep_understanding.py

- `call_provider_chain_generic(...)`: Shared function jo provider fallback chain try karta hai, usage log karta hai, aur response ko parse karta hai — poore module ka core call loop.
- `_call_provider_chain(text, chain, source_hint, content_type)`: Ek poore document (ya chunk) ke liye structured extraction attempt karta hai.
- `_merge_dictionary_terms, _merge_inferred_fields, _merge_lessons, _merge_conditions, _merge_strategies`: Multiple chunks se aane wale results ko ek single unified result mein merge karte hain (deduplication ke saath).
- `understand_document_structured(raw_text, use_ai, source_hint, content_type)`: Main entry point — agar document bara hai to chunk karta hai, har chunk ko AI se samjhwata hai, phir sab results ko merge karta hai. Result None hone par caller offline mode mein fall back karta hai.

multi_pass_extraction.py

- `_merge_scoped_strategies(entry_s, exit_s, filters_s)`: Teen alag scoped passes (entry/exit/filters) se aaye results ko ek strategy mein merge karta hai.
- `_describe_captured_strategy(strategy)`: Merged strategy ka plain-text summary banata hai comparison call ke liye.
- `run_multi_pass_extraction(raw_text, source_hint, content_type)`: Batch 3 ka multi-pass approach — pehle rule inventory (count+list), phir 3 scoped extraction passes (entry/exit/filters), phir comparison call jo batata hai kitne rules genuinely capture hue.
- `_merge_retry_fragment(existing, fragment)`: Retry pass se recovered fields ko already-captured strategy mein additive merge karta hai.
- `run_multi_pass_extraction_with_retry(raw_text, source_hint, content_type, max_retries)`: Multi-pass extraction ke baad, missing rules ke liye targeted retry calls karta hai jab tak sab capture ho jaye ya max retries khatam ho jayen.

config.py

- `_default_settings()`: Har AI provider (claude/groq/openai/gemini/deepseek) ke liye default settings banata hai, env variable se Groq key seed kar sakta hai (cloud deploy ke liye).
- `load_settings()`: Settings file load karta hai, agar naya provider add hua hai to defaults merge karta hai (backward compatibility).
- `save_settings(settings)`: Settings ko JSON file mein save karta hai.
- `get_provider_settings(provider)`: Ek specific provider ki settings return karta hai.
- `update_provider_settings(provider, fields)`: Provider settings update karta hai.

- `set_enabled(provider, enabled)`: Kisi provider ko enable/disable karta hai.
- `set_active_provider(provider)`: Active provider set karta hai.
- `record_test_result(provider, status, message, now_iso)`: Connection test ka result save karta hai.
- `mask_key(key)`: API key ko partially mask karta hai display ke liye (security).
- `active_provider_name()`: Current active aur usable provider ka naam deta hai.
- `provider_fallback_chain()`: Smart Provider Switching order banata hai — active provider pehle, phir baaki enabled providers priority order mein. Empty list matlab "Offline Mode".

providers.py

- `AIResult` (dataclass): AI call ka result structure (ok/text/error/tokens/latency).
- `_retry_after_seconds(header_value, default)`: Rate-limit response ke Retry-After header ko parse karta hai.
- `AIProvider.chat(user, system)`: Har provider ka core method — API ko call karta hai, retries handle karta hai (timeout, network error, rate limit 429 with backoff, server error 5xx), kabhi exception nahi throw karta, hamesha `AIResult` return karta hai.
- `AIProvider.test_connection()`: Provider ko test karta hai ek simple "OK" reply mangwakar.
- `ClaudeProvider`, `OpenAIProvider`, `GroqProvider`, `DeepSeekProvider`, `GeminiProvider` (classes): Har provider ke apne specific API request/response format ko handle karte hain (Anthropic Messages API, OpenAI-compatible chat completions, Gemini generateContent).
- `get_provider(name, settings)`: Provider naam se sahi provider class ka instance banata hai settings ke saath.

knowledge_compiler

__init__.py

- Sirf documentation — batata hai ke ye package deterministic, rule-based hai (koi AI/ML nahi), aur existing Strategy/Lesson Engine ko wrap karta hai.

sections.py

- `extract_title(text)`: Document ki pehli line agar title declaration jaisi lage ("Strategy Name: ...") to usay alag nikalta hai taake wo rule ki tarah scan na ho.
- `Section` (dataclass): Ek document section ki representation (kind, heading, text, line range).
- `_header_kind(heading_text)`: Heading text ko keyword-matching se ek section kind (entry_rules, exit_rules, risk, etc.) mein classify karta hai.
- `_match_header(line)`: Check karta hai ke koi line markdown header, bold header, ya colon header hai.
- `detect_sections(text)`: Poore document ko labelled sections mein todta hai (Summary, Entry Rules, Exit Rules, Risk, etc.), headers na milne to poora text ek "body" section ban jata hai.
- `inline_if_then_lines(text)`: Document mein kahin bhi IF-THEN style lines dhoondh leta hai, chahe header ke bina hi ho.
- `sections_for_rules(sections)` / `sections_for_lessons(sections)`: Sections ko unke kind ke hisaab se filter karte hain — rule extraction ke liye ya lesson extraction ke liye.

dictionary.py

- `DictEntry` (dataclass): Ek dictionary entry ka structure (canonical name, category, aliases, concept_key, description).
- Dictionary building code (module-level `_entry()` calls): Backtest engine ke existing concept/session keywords ko reuse karke, plus naye structural terms (PDH, PDL, mitigation, inducement, etc.) aur risk/psychology terms add karke poori canonical dictionary banata hai.
- `alias_in_text(alias, text_lower)`: Word-boundary-aware check karta hai ke koi alias text mein मौजूद hai (jaise "sma" ko "Smart" ke andar match hone se bachata hai).
- `lookup(term)`: Kisi bhi raw alias/term se uski `DictEntry` nikalta hai.
- `normalize_term(raw)`: Kisi alias ka canonical naam return karta hai.
- `lesson_category_for(canonical_or_alias)`: Kisi term ke liye best-guess lesson category deta hai.

- `normalize_text(text)`: Poore text mein har structural/indicator/session/trend alias ko canonical phrase se replace karta hai taake strategy_parser usay samajh sake. Risk/psychology terms ko replace nahi karta (kyunke wo strategy_parser ki apni regex logic tor sakta).

classifier.py

- `_count_keywords(text_lower, keywords)`: Text mein keywords ka total count nikalta hai.
- `_count_category(text_lower, category)`: Kisi dictionary category ke aliases ka total count text mein nikalta hai.
- `DocClassification (dataclass)`: Classification result ka structure (doc_type, confidence, scores).
- `classify_document(text)`: Deterministic keyword-bucket scoring se document ko classify karta hai (STRATEGY/LESSON/MIXED/PSYCHOLOGY/RISK_MANAGEMENT/INDICATOR_GUIDE/MARKET_STRUCTURE/UNKNOWN). Kabhi 100% confidence nahi deta, aur signal na mile to UNKNOWN return karta hai (guess nahi karta).

lesson_extractor.py

- `_split_candidates(section_text)`: Section text ko bullets ya sentences mein split karke candidate lesson statements banata hai.
- `_find_tags(line_lower)`: Ek line mein dictionary ke kaunse tags/terms maujood hain dhoondh leta hai.
- `_classify_kind(section_kind, line_lower, tags)`: Ek candidate line ko kind (mistake/strength/weakness/psychology/risk_advice/tip/example/rule) mein classify karta hai.
- `_title_for(line)`: Ek line se lesson ka title banata hai (pehle 9 words).
- `extract_lessons(text, detected_sections)`: Document ke educational sections se candidate lessons nikalta hai — dictionary hit ya explicit lesson-shaped section chahiye, warna unrelated narrative ignore ho jati hai.

rule_extractor.py

- `_normalize_spoken_phrasing(text)`: Bole gaye phrasing ("stop loss", "1 to 3 risk reward") ko abbreviated form (SL, TP, 1:3) mein convert karta hai jo parser samajh sake.
- `_split_combined_directives(text)`: Ek line mein multiple directives (SL, TP, RR, Risk) ho to unhe alag lines mein split karta hai taake parser sab pakar sake.
- `extract_strategy(text, name, detected_sections)`: Strategy-relevant sections ko combine karke, dictionary normalize karke, aur backtest_engine.strategy_parser ko call karke ek StrategyConfig banata hai. Ye khud koi parsing nahi karta, sirf routing/normalization layer hai.

normalizer.py

- `detect_source_type(text, hint)`: Text ke patterns (timestamps, [music], "notebooklm", "chatgpt", form-feed) se guess karta hai ke document kahan se aaya (YouTube transcript, NotebookLM, PDF, etc.).
- `CompiledStrategyResult, CompiledLessonResult, CompiledDocument (dataclasses)`: Poore system ka standard internal document structure — har input document isi format mein convert hota hai chahe wo kahin se bhi aaya ho.

compiler_validator.py

- `_find_similar_strategy(config)`: Knowledge library mein sabse zyada overlapping concepts wali strategy dhoondh leta hai (defaults borrow karne ke liye).
- `_fill_default_indicator_periods(config, resolved_defaults)`: Missing indicator periods ko standard defaults (EMA=20, RSI=14, etc.) se fill karta hai.
- `resolve_and_validate(config)`: Validator errors ko fix karne ki koshish karta hai — safe defaults, dictionary standards, ya similar strategy se borrow karke. Safety-critical cheezein (missing entry/exit rules, invalid indicators) kabhi guess nahi karta, sirf clarification ke liye chorta hai.
- `force_ready(config, clarification_notes, resolved_defaults)`: AI-confident imports ke liye ek aakhri safety-net — agar sirf stop-loss missing hai to default value laga deta hai, lekin agar entry conditions sab "raw" (unexecutable) hain to phir bhi clarification maangta hai.

compiler.py

- `_concepts_in_text(text)`: Raw text mein maujood saare dictionary terms dhoondh leta hai (tagging/stats ke liye).

- `_touch_concepts(concepts, now)`: Har mile hue concept ka usage record update karta hai.
- `_existing_lesson_dna_map()`: Saari existing lessons ka DNA-to-id mapping banata hai duplicate detection ke liye.
- `_finalize_and_save_strategy(config, doc_id, now, force_backtest_ready, hidden_rules, confidence_pct)`: Shared save logic — dedupe, conflict check, validate, duplicate detection, aur strategy library mein save/version karta hai. Dono pipelines (old text-based aur AI-native) yehi function use karte hain.
- `_save_lesson_objects(lesson_entries, doc_id, now)`: Lessons ko dedupe karke (DNA fingerprint se) database mein save karta hai.
- `_compile_strategy(...)` / `_compile_lessons(...)`: Purane text-based pipeline ke liye strategy/lesson extraction wrapper.
- `compile_document(...)`: Main text-based entry point — classify, section-detect, extract, validate, save karta hai. Ye Offline Mode ka fallback hai jab AI available nahi ho.
- `compile_from_ai_extraction(...)`: AI-Native entry point — pehle se bane StrategyConfig/Lesson objects ko lekar dedupe/validate/save karta hai, kabhi purana text-parser nahi chalata. AI ka confidence 60% se zyada ho to automatically Backtesting Ready ban jati hai.

quality.py

- `_condition_key(cond)`: Ek condition ka unique comparison key banata hai duplicate detection ke liye.
- `dedupe_conditions(conditions)` / `dedupe_rules(config)`: Exact duplicate conditions ko har bucket se hataata hai.
- `merge_concepts(concepts_used)`: Concepts list ko canonicalize aur deduplicate karta hai.
- `detect_conflicts(config)`: Same bucket mein contradictory conditions (jaise ek hi indicator "above" aur "below" dono required) dhoondh kar report karta hai (auto-resolve nahi karta).
- `strategy_dna(config)`: Ek stable fingerprint banata hai strategy ke liye, taake do differently-worded pastes agar same rules describe karte hain to same DNA par converge ho jayen.
- `lesson_dna(lesson)`: Lesson ke liye bhi similar fingerprint banata hai.
- `find_duplicate_strategy(dna, library_list_all, library_load)`: Library mein same DNA wali existing strategy dhoondh leta hai.
- `find_duplicate_strategy_groups(...)`: Poori library ko DNA ke hisaab se group karta hai taake duplicate groups nazar aayen (cleanup ke liye).
- `searchable_tags(doc_type, concepts_used, timeframes, direction, category)`: Search/filter ke liye tags ki list banata hai.

knowledge_engine

`__init__.py`

- Ye file khaali hai, koi content nahi.

scoring.py

- `compute_knowledge_score(report)`: Simple arithmetic formula se Knowledge Score calculate karta hai (lessons count, active lessons, approved applications ke points), max 100%.
- `_impact_pct_from_stats(stats)`: Ek lesson ka impact percentage nikalta hai — kitni baar wo actually trade ko block kar chuka hai.
- `lesson_estimated_impact(lesson_id)`: Ek lesson ka estimated impact percentage deta hai.
- `list_lessons_with_stats(status, category)`: Saari lessons ko unke stats ke saath list karta hai — performance ke liye bulk query use karta hai (pehle 220 alag queries lagti thin, ab ek hi bulk query).

condition_eval.py

- `_DISPATCHABLE_CONCEPT_NAMES`: Wo concept names jo evaluator actually check kar sakta hai — baaki sab always-false hain (dangerous case jahan ek lesson silently har trade block/approve karta rehta).
- `condition_is_executable(cond)`: Check karta hai ke koi condition genuinely evaluable hai ya sirf always-false placeholder hai.

- `_array(df, col, arr_cache) / _get(df, i, col, arr_cache)`: Performance-optimized column access — pandas Series ko numpy array mein cache karke bar-bar slicing se bachata hai.
- `_indicator_column(indicator_name, params)`: Indicator naam se uska data column naam banata hai.
- `evaluate_condition(df, i, cond, arr_cache)`: Har lesson condition (concept, indicator_compare, price_compare, session, trend) ko ek particular bar par evaluate karta hai — True/False deta hai. Strategy engine se alag evaluator hai (kabhi backtest engine ki apni logic ko regress nahi karta).

lesson.py

- `_detect_rule_type(text)`: Text mein keywords se decide karta hai ke lesson "veto" (block_if_true) hai ya "requirement" (require_if_true).
- `_detect_direction(text)`: Text se lesson ki direction (bullish/bearish) detect karta hai keywords se.
- `Lesson.is_enforceable()`: Check karta hai ke lesson ki koi condition genuinely evaluable hai — sirf present hone se kaam nahi chalta.
- `Lesson.matches_market_type(market_type) / matches_timeframe(timeframe)`: Check karta hai ke lesson kisi specific market/timeframe ke liye restrict hai ya sab par apply hoti hai.
- `new_lesson(...)`: Ek nayi Lesson banata hai — description text ko `strategy_parser.parse_conditions()` se real conditions mein convert karta hai (koi AI nahi, deterministic keyword parser).
- `from_storage_dict(d)`: Database dict se Lesson object banata hai.

maturity.py

- `_real_strategy_books()`: Real strategy accounts ki list deta hai (synthetic "`__lessons__`" row exclude karke).
- `_evolution_gate_completions(min_threshold)`: Kitni strategies ne evolution ka 100-trade gate complete kiya hai count karta hai.
- `compute_maturity_metrics()`: System maturity ke liye real numbers ikattha karta hai (total strategies, 25+ trades wali, statistically proven positive wali, evolution completions, signals sent).
- `compute_maturity_level(metrics)`: In metrics se ek 1-5 level ladder compute karta hai (Bootstrapping se Fully Validated tak) — har level pichle level par build hota hai, manual set nahi hota.

engine.py

- `KnowledgeEngine.for_backtesting(...)`: Backtesting ke liye active lessons load karta hai.
- `KnowledgeEngine.for_paper_trading(...)`: Paper trading ke liye lessons load karta hai, market type/timeframe ke hisaab se filter karta hai.
- `KnowledgeEngine.has_active_lessons()`: Check karta hai ke koi enforceable lesson maujood hai.
- `KnowledgeEngine.check(df, i, direction)`: Har prospective trade entry ke liye saari active lessons check karta hai priority order mein — pehla jo block kare wahi ruk jata hai. Har check ko log ke liye buffer karta hai.
- `KnowledgeEngine.flush()`: Buffered lesson applications ko ek bulk insert mein database mein save karta hai (performance ke liye — per-bar write bohot slow tha).
- `get_display_knowledge_report()`: Dashboard ke liye knowledge report banata hai, lekin sirf currently-enforceable lessons ke stats count karta hai (broken lessons ka purana data include nahi hota jo har trade ko silently veto kar rahi thin).

self_learning_engine

`__init__.py`

- Documentation — batata hai ke ye package naye candidate strategies discover karta hai existing concepts ko combine karke (evolution_engine se alag, jo existing strategies ko tweak karta hai).

combination_scorer.py

- `_paper_trading_dna_by_strategy()`: Har saved strategy ke DNA tags ka mapping banata hai.

- `score_combinations(max_combo_size, min_combo_size, min_sample)`: 2-3 concept DNA-tag combinations ko real data (BOT-lineage scores + live paper-trading scores) se score karta hai, aur har combo ke liye best-performing coins bhi nikalta hai. Kabhi random guess nahi karta, sirf real evidence use karta hai.

candidate_builder.py

- `build_candidate(dna_combo, timeframe, variant)`: Ek DNA-tag combo (jaise liquidity+volume) se ek poor StrategyConfig manually banata hai — `deterministic_builder` ke existing condition-construction helpers reuse karta hai. `variant` parameter se har cycle mein thora different concept draw hota hai taake repetition na ho.

memory.py

- `check_duplicate_against_library(concepts_used, exclude_strategy_id)`: Naye candidate ko existing saved strategies se compare karta hai genuine rule-structure similarity check (Jaccard) se.
- `has_been_rejected_before(dna_combo, concepts_drawn)`: Check karta hai ke exact yehi combo+concepts pehle propose ho kar reject ho chuka hai.
- `attempt_count_for_combo(dna_combo)`: Kisi combo ko kitni baar try kiya gaya hai count karta hai.
- `record_outcome(...)`: Har candidate attempt (accepted ya rejected) ko permanent log mein save karta hai — kabhi rejection chupata nahi.
- `attempt_history(limit)`: Purani attempts ki history return karta hai.

ai_advisor.py

- `_extract_json(text)`: AI response se JSON object nikalta hai.
- `_fallback_choice(candidates, reason)`: Agar AI available nahi hai to top-scored combo ko fallback ke roop mein choose karta hai.
- `select_next_combination(candidates)`: AI se ek short-listed combinations mein se best next-try combination choose karwata hai — AI sirf ranking karta hai, khud koi concept invent nahi karta, koi trading decision nahi leta. Sirf ek provider call, retry nahi.

validation_gate.py

- `compute_period_metrics(batch_id)`: Ek backtest period (discovery ya validation) ke liye `total_trades`, `win_rate`, `profit_factor`, `avg_risk_reward` calculate karta hai.
- `compute_win_rate_benchmark()`: Existing profitable strategies (25+ trades, net positive PnL) ke sabse kam win-rate ko benchmark banata hai — koi invented number nahi.
- `evaluate(discovery_metrics, validation_metrics, structural_risk_reward)`: Poora validation gate — dono periods (discovery + validation) independently check karta hai minimum trades, profit factor, risk:reward, aur win-rate benchmark ke against. Dono periods mein pass hona zaroori hai.

explainability.py

- `build_report(dna_combo, drawn_concepts, ai_suggestion, gate_result, outcome, strategy_id, early_rejection_reason)`: Har discovered candidate (accepted ya rejected dono) ke liye ek clear, readable report banata hai — kaunse concepts combine hue, kya validation dikhaya, final decision kya hua aur kyun. Sirf existing data ko format karta hai, koi nayi calculation nahi.

discovery_cycle.py

- `compute_global_split(exchange, symbols, train_fraction)`: Poore symbol universe ki history ko discovery aur validation periods mein chronologically split karta hai (shuffle nahi karta).
- `_run_real_period_batch(...)`: Ek period ke date range ke liye real backtest chalata hai (same pipeline jo har manual strategy use karti hai).
- `should_run_new_cycle()`: Check karta hai ke naya discovery cycle chalana chahiye ya nahi — weekly cap (7 din mein ek baar) hai, daily nahi.

- `run_discovery_cycle(...)`: Poora end-to-end discovery cycle orchestrate karta hai — combinations score karta hai, AI se pick karwata hai, candidate build karta hai, duplicate/rejection memory check karta hai, real out-of-sample backtest chalata hai, validation gate se guzarta hai, aur accept/reject decision ko permanently record karta hai. Resource limits (Governor) bhi respect karta hai.
- `_dump(obj)`: JSON serialize karne ka helper.

sindhu_strategy

__init__.py

- Documentation — batata hai ke ye package roz naye strategy candidates generate karta hai (11 per day, 1 AI-assisted + 10 deterministic), `evolution_engine` se alag.

ai_builder.py

- `_extract_json(text)`: AI response se JSON nikalta hai.
- `build_ai_candidate()`: Din ka ek AI-assisted candidate banata hai — ek hi provider call karta hai (retry loop nahi), AI ko sirf allowed concepts/indicators ki list deta hai taake wo koi fake naam invent na kare. Fail hone par exception raise karta hai (retry nahi hota, sirf "aaj AI candidate nahi bana" maana jata hai).

lifecycle.py

- `validate_and_backtest(bot_strategy_id, exchange, symbols, settings, use_multiprocessing)`: Ek bot-generated candidate ko real validate/backtest/report pipeline se guzarta hai (same jo user-imported strategies use karti hain), Evolution Score compute karta hai, aur agar ye kisi evolution comparison ka child hai to automatically rollback check bhi karwata hai. Invalid config ko delete nahi karta, sirf record karta hai (permanent history).

deterministic_builder.py

- `dna_score_correlations()`: BOT-lineage correlations aur real paper-trading scores ko combine karke best-first DNA combo list banata hai (system ka poora accumulated data use karta hai).
- `_pick_dna_combo(index, correlations)`: Din ke har slot (0-9) ke liye deterministic tareeke se ek combo choose karta hai — real correlations khatam hone par DNA category pairs cycle karta hai.
- `_condition_for(name)`: Ek DNA name (indicator ya concept) ko sahi condition type mein convert karta hai — price-level indicators (EMA/SMA/VWAP) ke liye `price_compare`, RSI ke liye `indicator_compare` (oversold), MACD ke liye event concept, ATR/volume ko exclude karta hai (koi direction nahi hoti). Ye bug fix hai — pehle numeric indicators ko boolean concept mein wrap kiya jata tha jo kabhi true nahi hoti thi.
- `_usable_dna_names(pool)`: Non-directional indicators (ATR, volume) ko hata deta hai kyunke unka koi honest directional meaning nahi.
- `build_candidate(index, timeframe)`: Din ke slot ke liye ek deterministic (zero-AI) StrategyConfig banata hai — real correlation data se DNA combo choose karta hai, concepts draw karta hai, aur poori config assemble karta hai.

generator.py

- `_today(now_iso)`: ISO timestamp se date prefix nikalta hai.
- `generate_daily_candidates(now_iso, timeframe)`: Poora daily generation cycle — pehle ek AI-assisted candidate (agar reserve mil jaye), phir baaki slots deterministic candidates se bharta hai. Failed Hypothesis Memory check karta hai taake pehle se reject/rollback hui DNA combinations dobara na try ho. Daily cap (11) database-level atomic reservation se enforce hota hai, Python counting se nahi.
- `today_candidates(now_iso)`: Aaj generate hui saari candidates ki list deta hai.
- `_scheduler_loop()`: Background thread jo har ghante check karta hai ke naya din shuru hua hai ya nahi aur generation trigger karta hai.
- `start_daily_scheduler_thread()`: Scheduler thread ko start karta hai (agar pehle se nahi chal raha).

backtest_engine / strategies

backtest_engine

__init__.py

- Empty file, koi code nahi hai — sirf Python package marker hai.

strategy_loader.py

- `discover_strategies()` — `strategies/` folder ke andar sab `.py` files scan karke jo bhi `Strategy` class ke subclass milte hain unko dynamically import karta hai aur `{name: class}` dictionary return karta hai. Ye khud koi strategy nahi banata, sirf jo file mein already saved hai usko dhoondta hai.
- `get_strategy(name)` — naam se ek specific strategy class dhoond kar uska instance return karta hai; agar naam na mile to clear error deta hai available names ke sath.

mtf_context.py

- `MultiTimeframeContext.__init__` — multiple timeframes (bias/trend/entry/confirmation jaisi roles) ke liye har role ka OHLCV data load karta hai exchange aur symbol ke hisaab se.
- `MultiTimeframeContext.is_empty()` — check karta hai ke kisi bhi role ka data khali to nahi hai.
- `MultiTimeframeContext.build()` — sab timeframes ko entry-timeframe ke index par merge karta hai, is tarah ke koi look-ahead na ho — higher-timeframe ka data sirf tab visible hota hai jab wo bar band ho chuka ho (`merge_asof` backward direction se).

queue_runner.py

- `run_queue()` — multiple strategies ko ek ke baad ek queue mein chalata hai; har item apna khud ka batch ban jata hai, pause/stop/resume queue level par bhi kaam karta hai, aur progress callbacks queue-level aur batch-level dono report karte hain.

export.py

- `export_csv()` — batch ke result summary aur trades ko do CSV files mein export karta hai.
- `export_excel()` — ek Excel file banata hai jisme Summary, Coin Ranking, Timeframe Ranking, Session Analysis, Trades — alag alag sheets mein.
- `export_pdf()` — ek readable PDF report banata hai (reportlab use karke) jisme overview, coin ranking, timeframe ranking, session analysis tables hote hain.

reports.py

- `quick_batch_summary()` — list views (jaise Backtest History page) ke liye sasta/fast summary deta hai bina heavy ranking/session-analysis recompute kiye.
- `_aggregate_by()` — symbol ya timeframe ke hisaab se average profit % nikalta hai, best/worst coin/timeframe pick karne ke liye.
- `_rank_by()` — coin ya timeframe ka poora ranked list banata hai (win rate, avg profit, drawdown, profit factor sab ke sath) — best pehle.
- `_session_analysis()` — har trade ko uske entry time ke UTC session (Asian/London/NY) mein group karke session-wise performance dikhata hai.
- `generate_report()` — poora batch ka comprehensive summary banata hai (total trades, win rate, profit factor, best/worst coin/timeframe/session, lessons stats) aur JSON + text file dono mein save karta hai.
- `_render_text()` — summary dictionary ko readable plain-text report format mein convert karta hai.

sanity_check.py

- `run_sanity_check()` — pura 50-coin backtest chalane se pehle, sirf 2 coins aur 14 din ka chota sample chala kar check karta hai ke agar 0 trades aayen AND koi entry condition kabhi bhi true na hui ho, to ye "structurally broken" strategy hai — is case mein pura backtest rok deta hai taake time waste na ho. Agar sab conditions individually fire hoti hain lekin kabhi ek sath nahi, to ye ek genuinely strict strategy hai, bug nahi — is case mein full run allow karta hai.

diagnostics.py

- `_describe_condition()` — ek condition object ko readable English text mein describe karta hai (jaise "bullish bos (within 10 bars)").
- `condition_hit_report()` — jab backtest 0 trades de, to har entry/confirmation condition individually kitni baar true hui (aur sab ek sath kitni baar true hui) uska count nikal kar diagnostic report banata hai — real engine ka exact evaluator (`ConfiguredStrategy._eval`) reuse karta hai taake report kabhi bhi real engine se disagree na kare.
- `plain_language_diagnosis()` — `condition_hit_report` ke data ko ek simple English sentence mein convert karta hai jo bataye ke 0 trades kyun aaye — kaunsi condition kabhi fire hi nahi hui, ya sab fire hui lekin kabhi ek sath nahi.

metrics.py

- `_max_drawdown_pct()` — equity curve se maximum drawdown percentage calculate karta hai (peak se sabse bada gira hua point).
- `compute_metrics()` — poora set of trading metrics banata hai: total trades, win rate, profit %, avg win/loss, gross profit/loss, net profit, expectancy, profit factor, risk:reward, aur total commission/slippage/spread costs.

strategy_verifier.py

- `describe_condition()` — ek condition ko uske type ke hisaab se readable label deta hai (verification report mein use hota hai).
- `install_coverage_trace()` — strategy ke andar ek hook laga deta hai jo track karta hai ke har condition kitni baar evaluate hui aur kitni baar true nikli — real backtest chalate waqt live record karta hai.
- `verify_rule_coverage()` — config ke andar har saved rule ko coverage counts ke sath cross-check karta hai aur batata hai ke wo rule SKIPPED (kabhi reach hi nahi hui), NEVER_TRUE (evaluate hui lekin kabhi true nahi hui), ya OK (genuinely use hui) — ye prove karta hai ke saved JSON ka har rule Rule Engine tak pahuncha ya nahi.

statistics_verifier.py

- `_max_drawdown_pct()` — equity curve se drawdown nikalta hai (metrics.py wali hi copy, independently).
- `verify_statistics()` — trades aur equity curve se win rate, profit factor, net profit, max drawdown ko independently dobara calculate karke reported metrics ke sath compare karta hai — agar mismatch mile to flag karta hai; ye ek "dusra checker" hai jo metrics.compute_metrics() ko kabhi call nahi karta, taake apna hi bug dono jagah repeat na ho.

engine_health_report.py

- `_now()` — current UTC timestamp ISO format mein deta hai.
- `_categorize_trade_issue()` — ek trade validation issue ko uske text ke basis par PnL/Execution/Trade categories mein classify karta hai, report organize karne ke liye.
- `run_engine_health_report()` — sabse top-level orchestrator hai jo Strategy Verification, Data Verification, Execution Verification, PnL Verification, Trade Verification, aur Statistics Verification — sab ko ek sath chalata hai aur ek overall PASS/FAIL verdict deta hai. Koi bhi section jo genuinely nahi chal saka (jaise missing raw data) usko "not_run" bolta hai, kabhi silently PASS nahi maanta.

verification_engine.py

- `_now()` — current UTC timestamp helper.
- `run_verification()` — ek real, fully-instrumented backtest chalata hai jisme safety check, rule coverage tracing, aur debug log (strategy_loaded -> rule_loaded -> trade_open -> trade_closed -> pnl -> verification) sab شامل hain, aur ek final PASS/FAIL verdict deta hai — Strategy Safety Check bhi is path mein zaroori tor par lagta hai taake koi bhi strategy bina check ke backtest engine tak na pahunchke.

mtf_worker.py

- `_relay_loop()` / `_ensure_relay()` / `_emit()` — worker process se progress updates ko safely, bina simulation ko block kiye, main process tak relay karne ka mechanism — agar relay queue slow ho jaye to messages drop ho jate hain lekin simulation kabhi ruk nahi ti.
- `_failure()` — koi bhi backtest failure ko ek structured dictionary mein deta hai jisme stage, function, reason, suggested_fix, file, line, stack_trace sab شامل hote hain — kabhi bhi raw traceback CEO tak nahi pahunchta.
- `run_one_symbol()` — ek symbol ka poora multi-timeframe backtest chalata hai: safety check -> timeframes detect -> data load -> indicators prepare -> rules load -> bar-by-bar simulate -> results generate, har stage par progress emit karte hue; har exception ko gracefully catch karke structured failure return karta hai (kabhi crash nahi hota).

strategy_safety_check.py

- `_cond_signature()` — ek condition ko uske actual meaning (indicator/concept/params/threshold) ke basis par ek comparable key banata hai, duplicate detect karne ke liye.
- `_describe()` — condition ko readable string mein describe karta hai.
- `_entry_buckets()` — strategy ke sab entry-condition buckets (entry/long/short/rule_groups) ko ek jagah list karta hai.
- `check_duplicate_entry_exit_clauses()` (CHECK 1) — agar exit condition entry condition se hubahu match kare, to flag karta hai — kyunki aisi exit gate entry hote hi already satisfied ho jati hai (real bug jo ek strategy ka account commission se zero kar chuka tha).
- `check_exit_gives_realistic_room()` (CHECK 2) — agar poora exit sirf slow-moving price-level concepts (support/pdh/poc waghera) par based ho, ya entry ke sath same signal ho different threshold par, to flag karta hai — trade ko stop/target tak pahunchne ka mauka hi nahi milta.
- `check_contradictory_entry_gates()` (CHECK 3) — agar entry mein ek sath "pdh se upar" AND "pdl se neeche" required ho (jo kabhi possible nahi, kyunki pdh hamesha pdl se bada hota hai), to impossible combination flag karta hai.
- `check_dead_entry_buckets()` (CHECK 4) — agar entry_rule_groups populated ho lekin purane entry_conditions/long_entry_conditions/short_entry_conditions bhi bhare hon, to unko dead/unreachable rules bolta hai kyunki engine sirf rule_groups ko priority deta hai.
- `run_safety_check()` — upar ke sab 4 checks ko chala kar ek combined "ready"/"needs_review" verdict deta hai — koi bhi strategy backtest engine tak pahunchne se pehle isse guzarni zaroori hai.

trade_validator.py

- `_bar_at()` — kisi millisecond timestamp ke nazdeek wala candle bar dhoondta hai dataframe mein.
- `_pct_diff()` — do numbers ke beech percentage difference nikalta hai.
- `validate_trade()` — ek single trade ka pura independent cross-check karta hai: direction valid hai, position size positive hai, stop-loss/take-profit sahi side par hain, exit price stop/target se match karta hai, win/loss exit_reason se consistent hai, PnL sign price-movement se match karta hai, PnL arithmetic (gross - commission = net) sahi hai, risk:reward plausible hai, aur fill prices actual candle range ke andar hain — koi bhi mismatch hone par specific issue string return karta hai.
- `validate_all_trades()` — poore batch ke sab trades par `validate_trade()` chalata hai plus duplicate trades detect karta hai, overall pass/fail deta hai.

monte_carlo.py

- `run_monte_carlo()` — ek complete backtest ke real trades ko baar baar random order mein shuffle karke (default 1000 baar) dekhta hai ke final equity kitni waari alag alag nikalti hai — isme ek "risk of ruin" rule bhi hai jo agar drawdown 50% se zyada ho jaye to us simulated run ko rok deta hai, jisse order genuinely matter karta hai. Percentile (p5/median/p95), profitable-outcomes %, aur risk-of-ruin % return karta hai — batata hai ke koi strategy genuinely robust hai ya sirf lucky trade-order ki wajah se acchi lagi thi.

stress_test.py

- `find_worst_week()` — symbol ki पूरी history mein se sabse zyada price-movement wala 7-din ka real hafta dhoondta hai (synthetic crash nahi, actual data se).
- `run_stress_test()` — strategy ko us sabse kharab historical hafte ke against dobara chalata hai, dekhne ke liye ke strategy real extreme conditions mein kaisi perform karti hai.

runner.py

- `new_batch_id()` — ek unique batch ID banata hai (timestamp + random hex).
- `_now_iso()` — current UTC time ISO format mein.
- `_DirectCallbackQueue` — sequential (non-multiprocessing) run ke liye ek queue-jaisa adapter jo direct callback ko forward karta hai — taake worker code multiprocessing aur sequential dono mode mein same rahe.
- `run_batch()` — ek purani-style (per symbol+timeframe combo) strategy ko sab symbols/timeframes ke against chalata hai, resume-capable (already-completed combos skip kar deta hai).
- `run_mtf_batch()` — main entry point jo ek multi-timeframe strategy ko multiple coins ke against chalata hai — multiprocessing (max 4 workers) support karta hai speed ke liye, agar process pool crash ho jaye to automatically sequential mode mein switch ho jata hai, live progress/ETA report karta hai, aur resume-capable hai (already-completed coins dubara nahi chalata).

performance_dashboard.py

- `find_last_completed_batch()` — strategy ke naam se uska sabse recent completed backtest batch dhoondta hai.
- `_pooled_batch_metrics()` — batch ke sab symbols ke trades ko pool karke total trade count, dollar expectancy, aur average profit factor nikalta hai (per-symbol average ka average nahi, taake chote-sample symbols overall number ko skew na karein).
- `_check_expectancy()` / `_check_profit_factor()` / `_check_trade_count()` / `_check_walk_forward()` — 4 alag alag pass/fail checks: expectancy > 0, profit factor >= 1.3, trade count >= 100, aur Walk-Forward Test == PASS.
- `evaluate_strategy_performance()` — in 4 factors ko combine karke ek simple GREEN ("Aage Badhao") ya RED ("Abhi Ready Nahi") verdict deta hai — agar koi bhi factor fail ho ya data available na ho to RED, aur exact wajah bhi batata hai.

wizard.py

- `known_concept_catalog()` — engine ke sab real, currently-supported concepts aur indicators ki list deta hai, taake Wizard UI hamesha up-to-date rahe.
- `_condition_from_spec()` — Wizard ke ek form-based condition input ko ek structured Condition object mein convert karta hai — agar user ka input kisi known concept se match na kare to usko "raw"/manual_review mark kar deta hai, kabhi guess nahi karta.
- `_collect_indicators()` — condition specs se zaroori indicators (jaise EMA with period) ko collect karta hai taake wo actually compute ho sakein.
- `_collect_concepts_used()` — sab conditions se used concepts/indicators ki list banata hai.
- `mirror_condition()` / `mirror_conditions()` — ek condition ka direction/operator opposite side mein flip karta hai ("Both — mirror rules" wale mode ke liye), manual_review conditions ko unchanged copy karta hai.
- `_build_sl()` / `_build_tp()` — Wizard ke stop-loss/take-profit selection ko engine-executable SLTPSpec mein convert karta hai.
- `_sl_tp_manual_review()` — agar SL/TP type engine support nahi karta (jaise "Fixed Points"), to usko manual-review item bana deta hai instead of silently ignore karne ke.
- `build_strategy_config()` — Wizard ka main entry point — poora wizard_data form input leke ek complete StrategyConfig banata hai (direction mode, entry/exit conditions, filters, SL/TP, risk) plus ek "trust report" jo batata hai kitna % genuinely user-set hai aur kitna manual-review mein hai.
- `has_manual_review()` / `list_manual_review_conditions()` — check karta hai ke strategy mein koi unresolved manual-review condition hai ya nahi — aisi strategy ko live run karne se roka jata hai.

claim_validation.py

- `compare_claim_to_backtest()` — strategy ke original document mein diya gaya performance claim (jaise "60% win rate") aur uska real measured backtest result compare karta hai — agar dono mein 15+ percentage points ka farq ho to "diverges" flag karta hai, aur sample size chota ho to usko honestly "not reliable" bhi bolta hai.

strategy_parser.py

- `_is_bare_timeframe_fragment()` — check karta hai ke koi text segment sirf ek timeframe mention hai (jaise "on the 1-minute chart") aur usme koi real rule content nahi hai.
- `_detect_role()` — line ke start mein role label (Entry:/Bias:/Trend:) detect karta hai.
- `_detect_condition_role()` — ek rule line ke andar inline timeframe hint dhoondta hai (jaise "on the 1H chart") taake us condition ko sahi timeframe role assign ho sake.
- `_detect_section_header()` — line mein rule-section keyword (Entry Rules:, Exit Rules:, etc.) dhoondta hai.
- `_is_pure_timeframe_remainder()` — batata hai ke colon ke baad sirf ek timeframe hai, koi rule text nahi.
- `_detect_direction()` — text mein bullish/bearish direction words dhoondta hai.
- `_is_bare_heading()` / `_is_meaningful_bare_heading()` / `_strip_scaffolding_line()` — document ke andar se pure scaffolding (jaise "RISK MANAGEMENT:", "Source:" citations, "STEP N --" headings) ko rule-parsing se pehle hata deta hai taake wo fake "unclear rule" na ban jaye.
- `_reassemble_lines()` — ek numbered/bulleted rule jo document ke line-wrapping ki wajah se multiple lines mein toot gaya ho, usko wapas ek statement mein jodta hai.
- `_parse_conditions_from_line()` — ek rule line ko atomic Condition objects mein todta hai — indicator-vs-indicator, price-vs-indicator, indicator-compare, session, concept keywords, trend_filter, sab detect karta hai; jo match na ho wo "raw" ban jata hai.
- `parse_conditions()` — Knowledge Engine ke liye public entry point jo isi line-parser ko reuse karta hai lessons ke free-text description parse karne ke liye.
- `repair_condition_roles()` — pehle se saved strategy mein agar concept condition ka role missing ho, to raw_text se dobara scan karke role fill karne ki koshish karta hai (kabhi overwrite nahi karta already-set role ko).
- `_match_stop_loss()` / `_match_take_profit()` — text se stop-loss/take-profit ka type aur value detect karta hai (fixed %, ATR multiple, structure-based, signal-candle, ya level-based).
- `_match_rr()` — text mein risk:reward ratio ko kai alag formats mein (1:3, "1 to 3", "RR 4", "4R", etc.) detect karta hai.
- `_detect_lookback()` — text mein "within N bars" jaisa lookback window detect karta hai.
- `parse_strategy_text()` — main parser entry point — poora strategy document text leke line-by-line scan karta hai aur ek complete StrategyConfig banata hai (timeframes, entry/exit rules, SL/TP, risk %, session filter, concepts_used sab).
- `_ensure_indicators_for_conditions()` — agar koi condition kisi indicator ko reference kare jo abhi tak config.indicators mein register nahi hua, to usko automatically add kar deta hai taake uska column zaroor compute ho.
- `_fill_missing_and_warnings()` — missing fields (entry timeframe, entry rules, stop loss, etc.) aur unclear conditions ko list karta hai taake validation/clarification flow use kar sake.

strategy_config.py

- Condition dataclass — ek single testable rule ko represent karta hai (concept, indicator_compare, price_compare, indicator_vs_indicator, session, trend, candle_range_pct, candle_body_pct, raw), plus manual_review aur exit_direction jaisi extra fields.
- Condition.is_unclear() — batata hai ke ye condition genuinely "unclear/needs clarification" hai ya already resolved manual-review item hai.
- SLTPSpec dataclass — stop-loss/take-profit ka type (fixed_pct, atr_multiple, structure, rr, level, signal_candle) aur value store karta hai.
- StrategyConfig dataclass — पूरी strategy ka complete schema hai: timeframes, indicators, entry/exit/confirmation conditions, long/short conditions, entry_rule_groups (branching entries), SL/TP, risk %, discard filters, trade-execution settings (entry_type, partial TP, trailing stop, time exit), tags, claimed win rate — ye woh structure hai jo pura engine directly execute karta hai.

- `StrategyConfig.to_dict()` / `from_dict()` — `StrategyConfig` ko JSON-serializable dictionary mein convert karta hai aur wapas load karta hai (disk par save/load karne ke liye).

cost_model.py

- `real_round_trip_cost_pct()` — platform ke real commission+slippage settings se ek round-trip transaction cost calculate karta hai.
- `check_buffer_safety()` — kisi stop-loss buffer % ko real transaction cost ke against check karta hai — agar buffer bahut chota ho (2x cost se kam) to warn karta hai ke stop sirf commission/slippage noise se hi hit ho sakta hai, real price move ki wajah se nahi.

lifecycle_checkpoint.py

- `load()` — kisi bade sequential batch job ka checkpoint disk se load karta hai, ya naya bana deta hai agar exist nahi karta.
- `read_only()` — checkpoint ko sirf display ke liye padhta hai, naya nahi banata.
- `save()` — checkpoint ko atomically disk par likhta hai (temp file phir rename) taake crash ke waqt corrupt file na bane.
- `mark_in_progress()` / `mark_done()` / `mark_skipped()` — ek item ka status update karta hai.
- `pending()` — abhi tak jo items complete nahi hue unki list deta hai (resume ke liye).
- `summary()` — checkpoint ka overall progress summary (total/done/skipped/pending) deta hai.

engine.py

- `_ts_to_ms()` — pandas timestamp ko milliseconds mein convert karta hai.
- `_apply_slippage()` / `_apply_spread()` — trader ke against hamesha worse fill price simulate karta hai (real-world execution friction).
- `_position_size()` — risk-based position sizing calculate karta hai (agar stop-loss diya ho) ya fixed % sizing (agar na ho), leverage cap ke sath.
- `_risk_amount()` / `_reward_amount()` — position ke risk/reward dollar amount nikalta hai.
- `_check_forced_exit()` — dekhta hai ke is bar par stop-loss ya take-profit hit hua ya nahi.
- `_check_partial_take_profit()` — configured trigger_rr par position ka ek hissa close karta hai (partial take-profit feature).
- `_update_trailing_stop()` — trailing stop ko update karta hai (percent-based, ATR-based, ya structure-based) — kabhi stop ko loosen nahi karta, sirf tighten karta hai.
- `_pending_trigger_price()` — limit/stop/signal-candle pending order ka trigger price resolve karta hai.
- `_check_pending_fill()` — check karta hai ke koi pending order is bar par fill hui ya nahi (limit/stop/next-candle-open/signal-candle types).
- `run_backtest()` — poora bar-by-bar backtest simulation engine — signal generate karta hai, position open/close karta hai, commission/slippage/spread apply karta hai, partial TP/trailing stop/time-exit/breakeven manage karta hai, daily-loss-limit aur max-drawdown circuit breakers implement karta hai, aur end mein trades, equity curve, final balance return karta hai. Ye pura codebase ka core simulation loop hai.
- `_close()` (nested in `run_backtest`) — ek position (full ya partial) close karne ka shared path — gross/net PnL, commission, slippage/spread cost sab yahin compute hote hain taake ye teeno exit paths (full close, partial close, forced end-of-data close) kabhi ek dusre se drift na karein.
- `_open_position()` (nested in `run_backtest`) — ek naya position open karne ka shared path — slippage/spread apply karta hai, wrong-side stop-loss/take-profit ko discard karta hai, missing stop-loss ke liye emergency 1% fallback lagata hai, aur position dictionary return karta hai.

concepts.py

- `ema()`, `sma()`, `rsi()`, `macd()`, `atr()` — standard technical indicators (Exponential/Simple Moving Average, RSI, MACD, Average True Range).
- `macd_signal_crossover()` / `macd_zero_crossover()` — MACD line ka signal-line ya zero-line ke sath edge-triggered crossover detect karta hai.

- `rolling_high()` / `rolling_low()` — pichle N bars ka highest/lowest value, causal (current bar exclude karke).
- `vwap_daily()` — har UTC din par reset hone wala Volume Weighted Average Price.
- `volume_filter()` — volume spike detect karta hai (average se multiplier guna zyada).
- `trend_filter()` — close vs EMA ke basis par simple "up"/"down" trend.
- `session_of_hour()` / `session_column()` — UTC hour se trading session (Asian/London/NY) determine karta hai.
- `session_high_low()` / `previous_session_high_low()` — current ya pichle closed session ka high/low track karta hai.
- `session_sweep_reclaim()` — session range ke bahar wick jaane ke baad wapas andar close hone ka pattern.
- `session_open_price()` — current session ka opening price.
- `day_of_week_column()` — har bar ka calendar weekday name.
- `swing_points()` — confirmed swing high/low pivots (lookback bars ke baad confirm hote hain).
- `support_resistance()` — swing points se support/resistance levels bana kar forward-fill karta hai.
- `break_of_structure()` — bullish/bearish BOS (close pichle swing high/low se aage nikal jaye).
- `change_of_character()` — CHoCH — opposite direction mein BOS jo trend reversal ka pehla sign hota hai.
- `fair_value_gap()` — 3-candle imbalance/gap detect karta hai.
- `order_blocks()` — BOS se pehle wala opposite-direction candle, "smart money zone".
- `consolidation_impulse_zones()` — ek tight consolidation range ke baad sharp impulse move — demand/supply zone define karta hai.
- `valid_structure_trend()` — ek stricter, stateful trend-tracking rule jisme swing low/high tabhi "valid" mante hain jab opposite level break ho.
- `liquidity_sweep()` — swing support/resistance ke bahar wick jaake wapas andar close hona (stop hunt pattern).
- `sequential_event()` — do events ke beech genuine ordering check karta hai (event A pehle, phir event B) — sirf co-occurrence nahi.
- `sweep_invalidation_state()` — ek setup active rehta hai jab tak opposite event use invalidate na kare.
- `level_sweep_reclaim()` — sweep aur reclaim ke beech LATER bar par bhi allow karta hai (same-bar zaroori nahi).
- `ema_no_touch_trigger()` — candle jo EMA line ko bilkul touch nahi karti (bilkul upar ya bilkul neeche).
- `reaction_at_level()` — external level ke bahar wick jaake close wapas andar hona.
- `next_prior_swing_level()` — current active level se ek level peeche wala swing point.
- `move_origin_target()` — jis opposing structural level se move shuru hua tha, wo find karta hai.
- `level_touch()` — external level ko wick se touch karna (bina reaction ke).
- `within_level_zone()` — ATR-scaled buffer band ke andar price hona check karta hai.
- `first_signal_per_level()` — ek specific level par sirf pehla signal count karta hai, dobara nahi.
- `candle_break()` — ek reference candle (bullish/bearish) ke high/low ko baad ki candle todti hai — edge-triggered.
- `previous_day_high_low()` — pichle UTC din ka high/low (PDH/PDL).
- `pdh_pdl_sweep()` — PDH/PDL ke bahar jaake wapas andar close hone ka classic liquidity-grab pattern.
- `fvg_zone()` — active FVG zone ke exact price bounds track karta hai.
- `true_within_lookback()` — ek boolean event pichle N bars mein kabhi true hua ya nahi.
- `volume_profile_previous_day()` — pichle poore din ka volume-at-price profile (POC/VAH/VAL).
- `volume_nodes_previous_day()` — Low/High Volume Nodes (LVN/HVN) pichle din ke profile se.
- `aggression()` — strong-bodied candle + volume spike ka combination (directional conviction).
- `breaker_blocks()` — order block jo price ne baad mein todh diya ho, wo polarity flip karke breaker ban jata hai.
- `mitigation_blocks()` — order_blocks jaisa hi lekin candle body (wick nahi) use karta hai.
- `imbalance()` — ek single candle ka body normal se bahut bada hona (inefficiency).
- `equal_highs_lows()` — do swing highs/lows ek dusre ke bahut kareeb hona (liquidity pool).
- `premium_discount_zone()` — swing range ke upar-adha "premium", neeche-adha "discount" zone.
- `rejection_blocks()` — long wick ke sath swing point par sharp rejection zone.

- `cumulative_volume_delta()` — CVD approximation, har candle ka volume close direction ke hisaab se +/- karke cumulative sum.
- `_first_window_range()` / `_first_window_range_tz()` — din ke pehle N minutes ka high/low range (Opening Range/Initial Balance ke liye base).
- `four_hour_range()` / `four_hour_range_breakout()` — din ki pehli 4-hour candle ka range aur uska breakout.
- `range_reentry_event()` — range ke bahar se wapas andar close hone ka edge-triggered event.
- `opening_range()` / `opening_range_breakout()` — Opening Range aur uska breakout.
- `initial_balance()` / `initial_balance_extension()` — Initial Balance range aur uske bahar trading hona.
- `anchored_vwap()` — VWAP jo latest confirmed swing point se anchor hoti hai (calendar reset ke bajaye).
- `kill_zone_column()` / `in_kill_zone()` — ICT-style London/NY kill zones detect karta hai.
- `doji_pattern()` — chota body wali candle (Doji).
- `morning_evening_star()` — 3-candle Morning/Evening Star reversal pattern.
- `candle_pattern_confirmation()` — ek pattern candle ke baad koi later candle uske high/low se aage close ho, tab confirm hota hai.
- `engulfing_candle()` — bullish/bearish engulfing pattern.
- `pin_bar()` — chota body + lambi wick wali rejection candle.
- `inside_bar()` — current candle jo pichle candle ke range ke andar ho.
- `breakeven_stop()` — `trigger_rr` reach hone par stop-loss ko entry price (breakeven) par move karta hai.
- `trend_regime()` — 3-way trend classification (up/down/sideways) EMA aur ATR ke basis par.
- `order_block_validity()` — kisi order block ka abhi bhi valid hona ya price ne todh diya (invalidate ho gaya).
- `trendline_breakout()` — do most-recent swing points ko jod kar trendline draw karta hai aur uska break detect karta hai.
- `range_breakout_volume_confirm()` — consolidation range ke breakout ko volume spike se confirm karta hai, badi candle par retest wait karta hai.
- `heikin_ashi()` — Heikin Ashi smoothed candles calculate karta hai.
- `fibonacci_retracement_zone()` — swing leg se Fibonacci retracement levels (38.2/50/61.8%).
- `fixed_range_volume_profile()` — swing-to-swing leg ka volume profile POC.
- `frvp_market_shape()` — Value Area, HVN/LVN zones, aur market-shape classification (P/B/D/thin/capital-B).
- `long_wick_candle()` — candle jiski ek wick body se bahut lambi ho (liquidity sweep signal).
- `nth_touch_of_level()` — ek level ko Nth baar touch hone tak True nahi hota.
- `ichimoku_cloud()` — standard Ichimoku Kinko Hyo indicator (Conversion/Base line, cloud spans, lagging span comparison).
- `lwti()` — Linear Weighted Trend Indicator, custom momentum oscillator.
- Dozens of per-strategy composite helper functions bhi hain (jaise range/session/day helpers) jo upar list ki gayi core primitives ko combine karte hain — ye sab causal (zero look-ahead) aur vectorized hain.

validator.py

- `known_indicator_names()` / `parameterized_indicator_names()` — engine ke sab known concepts/indicators ki public list deta hai.
- `normalize_timeframes()` — friendly timeframe names (jaise "daily") ko engine-supported exact strings ("1d") mein convert karta hai.
- `_numeric_bound()` — indicator_compare condition se pata karta hai ke wo upper bound hai ya lower bound.
- `validate()` — poori StrategyConfig ko validate karta hai: missing entry timeframe, unsupported interval, structure-SL bina proper concept ke, missing entry/exit rules, unclear conditions, missing SL/TP, invalid risk %, invalid entry_type, partial-TP/trailing-stop/time-exit ki sanity, invalid indicators/concepts, role-declared-lekin-timeframe-missing, concept used lekin concepts_used mein nahi, contradictory bullish+bearish same concept, aur numeric-threshold contradictions (jaise $RSI < 30$ AND $RSI > 70$ ek sath) — sab kuch human-readable error strings ki list return karta hai.
- `is_valid()` — `validate()` ka boolean shortcut.

result_plausibility.py

- `check_batch_plausibility()` — ek completed backtest ke numbers implausible to nahi (extreme win rate, absurd profit %, ya bahut zyada trades-per-coin) check karta hai — bug ka signal ho sakta hai, skill ka nahi.
- `sweep_recently_completed_batches()` — recent batches ko scan karke pehli baar flag hone par ek dashboard alert create karta hai (dobara alert nahi karta usi batch ke liye).
- `start_plausibility_sweep_thread()` — server startup par hourly background thread chalata hai jo naye completed batches ko automatically check karta rahe.

slippage_sensitivity.py

- `run_slippage_sensitivity_test()` — ek completed batch ke recorded trades par extra slippage levels (0% se 1% tak) apply karke dekhta hai ke strategy ka PnL kitni jaldi break ho jata hai — asli simulation dobara nahi chalata, sirf existing trades par extra cost lagata hai.

duration_tracker.py

- `compute_duration_stats()` — recent completed batches ke start/end timestamps se unki duration nikalta hai aur average/total/slowest-5 stats deta hai — batata hai backtests kitna time le rahe hain.

what_if_simulator.py

- `_apply_parameter_changes()` — strategy config ka ek clone banata hai jisme kuch parameters change kiye gaye hon.
- `run_what_if()` — ek strategy ke kisi ek parameter ko change karke usi real historical data (last 30 din, max 5 symbols) par dobara chala kar original vs modified result compare karta hai — genuine counterfactual re-simulation.

feature_importance.py

- `_condition_label()` — ek condition ka short readable label deta hai.
- `_net_profit()` — multiple symbols par ek config chala kar total net profit nikalta hai.
- `rank_feature_importance()` — strategy ke har entry/confirmation condition ko ek-ek karke hata kar dobara backtest chalata hai (leave-one-out ablation) — jo condition hatane se profit sabse zyada girta hai, wahi sabse important condition hai.

cross_coin_validation.py

- `_volatility_tercile_groups()` — coins ko unki volatility ke basis par low/medium/high groups mein baant deta hai (real-time score se, hardcoded list se nahi).
- `validate_across_coin_groups()` — check karta hai ke strategy ka win rate alag alag volatility groups mein consistent hai ya sirf ek group (overfit) mein hi achha hai — agar best/worst group ka win-rate farq 20 points se kam ho to "consistent" bolta hai.

strategy_variants.py

- `_concept_condition_indices()` — strategy ke concept-type entry conditions ke indices dhoondta hai.
- `generate_variants()` — ek existing strategy ka ek entry condition ko usi DNA-category ke doosre concept se swap karke multiple sibling variants banata hai.
- `test_variants()` — har variant ko real data (fast window) par chala kar baseline se compare karta hai, sabse behtar improvement wale variant ko highlight karta hai.

configured_strategy.py

- `ConfiguredStrategy.__init__` — ek StrategyConfig ko wrap karta hai aur optional trace hook (verification ke liye) setup karta hai.
- `prepare_context()` — har role (bias/trend/entry/etc.) ke apne native-resolution frame par indicators (EMA/SMA/RSI/ATR/VWAP/MACD/anchored-VWAP/CVD/highest-high/lowest-low) compute karta hai, phir concept columns bhi har relevant role par, aur end mein sab ko entry-timeframe par merge karta hai — ye function bahut saari per-strategy special-case logic bhi hold karta hai (Dumb Money Concepts, PDH/PDL MTF

Reversal, Fractal Sweep, CISD Entry, HTF Key Level Engulfing, Liquidity Sweep batches, Trendline Breakout, HTF-LTF Confluence, Heikin Ashi Reversal, Donchian/LWTI Confluence, waghera — har ek apna alag pre-merge zone/trend column compute karta hai).

- `_condition_roles()` — strategy ke sab conditions mein se kaunse-kaunse non-default timeframe roles use ho rahe hain, unki list deta hai.
- `_compute_concept_columns()` — used concepts ke basis par asal boolean/price columns compute karta hai (BOS, CHoCH, FVG, order blocks, support/resistance, liquidity sweep, candle patterns, session zones, volume profile, waghera) — ye method 50+ alag-alag named per-strategy composite concepts (jaise `htf_key_level_engulfing`, `sniper_headshot_entry`, `pdhl_mtf_reversal`, `cisd_entry`, `fractal_sweep_reversal`, `mss_reversal`, `order_block_reversal`, `eqhl_reversal`, `fvg_reversal`, `laxman_trigger`, `trendline_breakout`, `range_breakout_volume_confirm`, `fibonacci_golden_zone`, `frvp_poc_reversal`/`frvp_market_shape` variants, `sr_liquidity_sweep_sideways`, `fvg_momentum_pullback`, `fvg_pure_inverse`, `order_block_trading` loose/strict, `crt` loose/strict, `bos_choch_retest`, `ichimoku_system`) ko bhi individually implement karta hai — har ek apni specific entry/SL logic banata hai using existing `concepts.py` primitives ko combine karke.
- `_compute_role_zone_columns()` — HTF Key Level Engulfing strategy ke liye Order Block/FVG/Liquidity Sweep zones ek role ke raw frame par compute karta hai.
- `prepare()` — engine ke liye plain open/high/low/close/volume columns alias karta hai, aur wo composite concepts jo POST-merge hi compute ho sakte hain (kyunki unko do alag roles ka data ek sath chahiye) yahan implement karta hai — Dumb Money Concepts variants, HTF Key Level Engulfing, PDH/PDL MTF Reversal, CISD Entry, Fractal Sweep Reversal, Liquidity Sweep Multi-Confirm/CISD-Swing, Liquidity Sweep Engulfing, OTE Liquidity Sweep Reversal, Trendline Breakout HTF-gate, HTF-LTF FVG/OB Confluence, Heikin Ashi Reversal, Donchian/LWTI/Volume Confluence — har ek apna khud ka multi-role sequential logic hai.
- `_array()` — dataframe ke column ko numpy array mein cache karta hai (performance ke liye, pandas Series slicing se bachne ke liye).
- `on_bar()` — har bar par decide karta hai ke entry lena hai ya exit — `entry_rule_groups` (branching entries) sabse pehle check hota hai, phir long/short separate rules, phir legacy `entry_conditions`; `confirmation_conditions`, session filter, aur trend filter bhi gate karte hain; stop-loss/take-profit compute karta hai; pre-trade discard filters (SL distance %, min risk:reward) apply karta hai; exit ke liye direction-aware `exit_conditions` check karta hai.
- `_eval_long_short()` — long aur short entry rule sets ko independently evaluate karta hai, agar dono ek sath true ho jayen (contradictory bar) to koi signal nahi deta.
- `_eval_rule_groups()` — branching `entry_rule_groups` mein se pehla matching group (declared order mein) dhoondta hai, contradictory bar ko bhi handle karta hai.
- `manage_position()` — `breakeven_at_rr` configured ho to, unrealized profit trigger reach hone par stop-loss ko entry price par move karta hai.
- `_infer_direction()` — legacy `entry_conditions` se direction infer karta hai (agar explicit direction diya ho).
- `_indicator_column()` — kisi indicator (EMA/RSI/MACD/etc.) ka sahi column-name resolve karta hai, role aur period ke basis par.
- `_get()` — dataframe se ek value nikalta hai (NaN ko None mein convert karke).
- `_eval_traced()` — `_eval()` ko wrap karta hai taake verification trace record ho sake bina real evaluation logic change kiye.
- `_eval()` — main condition-evaluator — `concept`, `indicator_compare`, `price_compare`, `indicator_vs_indicator`, `session`, `trend`, `candle_range_pct`, `candle_body_pct` — sab condition types ko dispatch karta hai; concept type ke andar dozens ke individual concept-name handlers hain (`bos`, `choch`, `fvg`, `support/resistance`, `pdh/pdl`, `poc/value_area`, `lvn/hvn`, `session_high_low`, `demand/supply_zone`, `valid_structure_trend`, aur har per-strategy composite ka apna confirm-column bhi).
- `_compute_stop_loss()` — `fixed_pct`, `atr_multiple`, `structure` (priority-ordered zone search across dozens of strategy-specific SL columns), aur `signal_candle` types ke liye stop-loss price calculate karta hai.
- `_compute_take_profit()` — `fixed_pct`, `rr`, `atr_multiple`, `level`, `structure`, aur `structure_or_rr` types ke liye take-profit price calculate karta hai, including fallback logic jab koi structural target na mile (rolling 100-bar high/low last resort).
- `_compute_primary_target()` — `min_risk_reward_filter` ke liye ek reference price (highest high/lowest low of preceding N bars) nikalta hai.

- `_describe()` — matched conditions ki list ko ek readable "reason" string mein convert karta hai (trade record mein store hota hai).

strategy_library.py

- `_read_meta()` / `_write_meta()` — strategy ke meta.json ko padhta/likhta hai.
- `save_backtest_snapshot()` — strategy ke meta.json mein ek chota backtest performance summary (win rate, profit factor) save karta hai taake cloud dashboard bhi dikha sake (cloud DB mein backtest tables nahi hoti).
- `create()` — nayi strategy ko version 1 ke tor par save karta hai, automatic safety check bhi run karta hai.
- `save_version()` — existing strategy ka naya version save karta hai (edit history), reason bhi record karta hai.
- `recheck_safety()` — kisi strategy ke current version par safety check dobara chalata hai.
- `save_walk_forward_result()` — Walk-Forward Test ka result meta.json mein persist karta hai.
- `get_meta()` — strategy ka catalog metadata deta hai.
- `load()` — kisi strategy ka specific version load karta hai aur poora StrategyConfig return karta hai.
- `list_all()` — library ke sab strategies list karta hai.
- `list_all_including_synced()` — filesystem strategies plus cloud-synced strategies dono ko combine karta hai.
- `load_including_synced()` — pehle cloud-synced version try karta hai, phir filesystem fallback.
- `find_by_name()` — case-insensitive naam se strategy id dhoondta hai (duplicate save rokne ke liye).
- `search()` — query aur tag se strategies search karta hai.
- `find_similarity_warnings()` — Jaccard similarity (concepts_used ke overlap) se poori library mein 80%+ similar strategies dhoondta hai — duplicate-jaisi strategy banane se pehle warn karta hai.
- `rename()` / `set_tags()` / `set_favourite()` / `set_comment()` / `set_archived()` — strategy ke metadata fields update karte hain.
- `duplicate()` — poori strategy folder copy karke naya id deta hai.
- `delete()` — strategy ki poori folder delete karta hai (partial failure hone par silently fail nahi karta, error raise karta hai).
- `set_clarification()` / `get_clarification()` — strategy ke "needs clarification" record ko manage karta hai.
- `version_history()` — strategy ke sab saved versions ki list, unke reasons ke sath.
- `diff_versions()` — do versions ke beech field-level differences dikhata hai.
- `restore_version()` — kisi purane version ko wapas current version banata hai (naya version create karke, purana kabhi delete nahi hota).

strategies

__init__.py

- Empty file — sirf Python package marker.

base.py

- Signal dataclass — ek trading signal ko represent karta hai: action (buy/sell/exit), stop_loss, take_profit, reason, aur optional risk_multiplier (soft position-size adjustment ek entry ke liye).
- `Strategy.prepare()` — default no-op; subclass isme apne indicator columns add karta hai (ek baar per coin/timeframe, bar-loop se pehle).
- `Strategy.on_bar()` — abstract method jisme har bar ka trading decision (buy/sell/exit/none) hota hai — sab custom strategies isko implement karti hain.
- `Strategy.manage_position()` — default no-op; open position ko har bar par modify karne ka hook (jaise breakeven move), stop/take-profit check se theek pehle chalta hai.

data_engine / external_signals

data_engine

__init__.py` and `exchanges/__init__.py

- Dono files khaali hain (empty) — sirf Python package marker ke liye hain, koi logic nahi.

binance_client.py

- `_get(path, params)` — Binance REST API ko GET request bhejta hai, retry/backoff logic ke sath: network error par wait karke retry, rate-limit (429/418) par Retry-After header follow karta hai, aur 5xx server error par bhi retry karta hai. MAX_RETRIES ke baad fail hone par exception raise karta hai.
- `get_ticker_24hr()` — har symbol ka 24-hour ticker data (price, volume, % change) leke aata hai, coins ko volume ke hisaab se rank karne ke liye use hota hai.
- `get_exchange_info()` — Binance ke saare tradeable symbols aur unki trading rules ki poori list laata hai.
- `get_klines(symbol, interval, ...)` — kisi symbol/timeframe ke liye raw candle (OHLCV) data fetch karta hai, time range aur limit ke sath.

coingecko_client.py

- `get_top_market_cap_coins(limit)` — CoinGecko API se market-cap ke hisaab se top coins fetch karta hai, 100-100 ke pages mein paginate karta hai (max 5 pages), har request ke beech politeness ke liye 1.5s wait karta hai.

exchanges/registry.py

- `get_exchange_client(exchange_id)` — kisi exchange (binance/okx/bybit/bitget/gate) ka client object banata/return karta hai aur cache mein rakhta hai taake dobara instantiate na karna pade. Binance ke liye custom BinanceClient, baaki sab ke liye CCXTClient deta hai.
- CCXT_EXCHANGE_IDS / ALL_EXCHANGE_IDS — supported exchanges ki list define karta hai.

control.py

- DownloadControl class — background download loop ko pause/resume/stop karne ka thread-safe switch. stop(), pause(), resume(), reset() states set karte hain; should_stop()/is_paused() check karte hain; wait_if_paused() loop ko block karta hai jab tak pause hataya na jaye — is tarah dashboard se download ko safely control kiya ja sakta hai (kabhi mid-write nahi rukta).

symbols.py

- `_filter_tradeable(raw_tradeable)` — leveraged tokens (UP/DOWN/BULL/BEAR), stablecoins, aur non-crypto assets (gold-backed jaise XAUT) ko trading list se hata deta hai; sirf plain latin-alphanumeric tickers rakhta hai.
- `pick_top_symbols(exchange_client, n, quote)` — CoinGecko market-cap ranking use karke top N asli cryptocurrencies chunta hai jo exchange par bhi trade hoti hain — stablecoins/tokenized-stocks ko volume-based ranking ke bajaye market-cap ranking se avoid karta hai.

logging_setup.py

- `subscribe(callback) / unsubscribe(callback)` — dashboard ki live log window jaise listeners ko register/unregister karta hai taake har log line unhe bhi mile.
- `log(message)` — central logging function: console par print karta hai, data/logs/sindhu.log file mein append karta hai, aur har subscriber callback ko bhi bhejta hai (thread-safe lock ke sath).

downloader.py

- `download_symbol(exchange_client, symbol, log, control)` — ek symbol ke saare missing 1-minute candles ko exchange se download karta hai, jahan pichli baar chhoda tha wahin se resume karta hai; pause/stop control ko har batch ke beech check karta hai taake dashboard se safely rok sake.

- `download_all(exchange_client, symbols, log, control, progress_cb)` — saare symbols ke liye download loop chalata hai, progress callback se dashboard ko "current coin/progress" dikhata hai, aur ek symbol fail hone par bhi baaki symbols ke liye chalta rehta hai (error log karke).

exchanges/base.py

- `ExchangeClient` — common interface/contract jo har exchange client implement karta hai: `get_tradeable_symbols`, `get_ohlcv`, `get_tickers` — taake baaki codebase ko pata na chale ke wo kis exchange se baat kar raha hai.

exchanges/binance.py

- `BinanceClient.get_tradeable_symbols(quote)` — Binance ke saare active, spot-tradeable pairs (given quote asset ke against) return karta hai.
- `BinanceClient.get_ohlcv(...)` — Binance se raw candle data leta hai.
- `BinanceClient.get_tickers(quote)` — Binance ke 24h ticker data ko price/change%/volume ke standard format mein convert karta hai.

data_quality.py

- `check_missing_candles(df, interval)` — candle history mein gaps dhoondta hai; har continuous gap ko ek entry (gap_start, gap_end, missing_count) ke roop mein report karta hai — ek exchange outage ek hi gap dikhata hai, row-by-row nahi.
- `check_duplicate_candles(df)` — duplicate timestamps ki list return karta hai.
- `check_invalid_timestamps(df)` — timestamp index ka order check karta hai (strictly increasing hona chahiye) aur UTC timezone confirm karta hai; agar timezone missing ya wrong hai to issue report karta hai.
- `check_corrupted_ohlc(df)` — impossible OHLC relationships dhoondta hai (jaise high < low, open/close range se bahar, negative/zero price ya volume) aur har bad candle ki wajah (reasons) batata hai.
- `check_resampling(df_1m, df_resampled, interval)` — 1-minute data se resampled candle ko independently dobara compute karke verify karta hai ke resample sahi hua ya nahi (mismatches report karta hai).
- `run_data_quality_report(df, interval, df_1m_for_resample_check)` — upar ke saare checks ko ek report mein combine karta hai aur overall PASS/FAIL verdict deta hai.

resample_cache.py

- `load(exchange, symbol, interval, start_ms, end_ms, source_bounds)` — kisi exact resample request ka cached result disk se load karta hai, lekin sirf tab jab source data (1m candles) us waqt se fresh na huye hon — warna miss treat karta hai.
- `save(exchange, symbol, interval, start_ms, end_ms, df, source_bounds)` — resampled dataframe ko atomically (temp file + rename) disk par save karta hai; Windows par concurrent-write race conditions ko safely swallow karta hai.
- `clear_all()` — poora resample cache manually clear karne ka function (troubleshooting ke liye, normally zaroorat nahi padti kyunke invalidation automatic hai).
- Sharding design: har (exchange, symbol) ka apna subdirectory hota hai taake ek folder mein lakhon files jama na hon (NTFS performance issue se bachne ke liye).

data_quality_score.py

- `score_symbol(exchange, symbol)` — ek symbol ke recent (last 72 hours) candle data ko data_quality checks se guzar kar 0-100 ka health score banata hai: gaps, duplicates, invalid timestamps, corrupted OHLC, aur staleness (last candle kitni purani hai) — har issue score minus karta hai.
- `score_all_tracked_symbols(exchange)` — saare tracked symbols ka health score nikal kar overall average score, kitne symbols mein issues hain, aur per-symbol breakdown deta hai (worst score pehle).

live_candles.py

- `_normalize_row(r)` — kisi exchange se aaye raw kline row ko standard 9-field format mein convert karta hai (extra fields drop karke).

- `_fetch_page(...)` / `_fetch_range(...)` — exchange se paginated candle history fetch karta hai, rate-limit se bachne ke liye har page ke beech chhota delay leta hai.
- `_trim(df, end_ms)` — in-memory cache se purani (25 din se zyada purani) candles hata deta hai taake memory unbounded na badhe.
- `get_ohlcv_live(exchange, symbol, interval, start_ms, end_ms)` — cloud runner ke liye database-free candle source: exchange se seedha (live) candles fetch karta hai, in-memory cache mein rakhta hai, aur sirf naye candles hi refetch karta hai (resume-from-last-candle pattern) — bina local database ke bhi live pipeline chalane deta hai.
- `clear_cache()` — test/debug ke liye poora in-memory candle cache clear karta hai.

paths.py

- `ensure_folders()` — `data/` ke andar zaroori saare subfolders (database, market_data, logs, reports, settings, history, config, external_signals_media) create karta hai.
- `migrate_legacy_files()` — purane flat data/ layout ki files (jaise `sindhu.db`, purane logs) ko naye subfolder-based layout mein move karta hai, sirf agar destination pehle se exist nahi karta.
- `disk_usage_bytes()` — `data/` folder ka total disk usage (bytes mein) calculate karta hai.
- `SINDHU_DATA_DIR` env var support — cloud runner ko alag data directory point karne deta hai bina local data ko chhue.

resample.py

- `_rows_to_df(rows)` — raw kline rows ko UTC-timezone-aware DataFrame mein convert karta hai.
- `_compute(exchange, symbol, interval, start_ms, end_ms)` — 1-minute candles ko range ke hisaab se filter karke phir resample (aggregate: `open=first`, `high=max`, `low=min`, `close=last`, `volume=sum`) karta hai — cache se pehle wahi exact algorithm jo hamesha se tha.
- `get_ohlcv(exchange, symbol, interval, start_ms, end_ms)` — main entry point: agar `SINDHU_LIVE_CANDLES` flag on hai to `live_candles` module ko redirect karta hai; 1m interval ke liye seedha DB se; baaki intervals ke liye resample cache check karke, miss hone par compute+save karta hai.

what_changed.py

- `summarize_period(since_iso, until_iso, limit)` — "What Changed Today" feature: `audit_trail_log` ke saare events ko (entity, action) ke hisaab se group karke plain-language sentences banata hai (jaise "strategy was updated 3 times"), busiest activity pehle — pura automatic, hand-editing zero.

session_handoff.py

- `generate_handoff_summary(since_iso)` — ek narrative "kya hua / aage kya karna hai" summary banata hai: `what_changed` ke counts ko readable prose mein convert karta hai, aur current live state (kill switch active hai kya, drawdown pause hai kya, open incidents, strategy retirement suggestions) ko "what's next" section mein daalta hai.

feature_toggles.py

- `get_toggles()` / `set_toggle(key, value)` — dashboard ke "Feature Control Center" ke on/off switches ko JSON file mein persist karta hai (jaise auto-avoid, drawdown protection, dynamic risk sizing, backups, weekly/monthly reports, self-learning engine, etc.).
- `set_master_pause(value)` / `is_master_paused()` — ek master switch jo sab automated features ko ek saath band kar sakta hai.
- `is_enabled(key)` — check karta hai ke koi feature enabled hai ya nahi (master pause off ho AND uska apna flag on ho tab hi True).

config.py

- `env_flag(name)` — environment variable ko boolean ki tarah parhta hai (1/true/yes/on, case-insensitive, whitespace-tolerant) — deployment mein galat entry se silent bug na ho.

- `load_or_seed(filename, defaults)` — `data/config/` mein JSON settings file na ho to defaults se create karta hai, warna existing file read karta hai — dashboard se settings edit karne ka base pattern.
- `save_config(filename, data)` — settings JSON file ko disk par save karta hai.
- Cloud-mode default exchange logic — `SINDHU_CLOUD_MODE` on hone par Binance ki jagah Bybit ko default banata hai (kyunke Binance Render ke servers ko geo-block karta hai HTTP 451 ke sath), aur purani saved config file ko bhi override karta hai agar wo abhi bhi "binance" bola raha ho.
- `RESAMPLE_RULE` — har timeframe (1m se 1M tak) ka pandas resample rule define karta hai.

exchanges/ccxt_client.py

- `CCXTClient.get_tradeable_symbols(quote)` — ccxt library ke through kisi bhi exchange (okx/bybit/bitget/gate) ke active spot pairs nikalta hai.
- `CCXTClient.get_ohlcv(...)` — ccxt se candle data leta hai aur `close_time/quote_volume/trades` ko approximate karta hai (ccxt in fields ko directly nahi deta).
- `CCXTClient.get_tickers(quote)` — sirf real SPOT symbols ke liye tickers maangta hai (bug fix: pehle bybit par accidentally perpetual futures data aa raha tha kyunke unscoped default request perp market deta hai).
- `_retry(fn, ...)` — network/rate-limit errors par exponential backoff ke sath retry karta hai.

storage.py

- `_SCHEMA` — poori SQLite schema string (40+ tables) jo har feature ke liye tables define karta hai.
- `get_conn()` — connection context-manager: agar `DATABASE_URL` set hai to Postgres use karta hai, warna SQLite (WAL mode, synchronous=NORMAL for speed, busy_timeout=10s, foreign_keys=ON).
- `init_db()` — database schema create karta hai aur saare ek-baar-chalne-wale migrations (`_migrate_*` functions — column additions jaise `display_name`, `paused`, `capital_multiplier`, `MAE/MFE` columns, trade annotation column, lesson meta columns, waghera) chalata hai; Postgres ke liye seedha curated schema banata hai.
- `_migrate_add_exchange_column`, `_migrate_backtest_fk_constraints` — bade structural migrations: purani single-exchange tables ko multi-exchange schema mein convert karna, aur `backtest_results/backtest_trades` par FOREIGN KEY constraint add karna (orphan rows ko pehle safely recover karke).
- Dusre `_migrate_*` functions (compiled_document, ai_dictionary, ai_import_queue/cache, lesson_stats_summary backfill, paper_account_state per-strategy conversion, telegram log explanation/grade columns, waghera) — sab additive-only column upgrades hain, purana data kabhi delete/loss nahi hota.
- `save_symbols`, `load_symbols`, `load_all_exchanges` — symbols list save/load karta hai per exchange.
- `get_progress / set_progress` — har symbol ka download progress track karta hai (resume ke liye).
- `insert_klines`, `count_rows`, `count_all_rows` (fast approximate count via `MAX_ROWID`), `get_klines_range`, `get_symbol_time_bounds`, `db_file_size_bytes` — raw candle data ko insert/read/measure karta hai.
- `register_strategy`, `create_batch`, `get_batch`, `list_recent_batches`, `latest_completed_batch_for_strategy_name`, `update_batch_status`, `set_batch_display_name` — backtest batch lifecycle manage karta hai.
- `save_optimization`, `get_optimization_for_batch`, `list_optimizations` — strategy optimization runs ka record.
- `create_pipeline_job`, `update_pipeline_job`, `get_pipeline_job`, `list_running_pipeline_jobs`, `list_pipeline_jobs` — Automation Pipeline (import→backtest→optimize→compare→paper trading) ka durable checkpoint/resume system, server crash ke baad bhi resume kar sakta hai.
- `get_completed_result_keys`, `save_result`, `save_trades`, `get_trades`, `search_trades` (global trade search, dashboard ka search box), `get_batch_results` — backtest results/trades ka storage aur retrieval.
- `save_lesson`, `get_lesson`, `list_lessons`, `delete_lesson` — user-authored trading lessons ka CRUD.
- `record_lesson_application`, `record_lesson_applications_bulk`, `_bump_lesson_stats_summary` — har baar jab koi lesson kisi trade decision par apply hota hai to log karta hai, aur ek incremental summary table (`lesson_stats_summary`) ko update karta hai taake baar-baar 25 million rows scan na karna pade.
- `get_lesson_stats`, `get_lesson_stats_bulk`, `get_batch_lesson_stats` — O(1) speed se lesson usage stats nikalta hai (summary table se, full history scan se nahi).
- `get_knowledge_report()` — Home page ke liye lessons ka summary (total/active/disabled/draft counts, category breakdown) — pehle 40+ second lagte the, ab summary table se 0.47ms mein.
- `log_activity`, `list_activity` — dashboard ka live Activity Feed (500 rows tak capped).

- `get_kill_switch_state`, `activate_kill_switch`, `deactivate_kill_switch` — global emergency-stop switch: sab trading turant rok sakta hai.
- `record_audit_event`, `list_audit_trail_between`, `list_audit_trail`, `count_audit_trail` — permanent, kabhi-na-delete-hone-wala audit trail (har significant action yahan log hota hai).
- `create_incident`, `update_incident`, `get_incident`, `list_incidents` — structured incident-management workflow (problem → root cause → fix → resolution).
- `get_account_drawdown_state`, `update_account_drawdown_state` — account-wide drawdown circuit-breaker (peak balance track karke, threshold cross hone par pura account pause).
- `create_custom_alert_rule`, `list_custom_alert_rules`, `set_custom_alert_rule_enabled`, `delete_custom_alert_rule`, `mark_custom_alert_rule_triggered` — user-defined "alert me if X" rules.
- `save_feedback`, `list_feedback`, `set_feedback_status` — CEO ke feedback/request notes.
- `add_to_coin_blacklist`, `remove_from_coin_blacklist`, `list_coin_blacklist`, `get_coin_blacklist_symbols` — coins ko permanently trading se exclude karne ka deny-list.
- `create_quick_note`, `list_quick_notes`, `delete_quick_note` — chhota scratch-pad notes feature.
- `_row_to_paper_position` — DB row ko position dict mein convert karta hai, aur derived metrics compute karta hai: `is_win`, `rr` ($R\text{-multiple} = \text{pnl}/\text{risk_amount}$), aur `MAE%/MFE%` (Maximum Adverse/Favorable Excursion) — lowest/highest price seen se `entry_price/direction/size` ke against dollar-amount excursion calculate karta hai.
- `open_paper_position`, `close_paper_position` — position open/close karta hai; close hone par `paper_account_state` (per-strategy running totals) ko bhi automatically update karta hai.
- `update_position_excursion` — har tick par position ka lowest/highest price seen update karta hai (kabhi narrow nahi hota, sirf widen) — `MAE/MFE` ke liye.
- `update_position_stop_loss` — trailing stop-loss ko persist karta hai.
- `get_paper_realized_pnl_total`, `get_paper_account_summary`, `list_paper_account_states` — $O(1)$ speed se har strategy ("book") ka running PnL total nikalta hai.
- `reset_paper_balance`, `reset_strategy_stats`, `list_strategy_stat_archives`, `get_last_balance_reset_at` — "Reset Balance" button ki backend logic: balance zero karta hai lekin trade history kabhi delete nahi karta, pre-reset numbers ko archive table mein save karta hai.
- `get_open_paper_positions`, `get_open_paper_position_symbols`, `get_paper_position`, `set_trade_note`, `list_closed_paper_positions`, `last_closed_paper_position` — positions ko query/filter karne ke functions (cooldown check, trade annotation, waghera ke liye).
- `get_paper_period_summary` — kisi date range ke closed trades ka summary: total pnl, win rate, aur median $R\text{-multiple}$ (mean ke bajaye, kyunke chand outlier trades average ko bahut skew kar dete hain — real data mein mean ~9.9 vs median ~1.9).
- `earliest_paper_trading_activity`, `count_paper_trades_opened_since`, `list_closed_paper_trades_since` — Challenge Mode jaisi features ke liye historical activity data.
- `get_paper_strategy_period_stats` — ek specific strategy ka period-wise performance (wins/losses/breakeven/win-rate/open positions).
- `list_paper_coin_stats`, `list_paper_strategy_stats`, `list_paper_strategy_trading_since` — coin-wise aur strategy-wise aggregate performance ranking.
- `log_paper_decision`, `list_paper_decisions` — har trading decision (open/reject/skip) ka log, 2000 rows tak capped.
- `update_paper_strategy_performance`, `list_paper_strategy_performance` — har strategy ka rolling performance score compute karta hai: $\text{score} = \text{win_rate} * 0.5 + \text{avg_rr} * 10 + \min(\text{total_pnl}, 1000) * 0.01$.
- `update_paper_lesson_performance`, `list_paper_lesson_performance` — isi tarah ka scoring lessons ke liye bhi.
- `get_cloud_setting`, `save_cloud_setting`, `list_cloud_settings_by_prefix` — generic key/value settings storage jab Postgres (cloud) use ho raha ho (ephemeral filesystem ka fix).
- `get_paper_strategy_config`, `list_paper_strategy_configs`, `save_paper_strategy_config` — har strategy ka on/off, priority, allowed coins/market-types config.
- `set_strategy_risk_overrides`, `set_strategy_signal_filter_overrides` — per-strategy risk % aur signal filter (HTF confluence, volume spike, trailing stop) overrides.

- `get_paper_strategy_override`, `list_paper_strategy_overrides`, `save_paper_strategy_override` — manual "flag for Telegram alert" override.
- `create_paper_alert`, `list_paper_alerts`, `get_recent_paper_alert` — trading alerts ka history.
- `list_paper_daily_pnl_by_strategy` — strategy correlation ke liye daily PnL time-series.
- `list_paper_session_stats` — trading session (asian/london/ny) ke hisaab se performance split.
- `list_paper_hour_of_day_stats` — UTC ke har ghante ke hisaab se performance split (kaunsa ghanta consistently accha/bura hai).
- `list_paper_coin_strategy_matrix` — (coin × strategy) heatmap data — kya koi coin sabhi strategies mein consistently accha hai ya sirf ek outlier ki wajah se.
- `list_paper_coin_stats_by_strategy` — ek strategy ka coin-wise breakdown.
- `list_paper_coin_pattern_memory` — (strategy, coin, market_state, session) combination ke hisaab se historical performance memory.
- `get_paper_strategy_coin_reliability_stats` — (strategy, coin) level par exact wins/trades — Wilson confidence gate ko over-fragmentation se bachane ke liye (508+ chhote groups ki jagah ek meaningful sample).
- `list_paper_closed_trades_ordered`, `list_paper_pattern_trades_ordered` — streak/consecutive-loss analysis ke liye oldest-to-newest trade lists.
- `save_trade_review`, `get_trade_review` — ek trade ke liye AI-generated review text (ek trade, ek review — dobara nahi).
- `log_telegram_message`, `list_telegram_messages` — har Telegram send attempt (success ya fail) ka permanent log.
- `enqueue_telegram_retry`, `list_pending_telegram_retries`, `record_telegram_retry_attempt`, `list_telegram_retry_queue` — send fail hone par retry queue (max attempts ke baad "abandoned").
- `count_telegram_messages_since/between`, `get_last_daily_report_sent_at`, `list_telegram_send_texts_since`, `get_last_telegram_signal_sent_at` — reporting/rate-limiting ke liye counts aur timestamps.
- `count_telegram_signals_for_superseded_positions`, `get_versioned_paper_stats` — jab kisi strategy ka extraction correct kiya jata hai to purane (superseded) version ke trades ko naye se alag track karta hai.
- `list_telegram_signal_outcomes` — har bheja gaya signal ka real outcome (win/loss/pending/unknown) `paper_positions` se join karke deta hai — kabhi guess nahi karta.
- `log_research_run`, `count_research_runs_since`, `list_research_runs` — Autonomous Strategy Research ke rate-limiting ke liye run log.
- `save_confluence_score`, `list_confluence_history` — har signal ka confluence score history.
- `has_near_miss_for_position`, `save_near_miss`, `list_near_misses` — jo signal High Confidence threshold ke qareeb tha lekin miss ho gaya, uska log (purely observational).
- `list_generated_signals_with_delivery` — har generate hua signal (chahe Telegram par bheja gaya ho ya nahi) ko uski delivery attempts ke sath dikhata hai — honest reporting, sirf attempted signals nahi.
- `has_telegram_signal_for_position` — check karta hai kisi position ke liye signal bheja gaya tha ya nahi.
- `save_weekly_report/list_weekly_reports/get_latest_weekly_report` — trading performance ka weekly auto-report.
- `save_monthly_report/list_monthly_reports/get_latest_monthly_report` — monthly version.
- `save_evolution_weekly_report/list_evolution_weekly_reports/get_latest_evolution_weekly_report` — evolution/tuning activity ka weekly report.
- `save_infra_weekly_digest/list_infra_weekly_digests/get_latest_infra_weekly_digest` — system health (backups, incidents, disk size) ka weekly digest.
- `record_self_learning_attempt`, `list_self_learning_attempts`, `find_matching_self_learning_attempts` — har naye concept-combination candidate ki koshish (accept ya reject) ka permanent memory — taake same combination baar-baar na try ho bina naye evidence ke.
- `save_self_learning_cycle`, `list_self_learning_cycles`, `get_latest_self_learning_cycle` — weekly discovery cycle ka status/report.
- `create_challenge`, `get_challenge`, `list_challenges`, `update_challenge`, `archive_challenge` — ek se zyada independent dollar-target "challenges" track kar sakta hai (jaise \$1000→\$5000 in 30 din).

- `record_challenge_achievability_snapshot`, `list_challenge_achievability_snapshots` — challenge ki achievability score ka time-trend.
- `save_strategy_lab_scan`, `get_latest_strategy_lab_scan`, `list_strategy_lab_scans`, `approve_strategy_lab_scan` — scheduled scan jo genuinely profitable strategy dhoondta hai; approval sirf CEO ke explicit click se hoti hai.
- `bury_strategy`, `list_graveyard`, `is_strategy_buried` — permanently "buried" (abandoned) strategies ka record, taake future similar imports ko warn kiya ja sake.
- `save_paper_auto_avoid_rule`, `is_pattern_auto_avoided`, `list_paper_auto_avoid_rules`, `deactivate_paper_auto_avoid_rule` — (strategy, coin, market_state, session) pattern jo consistently lose kar raha ho, use auto-veto kar deta hai.
- `save_paper_auto_lesson`, `get_paper_auto_lesson`, `list_paper_auto_lessons`, `deactivate_paper_auto_lesson` — win-rate evidence ke basis par confidence ko boost/avoid karne wala soft filter.
- `set_strategy_paused`, `is_strategy_paused` — per-strategy pause switch.
- `set_strategy_capital_multiplier`, `get_strategy_capital_multiplier`, `list_capital_allocations` — Sharpe Ratio se derive hone wala capital multiplier (Capital Allocation Engine).
- `list_paused_strategies` — saari paused strategies ki list.
- `get_last_n_closed_paper_trades`, `get_paper_downgrade_state`, `list_paper_downgrade_states`, `set_paper_downgrade_state` — Auto-Downgrade Rule: last 100 trades ka Profit Factor < 1.0 hone par strategy ko downgrade classify karta hai (disable nahi karta, sirf label).
- `record_server_restart`, `list_server_restarts`, `count_server_restarts` — har process start ka log (Restart Analytics — crash detect nahi kar sakta, sirf start events).
- `set_coin_priority`, `remove_coin_priority`, `list_coin_priority` — coins ko "pinned" (hamesha include) ya "demoted" (exclude) karne ka reversible control.
- `create_user_goal`, `list_user_goals`, `mark_goal_achieved`, `archive_user_goal` — general trading goals (win rate, PnL target) set karna.
- `save_paper_lesson_candidate`, `list_paper_lesson_candidates` — repeated patterns jo lesson banne ke candidate hain, human review ke liye flag.
- `save_compiled_document`, `get_compiled_document`, `list_compiled_documents` — imported strategy/lesson documents ka structured record (sections, concepts, hidden rules, psychology notes — sab AI-assisted ho sakta hai).
- `touch_knowledge_concept`, `list_knowledge_concepts` — trading dictionary terms ka real-usage tracker.
- `save_condition_report`, `get_condition_report`, `list_condition_reports` — backtest condition-hit reports.
- `save_ai_usage_log`, `list_ai_usage_log`, `ai_usage_summary`, `ai_usage_since` — har AI provider call ka log aur aggregate usage/cost stats.
- `enqueue_ai_import`, `update_ai_import_queue`, `get_ai_import_queue_item`, `list_ai_import_queue`, `ai_import_queue_stats`, `claim_next_pending_ai_import` — AI-assisted document import queue, atomic claim-next pattern se double-processing avoid karta hai.
- `enqueue_pipeline_submission`, `update_pipeline_submission`, `get_pipeline_submission`, `list_pipeline_submission_queue`, `get_processing_pipeline_submission`, `claim_next_pending_pipeline_submission` — strategy submission queue jab ek pipeline already chal rahi ho.
- `save_ai_dictionary_entry`, `get_ai_dictionary_entry`, `list_ai_dictionary_entries` — self-building trading dictionary (AI se enrich hoti hai: aliases, examples, related concepts).
- `get_ai_import_cache`, `save_ai_import_cache` — same document dobara import hone par AI call skip karne ka cache.
- `save_extraction_fidelity_report`, `get_extraction_fidelity_report(_for_strategy)`, `get_latest_extraction_fidelity_reports_for_strategies`, `set_extraction_fidelity_strategy_id`, `set_extraction_fidelity_override` — strategy extraction kitni complete thi uska audit (missing rules count), CEO override ke sath.
- `save_knowledge_relationship`, `list_knowledge_relationships` — concepts/strategies/lessons ke beech relationships graph.

- `create_bot_strategy`, `update_bot_strategy_result`, `archive_bot_strategy`, `get_bot_strategy`, `list_bot_strategies`, `latest_generation_for_base`, `list_bot_strategy_base_ids`, `list_untested_bot_strategies` — auto-generated/evolved strategies ka permanent lineage record (har generation hamesha rakhi jaati hai, kabhi overwrite nahi hoti).
- `get_trade_gate`, `set_trade_gate_threshold`, `set_active_generation_id` — 100-trade evolution gate: sirf tab evolve hota hai jab trade count ek naya 100 ka multiple cross kare; rollback hone par active generation ko parent par pin kar deta hai.
- `create_evolution_comparison`, `get_pending_comparison_for_child`, `finalize_evolution_comparison`, `list_evolution_comparisons`, `list_evolution_comparisons_between` — parent vs child generation ka before/after performance comparison, permanent audit trail.
- `create_bot_lesson`, `get_bot_lesson`, `list_bot_lessons`, `latest_generation_for_lesson_base`, `list_bot_lesson_base_ids` — bot-generated lessons ka apna alag lineage.
- `create_evolution_job`, `update_evolution_job`, `get_evolution_job`, `list_running_evolution_jobs`, `list_evolution_jobs` — evolution background job ka checkpoint/resume.
- `save_champion`, `get_current_champion`, `list_current_champions` — har category (strategy/lesson/coin/session/timeframe) ka "best performer" record, append-only history.
- `create_knowledge_version`, `get_latest_knowledge_version`, `list_knowledge_versions` — poori knowledge base ka versioned snapshot, kabhi delete nahi hota.
- `get_daily_generation`, `try_reserve_ai_generation_call`, `increment_daily_candidates_generated` — hard limiter: din mein sirf EK AI-assisted strategy generation call ho sakti hai (atomic UPDATE se guarantee).
- `save_external_channel`, `list_external_channels`, `get_external_channel`, `set_external_channel_enabled`, `rename_external_channel`, `delete_external_channel` — external Telegram channels ka CRUD (isolated tables, kabhi paper_trading data se mix nahi hota).
- `save_external_message`, `list_unprocessed_external_messages`, `mark_external_message_processed`, `list_external_messages` — raw incoming messages ka capture/processing queue.
- `save_external_signal`, `list_external_signals` — parsed signals ka record (accepted ya rejected, dono).
- `open_external_position`, `get_external_position`, `list_external_positions`, `update_external_position_entries`, `update_external_position_stop_loss`, `close_external_position` — external-channel paper positions ka poora lifecycle.
- `get_external_channel_performance`, `update_external_channel_performance`, `external_channel_coin_breakdown` — har channel ka apna alag fake-money balance/PnL/win-rate track karta hai.

db_backend.py

- `env_flag/IS_POSTGRES` — DATABASE_URL set hai ya nahi, is se decide karta hai SQLite (local) ya Postgres (cloud) use karna hai.
- `_translate_placeholders(sql)` — SQLite ke ? placeholders ko Postgres ke %s mein convert karta hai (simple text substitution).
- `_PGCursorResult / _PGConnection` — psycopg2 connection ko sqlite3-jaisa interface deta hai (`.execute().fetchone()/fetchall()`) taake storage.py ke 5000+ lines ke query code ko bilkul unchanged rakha ja sake, dono backends ke liye.
- `get_postgres_conn()` — Postgres se ek connection kholta hai, 10-second connect timeout ke sath (taake unreachable DB pura app hang na kar de).
- `POSTGRES_SCHEMA` — cloud runner ke liye curated (chhota) schema — sirf paper trading + Telegram path ke zaroori tables, 45GB wali local database ke bade tables (klines, backtest, evolution) yahan nahi hain.
- `init_postgres_schema(conn)` — Postgres database par curated schema create karta hai (idempotent, IF NOT EXISTS).

external_signals

__init__.py

- Sirf module docstring hai (isolation design explain karta hai) — koi code nahi.

config.py

- `load()` / `save(settings)` / `update(fields)` — External Signal Tracker ke apne settings (Telegram user-session credentials, forwarding bot token/channel, proving-trades-required threshold, ingestion on/off) ko `external_signals_settings.json` mein persist karta hai — `paper_trading_settings.json` se bilkul alag file.

channels.py

- `add_channel(name, telegram_identifier)` — naya external channel add karta hai, uuid ID aur stable "Source A/B" label generate karke.
- `list_channels()` — saare channels ki list.
- `set_enabled(channel_id, enabled)` — channel ko on/off karta hai.
- `rename(channel_id, new_name)` — channel ka naam badalta hai.
- `remove(channel_id)` — channel row hataata hai, lekin uski saari messages/signals/positions history hamesha rehti hai (delete nahi hoti).

channel_stats.py

- `channel_report(channel_id)` — ek channel ka poora performance report banata hai: balance, closed/open/pending trades, win rate, avg R-multiple, best/worst coin, aur 25-trade Wilson-score reliability classification (`paper_trading` module se reuse kiya gaya, dobara implement nahi kiya) — 30-trade "proving" progress bhi dikhata hai.
- `all_channel_reports()` — har channel ka report ek saath.
- `comparison_view()` — saare channels ko side-by-side compare karta hai, har ek ko honest label deta hai (jaise "Unproven — sirf X/30 trades", "Reliable", ya "Proven sample, but not reliable edge") — chhote sample size ko kabhi lucky-streak ki tarah trustworthy nahi dikhata.

forwarder.py

- `is_channel_eligible_for_forwarding(channel_id)` — check karta hai kya channel ke signals CEO ke apne Telegram par forward karne layak hain: kam se kam 30 closed trades AND overall profitable hona zaroori hai (dono conditions, sirf trade count kaafi nahi).
- `_next_source_label()` — naye channel ke liye agla available "Source A/B/C..." label generate karta hai (privacy ke liye asli channel naam kabhi forward nahi hota).
- `format_forwarded_message(channel, signal, lang)` — forward hone wala Telegram message banata hai (emoji, coin, direction, entries, SL, TP), source ka sirf stable label use karke, asli naam kabhi nahi.
- `_raw_send(text, bot_token, channel_id, timeout)` — Telegram Bot API ko seedha HTTP POST karta hai (apna khud ka minimal sender, `paper_trading` ke bot se bilkul independent).
- `forward_signal_if_eligible(channel_id, signal, entry_time_ms)` — pura forwarding decision pipeline: eligibility check → settings check → freshness check (`paper_trading.telegram_bot.freshness_check` reuse karke, taake purana/stale signal kabhi der se na bheja jaye) → actual send.

telegram_reader.py

- `telethon_available()` — check karta hai Telethon library installed hai ya nahi.
- `_get_client()` — CEO ke already-completed login session se Telethon client banata hai; agar credentials/session missing hon to clear error deta hai.
- `start_interactive_login(api_id, api_hash, phone_number, code_callback, password_callback)` — one-time interactive Telegram login flow (SMS code + optional 2FA password), jiska result (session string) permanently save hota hai taake dobara login na karna pade.
- `fetch_new_messages(channel, limit)` — ek channel se naye messages (text/image/voice) fetch karta hai jo pehle capture nahi huye, aur unhe `ingest.capture_message()` ko bhejta hai (sirf raw storage, parsing nahi).
- `poll_all_enabled_channels(limit_per_channel)` — saare enabled channels ko ek tick mein poll karta hai — ye hi periodic background job ka core hai.

ingest.py

- `capture_message(...)` — Stage 1: message (text ya media) ko as-is store karta hai (external_messages table + media file disk par), kabhi parse/AI/OCR call nahi karta yahan — taake parsing fail hone par bhi original message kabhi na khojaye.
- `_resolve_text_for_message(msg)` — captured message ko plain text mein convert karta hai: text seedha, voice ko transcription se, image ko ocr se (jo abhi unsupported hai, honest error deta hai).
- `process_pending_messages(limit, use_ai_fallback)` — Stage 2: har pending message ko text mein resolve karke `parser.parse_message()` se guzarta hai, result save karta hai, aur agar real signal nikla to turant (same pass mein) `paper_engine.open_position_from_signal()` se paper-trade karta hai aur `forwarder.forward_signal_if_eligible()` se forwarding-eligibility check karta hai — speed ke liye kabhi baad ke batch job tak delay nahi karta.

parser.py

- `_extract_symbol/_extract_direction/_extract_entries/_extract_stop_loss/_extract_take_profits/_extract_leverage` — regex-based deterministic extraction functions jo Telegram signal message se coin, direction (long/short), entry price(s) (DCA/multi-entry support ke sath), stop-loss, take-profit targets, aur leverage nikalte hain.
- `_looks_like_update_or_commentary(text)` — check karta hai ke message ek naya signal hai ya sirf ek purane signal ka update/commentary (jaise "SL moved to breakeven", "TP1 hit") — aise messages ko kabhi naye trade ki tarah parse nahi karta.
- `parse_text_deterministic(text)` — Stage 1 parsing: free, regex-based; agar kaafi fields (coin/direction/entry) na milen to honestly reject kar deta hai (is_signal=False + reason), kabhi fabricate nahi karta.
- `parse_text_with_ai(text, ...)` — Stage 2 fallback: sirf tab chalti hai jab Stage 1 fail ho aur message signal-jaisa lage; ai_integration's existing provider chain reuse karta hai (dobara implement nahi karta), AI se structured JSON signal extract karta hai.
- `parse_message(text, use_ai_fallback)` — main entry point: pehle deterministic try karta hai, sirf tab AI fallback chalata hai jab message mein number ho aur wo update/commentary na lage (taake random chat messages par AI call waste na ho).

transcription.py

- `transcribe_voice_note(file_bytes, filename, timeout)` — Groq ke hosted Whisper (whisper-large-v3) API ko use karke voice note ko text mein convert karta hai — ek verified working path, jiska result phir usi `parser.parse_message()` se guzarta hai jaise normal text.

ocr.py

- `extract_text_from_image(file_bytes)` — honestly unimplemented: is environment mein na Tesseract OCR available hai na koi working vision-capable AI model — hamesha (None, reason) return karta hai. Image messages phir bhi fully ingest (store) hoti hain, sirf unprocessed reason ke sath mark hoti hain — data kabhi lost nahi hota, sirf abhi process nahi ho sakta.

paper_engine.py

- `_validate_stop_loss(direction, entry_price, stop_loss)` — agar stop-loss entry ke galat side par ho to use discard kar deta hai (galat SL, no-SL se bhi bura hai).
- `_emergency_stop(direction, entry_price)` — jab real stop-loss na mile to paper_trading wala hi fallback emergency-stop formula reuse karta hai (dobara implement nahi karta) — position kabhi unprotected nahi rehta.
- `open_position_from_signal(signal)` — external channel ke signal se ek naya fake-money position kholta hai: pehla entry turant fill hota hai, baaki DCA entries "pending" rehte hain jab tak price wahan na pahunche; invalid take-profits (galat side wale) filter ho jate hain.
- `_fetch_live_price(symbol, exchange)` — real exchange se live price leta hai (paper_trading wala hi function reuse karke) — kabhi simulate/estimate nahi karta.
- `_weighted_avg(filled_entries)` — multiple DCA entries ka size-weighted average entry price calculate karta hai.
- `_dca_entry_reached(direction, entry_price, live_price)` — check karta hai kya price kisi pending DCA entry tak pahunch gaya hai (long ke liye neeche, short ke liye upar).
- `_exit_hit(direction, level, live_price, is_stop)` — check karta hai kya stop-loss ya take-profit level hit hua hai.

- `check_price_updates(symbol)` — main live tick: har open/pending external position ko current price ke against check karta hai — DCA entries fill karta hai, aur stop/target hit hone par position close karta hai.
- `_close_position(pos, exit_price, exit_reason)` — position close karke PnL aur R-multiple calculate karta hai (stop-loss distance ke against), phir channel ka apna alag (isolated) balance/performance update karta hai.
- `close_position_manually(position_id, exit_price, exit_reason)` — "close now" jaisi channel updates ke liye manual close.
- `move_stop_loss(position_id, new_stop_loss)` — "SL to breakeven" jaisi channel updates ke liye stop-loss move karta hai (validate karke).

paper_trading (live trading engine + Telegram)

paper_trading

__init__.py

- Empty file, koi code ya logic nahi hai — sirf Python package marker hai.

lesson_matcher.py

- `relevant_lessons(market_state, timeframe)`: Active lessons ko filter karta hai jo Paper Trading ke liye enabled hain, current market state/timeframe se match karte hain, aur enforceable hain. Priority (Critical > High > Medium > Low) ke hisaab se sort karta hai.

live_feed.py

- `refresh_coins(exchange_client, symbols, log)`: Shortlisted coins ke naye (0-2) candles fetch karta hai data_engine ke existing downloader ko reuse karke. Ek coin ka error dusre coins ko fetch hone se nahi rokta.

ensemble_voting.py

- `agreeing_book_count(approved_candidates, direction)`: Ek symbol+direction par kitni independent strategies/lessons ("books") is tick mein agree kar rahi hain, count karta hai.
- `has_enough_agreement(approved_candidates, direction, min_agreeing)`: Check karta hai ke minimum required strategies ne same direction par agree kiya hai ya nahi (Ensemble Voting Confirmation gate — off by default, risk-reducing only).

position_size_calculator.py

- `calculate(balance, entry_price, stop_loss, risk_pct, take_profit, leverage)`: Engine ki asal position-sizing logic (`backtest_engine._position_size`) ko wrap karke CEO ko "is balance/risk/stop par kitna size khulega" dikhata hai bina real trade khole. Risk amount, reward amount, aur risk-reward ratio bhi calculate karta hai.

evolution.py

- `record_outcome(position, pnl, pnl_pct)`: Har closed trade ke baad strategy aur lesson ki performance stats (win/loss, win-rate) ko update karta hai — kabhi strategy/lesson ka actual content nahi badalta, sirf ranking/score data maintain karta hai.

coin_blacklist.py

- `add(symbol, reason) / remove(symbol) / list_all()`: Coin blacklist (permanent deny-list) manage karte hain.
- `filter_out_blacklisted(symbols)`: Blacklisted symbols ko list se hata deta hai taake wo kabhi score/rank/trade na ho.

dynamic_risk.py

- `compute_risk_pct(exchange, symbol, base_risk_pct)`: Coin ke current market regime (high_volatility/trending/ranging) ke hisaab se risk % adjust karta hai — high volatility mein risk ko half kar deta hai, warna configured default risk use karta hai. Insufficient data par bhi default risk hi rehta hai.

sparkline.py

- `make_sparkline(values)`: Numeric values ki list se Unicode block-character sparkline banata hai (text-based mini chart), Telegram signal messages mein use hone ke liye — bina kisi extra chart library ke. Bad/empty input par bhi crash nahi hota.

time_of_day_filter.py

- `is_blocked_now(now, settings)`: Configured UTC hour window (jaise raat ke illiquid hours) ke dauran naye paper-trading entries ko block karta hai. Overnight wraparound windows (jaise 23:00-07:00) ko bhi handle karta hai; zero-width window ka matlab "hamesha off" hota hai.

profit_lock.py

- `compute_trailing_stop(direction, entry_price, original_stop_loss, highest_price_seen, lowest_price_seen, trigger_r, trail_pct)`: Jab position apne original risk se `trigger_r` guna favorably move ho jaye, to stop-loss ko position ke best price ke peeche trail karke profit lock karta hai. Sirf tightening karta hai, kabhi stop ko loosen nahi karta.

reflection.py

- `build(position, exit_price, pnl, pnl_pct, exit_reason)`: Har closed trade ke baad deterministic (non-AI) template se "kyun enter kiya, kyun exit hua, kya sahi/galat hua" ka reflection banata hai — rules jaise quick stop-loss ya large loss ko mistakes mein aur target-hit ko success mein mark karta hai.
- `build_no_trade(reason, market_snapshot)`: Rejected signal ke liye bhi "kyun trade nahi hua" ka short note banata hai.

push_notifications.py

- `send_push(title, message, priority)`: ntfy.sh service ke through mobile push notification bhejta hai (real HTTP POST, no simulation) — agar topic configure nahi hua to clear error return karta hai, silent fail nahi hota.

health_check.py

- `_check_database()`: Database connection reachable hai ya nahi check karta hai.
- `_check_engine_state()`: Paper Trading engine running/stopped state check karta hai.
- `_check_last_candle_fetch()`: Exchange se live tickers fetch ho rahe hain ya nahi test karta hai.
- `run_health_check()`: Teeno checks ko chalake ek "Test Everything" summary deta hai — dashboard se one-click diagnostic ke liye.

coin_heatmap.py

- `compute_coin_heatmap(since_iso)`: Har coin ke liye "consistency" (kitni strategies us par profitable rahi hain) ke hisaab se rank karta hai — sirf total PnL se nahi, balke reliability se.
- `compute_coin_deep_dive(symbol, since_iso)`: Ek specific coin par har strategy ki performance side-by-side dikhata hai.

confidence.py

- `score(candidate, market_snapshot)`: Har signal candidate ko 0-100 confidence score deta hai (lesson influence, strategy version, historical win-rate, trend alignment jaise factors se) — sirf ranking/reporting ke liye, kabhi trade ko block nahi karta.
- `_strategy_perf(strategy_id) / _lesson_perf(lesson_id)`: Strategy/lesson ki historical win-rate lookup karte hain confidence calculation ke liye.

slippage_filter.py

- `estimate_slippage_pct(exchange, symbol)`: Symbol ke recent 1h candle ranges se expected slippage estimate karta hai.
- `check_entry(exchange, symbol, side, entry_price, stop_loss)`: Agar estimated slippage trade ke stop-distance risk budget ka bohot bara hissa kha jaye to entry reject karta hai (off by default).

strategy_matcher.py

- `relevant_strategies(symbol, market_state)`: Saved strategies mein se sirf wo select karta hai jo Paper Trading ke liye enabled hain aur (agar restrict hain) is coin/market-state ko support karti hain — synced (cloud-pushed) strategies bhi include karta hai, priority ke hisaab se sort karta hai.

coin_priority.py

- `pin(symbol, reason) / demote(symbol, reason) / clear(symbol) / list_all()`: Coins ko "pinned" (hamesha shortlist mein) ya "demoted" (shortlist se exclude) mark karne ka manual, reversible mechanism.
- `filter_out_demoted(symbols)`: Demoted coins ko list se hata deta hai.

- `ensure_pinned_included(shortlist, eligible_symbols, log)`: Pinned coins ko shortlist mein add karta hai agar wo already na ho (bina existing ranking ko disturb kiye).

frame_builder.py

- `_add_indicator(df, name, period)`: Dataframe mein ema/sma/rsi/atr indicator column add karta hai agar already na ho.
- `build_entry_frame(exchange, symbol, timeframe, lessons, start_ms, end_ms)`: Standalone lesson-triggered signals ke liye indicator-rich dataframe (BOS, CHoCH, FVG, order blocks, breaker blocks, support/resistance, liquidity sweep, etc.) taiyar karta hai.

sanity_check_alert.py

- `check_position(position, book_balance)`: Naye khule position par red-flag checks karta hai — bohot bara risk %, suspiciously tight/wide stop-loss, ya zero/negative size.
- `check_and_alert(position, book_key, strategy_name, book_balance)`: Agar red flags milein to informational alert save karta hai — trade ko kabhi block nahi karta, sirf second-look ke liye flag karta hai.

drawdown_guard.py

- `evaluate_strategy(strategy_id, strategy_name)`: Strategy ki apni losing streak ya drawdown-from-peak configured threshold cross kare to us strategy ko naye trades ke liye pause kar deta hai (existing open positions unaffected). Already-paused strategy ko graveyard burial ke liye bhi check karta hai.
- `resume_strategy(strategy_id)`: Manual pause ko reverse karta hai.

goal_system.py

- `create_goal(metric, target_value, comparison, scope_strategy_id, label)`: CEO-defined goal (win-rate, net PnL, total trades) create karta hai.
- `_current_value(metric, scope_strategy_id)`: Goal ke metric ki current real value fetch karta hai.
- `_is_achieved(current, target, comparison)`: Goal achieve hua ya nahi check karta hai.
- `list_goals_with_progress(include_archived)`: Har goal ka current progress dikhata hai aur newly-achieved goals ko mark karta hai.
- `archive_goal(goal_id)`: Goal ko archive karta hai.

maintenance_mode.py

- `get_state()`: Maintenance mode ki current state return karta hai.
- `enter_maintenance_mode(actor)`: Engine aur Telegram sending dono ko ek saath pause karta hai, unki pehli state save karke.
- `exit_maintenance_mode(actor)`: Engine/Telegram ko unki pehli (maintenance se pehle wali) state par restore karta hai — blindly ON nahi karta.

ai_trade_review.py

- `is_enabled() / set_enabled(enabled)`: AI Trade Review feature ka toggle (off by default).
- `_build_prompt(closed_position)`: Closed trade ke liye AI prompt banata hai plain-language explanation ke liye.
- `review_trade(closed_position)`: Har trade close ke baad, agar enabled ho, ek hi AI call se short plain-language review generate karta hai (duplicate-safe, fails gracefully agar koi provider available na ho).

trade_journal_export.py

- `export_trade_journal_pdf(strategy_id, limit)`: Closed paper trades ka trade-by-trade PDF journal banata hai (reportlab se) — per-strategy ya sab strategies ke liye.

regime.py

- `classify_regime(exchange, symbol)`: Coin ko "trending", "ranging", ya "high_volatility" regime mein classify karta hai ATR aur moving-average slope ke basic heuristic se — display/filter-only signal, trade decision nahi karta.

- `classify_all(exchange, symbols)`: Multiple symbols ke liye bulk regime classification karta hai, ek symbol ka error dusre ko affect nahi karta.

signal_explainer.py

- `_confluence_ratio(confluence_result)`: Confluence ka pass/total ratio calculate karta hai.
- `grade_signal(confluence_result, reliability_result)`: Signal ko A+/A/B/C grade deta hai confluence strength aur statistical reliability ke combination se.
- `explain_signal(confluence_result, reliability_result)`: Confluence factors aur historical win-rate se ek Roman Urdu explanation sentence banata hai — koi naya AI call nahi, purely deterministic.

auto_avoid.py

- `evaluate_pattern(strategy_id, strategy_name, symbol, market_state, session, since)`: Har trade close ke baad ek exact pattern (strategy+coin+market+session) ka statistically confirmed losing record check karta hai (Wilson interval, 25+ trades) aur agar confirm ho to naye entries suppress karne ka rule bana deta hai — ya agar pattern recover ho gaya ho to rule deactivate karta hai.
- `is_avoided(strategy_id, symbol, market_state, session)`: Engine ke liye check karta hai ke ye exact pattern currently avoided hai ya nahi.

htf_confluence_filter.py

- `higher_timeframes_for(entry_timeframe)`: Entry timeframe ke liye relevant higher timeframes (HTF) return karta hai.
- `htf_trend(exchange, symbol, htf_timeframe)`: Higher timeframe ka structural trend ("up"/"down"/None) nikalta hai.
- `check(exchange, symbol, entry_timeframe, direction)`: Agar signal ki direction higher-timeframe trend ke against ho to signal ko suppress karta hai (opt-in per-strategy filter).

risk_manager.py

- `account_balance(book_key, initial_balance)`: Strategy ki current real balance calculate karta hai (initial balance x capital multiplier + realized PnL).
- `evaluate(book_key, symbol, candidate, settings, exchange)`: Har naye trade candidate ko approve/reject karta hai — kill switch, account drawdown pause, time-of-day filter, max-coins cap, missing stop-loss, slippage filter, aur position sizing sab check karta hai. Ye system ka main risk gate hai.

pattern_stats.py

- `wilson_interval(wins, n, z)`: Ek win/loss record ke liye Wilson score confidence interval calculate karta hai (statistically sound method).
- `classify(wins, n, min_sample_size)`: Pattern ko "insufficient_data", "reliable_good", "reliable_bad", ya "reliable_inconclusive" classify karta hai — 25-trade minimum sample size ke saath. Ye Pattern Auto-Avoid aur Lesson Auto-Apply dono ka shared statistical foundation hai.

lesson_auto_apply.py

- `promote_candidates()`: Coin-specific pattern memory scan karke jo patterns statistically reliable (Wilson gate, 25+ trades) hain unhe active "auto-lesson" rule mein promote karta hai — ya jo drift ho gaye hain unhe deactivate karta hai.
- `get_influence(strategy_id, symbol, market_state, session)`: Confidence score ke liye ek chota nudge (+/-10) return karta hai agar exact pattern ka active auto-lesson mile — kabhi trade block nahi karta.

graveyard.py

- `bury_if_abandoned(strategy_id, strategy_name)`: Agar koi strategy paused hai aur uski losing streak bohot zyada lambi ho gayi hai to usko permanent "graveyard" record mein bury karta hai (informational, strategy delete/disable nahi karta).
- `compute_retirement_suggestions()`: Buried lekin abhi archive nahi hui strategies ko retirement suggestion ke liye surface karta hai.

- `check_similarity_warnings(concepts_used)`: Naya strategy import hone par buried strategies se concept-overlap warning deta hai.

confluence.py

- `score_confluence(strategy_id, symbol, exchange, market_state, session, direction)`: Ek signal ke liye 4 independent quality factors (regime alignment, drawdown-pause status, coin-crowding by other strategies, positive pattern memory) count karke ek "Strong/Moderate/Weak" label deta hai — display/ranking ke liye hai, trade ko block nahi karta.

custom_alerts.py

- `create_rule(name, metric, comparison, threshold, strategy_id)`: User-defined alert rule create karta hai (fixed, safe metric list se — koi arbitrary expression nahi).
- `list_rules()` / `delete_rule(rule_id)` / `set_rule_enabled(rule_id, enabled)`: Rules manage karte hain.
- `_current_value(rule)`: Rule ke metric ki current real value fetch karta hai.
- `_breaches(rule, value)`: Value threshold cross kar rahi hai ya nahi check karta hai.
- `sweep_custom_alert_rules()`: Har enabled rule check karke breach hone par alert create karta hai (throttled, 6-hour recheck window).

account_drawdown_guard.py

- `_combined_balance()`: Har strategy book ki combined balance calculate karta hai.
- `status()`: Current drawdown %, peak balance, aur pause state return karta hai.
- `is_globally_paused()`: Account-wide pause active hai ya nahi check karta hai.
- `evaluate_account()`: Combined balance apne all-time peak se configured % gir jaye to sab strategies ke naye trades ek saath pause kar deta hai (existing positions untouched).
- `resume_account(actor)`: Account-wide pause ko manually reverse karta hai.

kill_switch.py

- `is_active()` / `status()`: Kill switch ki current state check karta hai.
- `activate(reason, actor, close_positions)`: Sara trading turant halt karta hai — engine stop, sab open positions force-close (agar chaha jaye), Telegram sends bhi silence.
- `deactivate(actor)`: Kill switch clear karta hai — lekin engine ko wapis start nahi karta, manual "Start Engine" click zaroori hota hai.
- `_close_all_open_positions(actor)`: Sab open positions ko current market price par force-close karta hai.

challenge_ai_advisor.py

- `_fallback_explanation(progress, difficulty)`: Bina AI ke plain-language explanation deta hai ke challenge target realistic hai ya nahi.
- `explain(progress, difficulty, best_worst_likely)`: Real, already-computed challenge numbers ko AI se plain-language mein explain karwata hai (agar AI provider available ho), warna fallback explanation use karta hai — kabhi risk_pct ya trading behavior nahi chhta.

capital_allocation.py

- `compute_multiplier(sharpe_ratio)`: Strategy ke Sharpe Ratio se ek bounded (0.5x-1.5x) capital multiplier calculate karta hai.
- `recompute_all_allocations()`: Har strategy ka current Sharpe Ratio use karke uska capital multiplier update karta hai (5+ trades required).
- `compute_recommended_risk_pct(base_risk_pct, sharpe_ratio)`: Same formula se recommended risk % calculate karta hai.
- `compute_all_risk_pct_recommendations(base_risk_pct_default)`: Har strategy ke liye risk % suggestion deta hai (human-approved, auto-apply nahi hota).

telegram_analytics.py

- `signal_period_summary(since_iso, until_iso)`: Ek period mein bheje gaye Telegram signals ka win/loss/pending summary deta hai (25-trade gate ke baad hi win-rate show karta hai).
- `hypothetical_pnl(since_iso, until_iso)`: Real R-multiples ko ek hypothetical \$100 account par rescale karke dikhata hai.
- `strategy_breakdown(since_iso, until_iso)`: Per-strategy Telegram signal outcomes ka breakdown deta hai.
- `best_performing_strategy(since_iso, until_iso)`: Telegram-signaled trades mein sabse best real-PnL strategy return karta hai.

market_state.py

- `classify(exchange, symbol)`: Coin ka current market state (trending_up/down, ranging, volatile, breakout) cheap deterministic rules se classify karta hai, structure changes (BOS/CHoCH) bhi detect karta hai.
- `EventTracker.check(snapshot)`: Har coin ke last state ko yaad rakhta hai aur sirf tab tick continue karta hai jab kuch genuinely change ho (new candle, trend flip, breakout, volume spike, structure change) — ye "har coin har tick scan mat karo" ka gate hai.

cloud_sync.py

- `build_snapshot()`: Cloud runner ka lightweight paper-trading/Telegram data (open/closed positions, signal log, strategy stats) snapshot banata hai.
- `run_sync()`: Snapshot generate karke persist karta hai.
- `get_latest_snapshot()`: Latest saved snapshot return karta hai.
- `_should_run_now()`: 24 ghante guzar gaye hain ya nahi check karta hai (drift-proof hourly gate).
- `start_cloud_sync_scheduler_thread()` / `stop_cloud_sync_scheduler_thread()`: One-way daily backup scheduler ko start/stop karte hain (sirf cloud runner par chalti hai).

guards.py

- `book_key(candidate)`: Candidate ke liye strategy_id ya "__lessons__" shared book return karta hai.
- `position_locked(book_key, exchange, symbol, direction)`: Same strategy ka identical position already open hai ya nahi check karta hai (duplicate prevent).
- `opposite_open_position(book_key, exchange, symbol, direction)`: Opposite direction ka open position dhoondta hai.
- `resolve_opposite_signal(book_key, exchange, symbol, direction, policy)`: Opposite signal aane par proceed/block/close-and-reverse decide karta hai.
- `cooldown_active(book_key, exchange, symbol, direction, cooldown_minutes)`: Position close hone ke baad cooldown period active hai ya nahi check karta hai.
- `signal_fingerprint(exchange, symbol, candidate)`: Same market condition ke duplicate signal detect karne ke liye unique fingerprint banata hai.
- `GuardState.reserve/release_all/is_duplicate`: In-memory tick-level trade reservation aur decision-cache (duplicate signal protection) maintain karta hai.
- `rank_candidates(candidates, rule)`: Multiple candidates mein se ek winner choose karta hai (confidence/win_rate/profit/manual priority rule se).

strategy_profile.py

- `get_strategy_profile(strategy_id, exchange, correlation_warnings)`: Ek strategy ki har cheez (confidence, streak, risk metrics, VaR, health score, aging, MAE/MFE, pause status, lesson candidates, correlation warnings, genealogy, real-trading readiness, extraction-correction supersession) ko ek jagah assemble karta hai — koi naya calculation nahi karta, sirf existing verified data combine karta hai.

coin_filter.py

- `_coin_activity_score(exchange, symbol)`: Volume, volatility, trend strength se coin ka activity score calculate karta hai (cached).

- `shortlist(exchange, symbols, top_n, log)`: Sab tracked coins ko rank karke top-N shortlist banata hai — data-freshness errors ko explicitly log karta hai taake empty shortlist silently na ho.

config.py

- `load()` / `save(settings)` / `update(fields)`: Paper Trading ke sab settings (dry_run, risk defaults, max trades, cooldown, drawdown thresholds, time filter, profit-lock, ensemble voting, ntfy topic, etc.) ko JSON file ya cloud Postgres mein persist karta hai — dashboard se restart ke bina edit ho sakte hain.

strategy_lab.py

- `scan_for_profitable_strategy()`: Weekly, har strategy ke real paper-trading stats check karta hai (25+ trades, positive PnL, 50%+ win-rate) aur agar koi qualify kare to save karta hai.
- `maybe_run_strategy_lab_scan()`: 7-day interval gate ke sath scan trigger karta hai.
- `start_strategy_lab_scheduler_thread()`: Background scheduler chalata hai.
- `approve_candidate(scan_id, strategy_id)`: CEO ka explicit one-click approval jo qualifying strategy ko live Paper Trading aur Telegram ke liye enable karta hai.

correlation.py

- `_returns(exchange, symbol)`: Symbol ke recent price returns nikalta hai.
- `_correlation(exchange, symbol_a, symbol_b, cache)`: Do symbols ke returns ka Pearson correlation calculate karta hai.
- `detect_warnings(exchange)`: Currently open positions mein se agar 2+ strategies same direction mein correlated coins par ho to warning deta hai (informational, kabhi trade block nahi karta).
- `_pearson(xs, ys)`: Plain Pearson correlation formula.
- `strategy_correlation_matrix(strategy_ids, lookback_days)`: Har strategy pair ke daily realized PnL ka correlation matrix banata hai — asal diversification check karta hai.

status_ping.py

- `build_status_message()`: Engine ki current status (running/stuck/off, balance, open trades) ka message text banata hai.
- `send_status_ping_now()`: CEO ke private Telegram chat par ye status ping bhejta hai.
- `_should_send_now()`: Har ~5 ghante gate check karta hai.
- `start_scheduler_thread()` / `stop_scheduler_thread()`: Periodic status-ping scheduler chalate hain, engine "stuck" hone ki detection bhi karta hai (30+ min bina tick).

signal_generator.py

- `_lookback_start_ms(settings)`: Indicator warmup ke liye fixed calendar window calculate karta hai.
- `_apply_lessons(df, i, direction, lessons)`: Lessons ko tracking/logging ke liye evaluate karta hai (kabhi strategy signal ko veto nahi karta).
- `strategy_candidates(exchange, symbol, strategies, lessons, settings)`: Har relevant strategy ko latest bar par evaluate karke buy/sell signal candidates generate karta hai.
- `lesson_candidates(exchange, symbol, lessons, settings)`: Standalone lesson-based signals generate karta hai (apna stop-loss/take-profit calculate karke).
- `generate_candidates(...)`: Dono paths (strategy + lesson) ko combine karta hai — ye Decision Engine ka core hai.

cloud_aware_auto_stop.py

- `check_cloud_status(cloud_url, sync_secret, timeout)`: Cloud deployment ka paper-trading/Telegram status remote se check karta hai.
- `check_and_maybe_stop_local()`: Agar cloud par dono (Paper Trading + Telegram) ON hain to local engine ko duplicate-trading avoid karne ke liye auto-stop kar deta hai — sirf stop karta hai, kabhi khud start nahi karta.
- `resume_local_after_cloud_pause()`: Cloud-pause state clear karta hai (dashboard button se).

- `start_scheduler_thread()` / `stop_scheduler_thread()`: Har 5 minute cloud status check karne wala scheduler.

monthly_report.py

- `generate_monthly_report()`: 30-day period ka strategy performance summary (doing well/poorly, paused strategies, auto-lessons, avoid rules, recommendations) plain-language mein banata hai.
- `maybe_generate_monthly_report()`: 30-day gate ke sath report generate karke Telegram par bhejta hai.
- `start_monthly_report_scheduler_thread()`: Scheduler chalata hai.

auto_downgrade.py

- `_peak_to_trough_drawdown(pnls_oldest_first)`: Last N trades ke pnl curve ka max drawdown calculate karta hai.
- `evaluate_strategy(strategy_id, initial_balance)`: Last 100 live paper-trades ke Profit Factor aur drawdown ko check karke strategy ko "downgraded" classify karta hai — backtest-based label se alag, live performance ka signal (kabhi auto-disable nahi karta).
- `check_and_log_if_changed(strategy_id, initial_balance)`: Downgrade status change hone par hi permanent audit log entry likhta hai.
- `check_all_enabled_strategies()`: Har enabled strategy ko check karta hai.

signal_tracker.py

- `live_signal_feed(limit)`: Har real Telegram signal ka live outcome feed deta hai.
- `_backtest_win_rate(strategy_name)`: Strategy ka latest backtest win-rate fetch karta hai.
- `strategy_match_table()`: Backtest vs paper vs Telegram-sent win-rates ko side-by-side dikhata hai, divergence flag karta hai (25-trade gate ke sath).
- `check_and_alert_divergence(now_iso)`: Backtest-vs-paper divergence honest alert banata hai (24-hour throttle).

telegram_delivery.py

- `classify_attempt(attempt)`: Ek Telegram send-attempt ko status id mein classify karta hai (sent, network-blocked, withheld, etc.).
- `classify_signal(signal, auto_send_enabled)`: Ek generated signal ki delivery status (sent/queued/never_sent/etc.) decide karta hai — sirf real successful send ko "Sent" kehta hai.
- `_outcome(signal)`: Trade ka real outcome (win/loss/pending) determine karta hai.
- `delivery_rows(signals, auto_send_enabled)`: Raw signals ko delivery status + outcome ke sath decorate karta hai.
- `delivery_summary(rows)`: Period ka overall delivery aur outcome summary deta hai.

telegram_commands.py

- `_get_updates(offset)`: Telegram se naye incoming messages long-poll karta hai.
- `_reply(chat_id, text)`: Telegram ko reply bhejta hai.
- `_is_authorized(chat_id)`: Sirf configured channel_id se aane wale commands ko authorize karta hai — koi aur sender silently ignore ho jata hai.
- `_status_reply()` / `_pause_reply()` / `_resume_reply()` / `_help_reply()`: `/status`, `/pause`, `/resume`, `/help` commands ka actual implementation — engine ko dashboard buttons ki tarah start/stop karte hain.
- `handle_update(update)`: Incoming message ko authorize karke sahi command handler ko route karta hai.
- `poll_once()` / `start_command_polling_thread()`: Telegram se incoming commands ka polling loop chalate hain.

portfolio.py

- `compute_portfolio_analytics(exchange)`: Sab strategies ke combined exposure, open risk, realized PnL, aur correlation-based concentration % ka aggregate view deta hai.
- `compute_portfolio_risk_score(strategy_ids, since)`: Har strategy ke Sharpe/drawdown se ek combined 0-100 portfolio risk score banata hai (documented, transparent heuristic).
- `compute_coin_exposure(exchange)`: Har coin par cross-strategy total risk/notional allocation dikhata hai.

- `detect_duplicate_exposure_warnings(exchange, min_strategies)`: 2+ independent strategies same coin par trade kar rahi hain to warning deta hai.
- `compute_strategy_exposure(exchange)`: Per-strategy total risk/notional exposure dikhata hai.
- `compute_direction_exposure(exchange)`: Long vs short direction ke hisaab se total portfolio exposure split karta hai.

weekly_report.py

- `_daily_pnl_sparkline(period_start_iso, period_end_iso)`: Period ke daily realized PnL ka text-sparkline banata hai.
- `generate_weekly_report()`: 7-day plain-language performance summary (doing well/poorly, paused, auto-lessons, avoid rules, recommendations) banata hai — non-technical CEO ke liye.
- `maybe_generate_weekly_report()`: 7-day gate ke sath report generate karke Telegram par bhejta hai.
- `start_weekly_report_scheduler_thread()`: Scheduler chalata hai.

challenge_multi.py

- `create_challenge(...)`: Ek naya, independent challenge (max 3 simultaneous) create karta hai — daily/weekly/monthly ya custom timeframe ke sath.
- `extend_deadline(challenge_id, new_days)`: Challenge ki deadline extend/shorten karta hai.
- `archive_challenge(challenge_id)`: Challenge archive karta hai.
- `compute_progress_for(challenge_id, now_iso)`: Ek challenge ka full progress dict return karta hai.
- `compute_all_progress(now_iso)`: Sab active challenges ka progress ek sath dikhata hai.
- `compute_compounding_current_amount(challenge_id)`: Real trades ko compounding risk se recalculate karta hai (fixed-risk vs compounding comparison).
- `record_achievability_snapshot(challenge_id)`: Challenge ki achievability score (0-100) ko snapshot karke history banata hai.
- `achievability_trend(challenge_id, days)`: Achievability ka 7-day trend history return karta hai.
- `start_achievability_snapshot_scheduler_thread()`: Periodic snapshot scheduler chalata hai.

daily_report.py

- `_today_start_iso(now) / _month_start_iso(now)`: Date boundary helpers.
- `generate_daily_report(now)`: Roz ki Roman Urdu Telegram report banata hai — signals sent, trades closed, best/worst strategy, losing streaks, month-to-date PnL, challenge progress — sab existing data se, koi naya calculation nahi.
- `maybe_send_daily_report(now)`: Din mein sirf ek baar report bhejta hai (date-based gate).
- `start_daily_report_scheduler_thread()`: Scheduler chalata hai.

strategy_sync.py

- `get_or_create_sync_secret()`: Local-to-cloud sync ke liye ek persistent secret generate/fetch karta hai.
- `get_sync_target() / save_sync_target(cloud_url, sync_secret)`: Local machine ka target cloud URL aur secret manage karta hai.
- `push_strategy_to_cloud(strategy_id, cloud_url, sync_secret, timeout)`: Local strategy ko validate + safety-check karke cloud deployment par push karta hai.
- `receive_synced_strategy(strategy_id, name, tags, config_json, provided_secret)`: Cloud side par received strategy ko re-validate karke store karta hai aur auto-enable karta hai (never trusts sender).
- `get_synced_strategy(strategy_id) / list_synced_strategies()`: Cloud-only synced strategies fetch karte hain.
- `get_sync_log(limit)`: Local sync audit trail return karta hai.

private_performance_report.py

- `compute_period_performance_metrics(period_start_iso, period_end_iso)`: Account-wide win-rate, profit factor, net profit, max drawdown, avg win/loss calculate karta hai.

- `original_vs_evolution_comparison_rows(period_start_iso, period_end_iso)`: Original vs Evolution-generated strategy versions ka comparison table banata hai.
- `generate_report_pdf(period_label, period_start_iso, period_end_iso)`: PDF report banata hai (reportlab) in Hinglish/Urdu formatting ke sath.
- `send_report(period_label, period_start_iso, period_end_iso)`: PDF generate karke CEO ke private Telegram chat par bhejta hai (public channel nahi).
- `maybe_send_weekly_report()` / `maybe_send_monthly_report()`: 7/30-day gates ke sath private reports schedule karte hain.
- `start_private_report_scheduler_thread()`: Scheduler chalata hai (local-only, PDF generation cloud runner par nahi).

position_manager.py

- `auto_tags(market_snapshot, timeframe)`: Position ke liye automatic tags (Trend, Breakout, Scalping, Swing, London/NY session, High Volume, Liquidity) generate karta hai.
- `open_position(exchange, symbol, candidate, size, risk_amount, confidence, market_snapshot)`: Naya paper position khol ta hai — slippage apply karta hai, stop-loss/take-profit ko re-validate karta hai (wrong-side discard), missing stop-loss ke liye emergency fallback deta hai.
- `_check_exit(position, latest_price, high, low)`: Candle ke high/low ko check karke stop-loss/take-profit hit hua ya nahi decide karta hai — backtest engine jaisa hi intrabar-touch detection.
- `monitor_and_close(exchange, symbol, latest_price, high, low)`: Har open position ko monitor karta hai, MAE/MFE excursion update karta hai, trailing stop (Profit-Lock) apply karta hai, aur exit-condition milne par close karta hai.
- `force_close(position_id, latest_price, reason)`: Kisi bhi position ko manually/emergency close karta hai.
- `_close(pos, exit_price, exit_reason)`: Position close karne ka poora lifecycle handle karta hai — pnl calculate, reflection build, evolution outcome record, bot-lesson generation, auto-avoid re-check, drawdown-guard re-check, Telegram close-followup, aur AI trade review — sab non-fatal try/except ke sath.

challenge_mode.py

- `load()` / `save(settings)`: Challenge Mode settings persist karta hai.
- `set_challenge(start_amount, target_amount, days, telegram_report_enabled, ...)`: Naya challenge (target) set karta hai, optional scoped strategy+coin ke sath.
- `clear_challenge()`: Challenge clear karta hai.
- `_required_daily_rate(start_amount, target_amount, days)`: Target hit karne ke liye zaroori daily compound growth rate calculate karta hai.
- `_real_demonstrated_daily_rate()`: System ki real, all-time demonstrated daily pace nikalta hai (real trades se).
- `_current_amount(start_amount, started_at)`: Real trades ko rescale karke "agar ye account challenge se live hota to abhi kahan hota" batata hai.
- `_scoped_daily_rate` / `_scoped_current_amount`: Ek specific strategy+coin combination ke liye same calculations karte hain.
- `_projected_finish_date(...)`: Real realized pace se projected completion date calculate karta hai.
- `_honest_note(...)`: Plain Roman Urdu mein target realistic hai ya nahi honestly batata hai.
- `compute_progress(now_iso, settings)`: Poora challenge progress dict return karta hai — target achievability, drift check, sab kuch.

insights.py

- `fresh_session_start()`: Data-reset timestamp return karta hai taake purani legacy trades naye session ke stats mein mix na hon.
- `all_confidence_scores(lookback)`: Har strategy ka average recent confidence score ek query se nikalta hai.
- `humanize_reason(reason_text)`: Technical rule string ko plain-language sentence mein badalta hai.
- `detect_alerts(strategy_stats, streaks)`: Strong-performance aur drawdown (losing-streak) alerts raise karta hai (30-min dedupe).
- `classify_win_loss(position)`: Closed trade ko plain-language win/loss reason tag deta hai.

- `compute_streak(strategy_id) / all_streaks(limit)`: Consecutive win/loss streak calculate karte hain.
- `detect_lesson_candidates(min_sample, win_rate_extreme)`: Strongly one-sided patterns ko human-review ke liye flag karta hai (kabhi khud lesson nahi banata).
- `compute_risk_metrics(strategy_id, since)`: Sharpe Ratio, Sortino Ratio, aur Max/Current Drawdown % calculate karta hai (standard formulas).
- `compute_value_at_risk(strategy_id, confidence, since)`: Historical simulation method se Value-at-Risk (95th percentile loss) calculate karta hai (25-trade gate).
- `compute_mae_mfe_stats(strategy_id, since)`: Winners vs losers ke liye average MAE/MFE (adverse/favorable excursion) calculate karta hai.
- `compute_strategy_health_score(strategy_id, since)`: Win-rate, profit-factor, drawdown, Sharpe, aur sample-size se ek 0-100 composite health score banata hai (documented weights).
- `detect_win_rate_decay(strategy_id)`: Recent trades ka win-rate baseline se compare karke performance decay detect karta hai.
- `sweep_win_rate_decay_alerts()`: Har strategy check karke decay alert raise karta hai (24-hour throttle).
- `compute_strategy_aging(strategy_id, window_size)`: Trade-windows ke through strategy ki performance trend (improving/weakening/stable) detect karta hai.
- `real_trading_readiness(strategy_id, strategy_stats, safety_passed, walk_forward_status)`: Fixed checklist (30+ trades, 55%+ win-rate, no big losing streak, safety-check pass, walk-forward pass) se decide karta hai ke strategy real trading ke liye consider karne layak hai ya nahi.

challenge_analysis.py

- `_closed_rows(strategy_id, symbol) / _duration_minutes(row) / _max_drawdown(...) / _metrics_for(rows)`: Basic per-combination trade metrics (win-rate, profit-factor, drawdown, duration) calculate karne wale helpers.
- `granular_breakdown()`: Har strategy, coin, aur strategy+coin combination ki real performance breakdown deta hai, best/top combinations highlight karta hai.
- `suggest_best_portfolio(top_n)`: Top distinct, statistically-trusted strategy+coin combinations ka diversified portfolio suggest karta hai.
- `consistency_check(strategy_id, symbol, window_days)`: Check karta hai ke combination ka profit ek hi lucky window se aaya hai ya sustained hai.
- `_combo_daily_rate(strategy_id, symbol, rows)`: Ek combination ki real demonstrated daily growth rate calculate karta hai.
- `recommend_paths(start_amount, target_amount, days, ...)`: Multiple real, ranked paths suggest karta hai target achieve karne ke liye — har ek apni real history se.
- `check_drift(strategy_id, symbol, baseline_win_rate_pct, window_trades)`: Combination ka performance apne baseline se drift ho gaya hai ya nahi check karta hai.
- `difficulty_rating(required_daily_rate_pct, real_daily_rate_pct)`: Easy/Moderate/Hard/Extremely Unlikely label deta hai.
- `best_worst_likely_range(...)`: Best-case/likely-case/worst-case real combinations ki range dikhata hai.
- `suggest_adaptive_risk_pct(...)`: Target hit karne ke liye specific suggested risk % calculate karta hai.
- `risk_level_warning(suggested_risk_pct, current_risk_pct)`: Agar suggested risk bohot zyada ho to warning deta hai.
- `give_up_point_check(remaining_days, best_case_days_to_target)`: Target mathematically ab possible nahi raha to honest alert deta hai.
- `_max_losing_streak(rows_sorted_by_close)`: Combination ka worst historical losing streak nikalta hai.
- `loss_streak_impact(strategy_id, symbol, start_amount, risk_pct)`: Agar worst losing streak dobara ho jaye to uska dollar impact simulate karta hai.
- `find_best_historical_period(strategy_id, symbol, window_days)`: Combination ka best historical growth window dhoondta hai.
- `replay_challenge(start_amount, target_amount, days_ago_started, strategy_id, symbol)`: "Agar N din pehle challenge start kiya hota" ka real-trade replay karta hai.

- `strategy_rotation_suggestion(candidate_strategy_ids)`: Do strategies jo alag-alag market conditions mein best perform karti hain, unko rotate karne ka suggestion deta hai.

engine.py

- `_default_exchange()`: Default exchange return karta hai, cloud mode mein Binance (geo-blocked) ki jagah Bybit use karta hai.
- `PaperTradingEngine.start(log, on_event)`: Engine ko background thread mein start karta hai (kill-switch active ho to refuse karta hai).
- `PaperTradingEngine.stop()`: Engine ko stop karta hai.
- `PaperTradingEngine.status()`: Overall aur per-strategy balance, open trades, tick info, scan progress dikhata hai.
- `run_single_tick_now()`: Manual dry-run ke liye ek tick synchronously chalata hai.
- `run_single_coin_scan_now(symbol)`: Ek specific coin ka manual scan karta hai (debugging ke liye), sab normal gates ke sath.
- `_loop()`: Engine ka main 24/7 background loop.
- `_tick()`: Ek poora trading cycle — coin filter, blacklist/priority filter, shortlist, market-state classify, decision engine, hourly sweeps (Telegram, divergence check, retry queue, win-rate decay, custom alerts) sab chalata hai.
- `_monitor_orphaned_positions(exchange, client, shortlisted_symbols)`: Un coins ke open positions ko bhi monitor karta hai jo shortlist se bahar ho gaye hain; delisted/missing-ticker symbols ke liye alert deta hai.
- `_process_coin(exchange, symbol, snapshot, settings)`: Ek coin ke candidates generate/approve karta hai, book-wise group karke ensemble voting check karta hai.
- `_open_if_allowed(book, exchange, symbol, pick, snapshot, settings)`: Trade khulne se pehle sab gates (pause, auto-avoid, HTF filter, volume filter, position lock, cooldown, opposite-signal, risk-manager) sequentially check karta hai, phir position khol ta hai aur confluence/sanity-check/Telegram auto-send trigger karta hai.
- `_log_decision(...)`: Har decision (approved/rejected/opened) ko permanent decision log mein save karta hai.
- `resume_engine_on_startup()`: Server restart ke baad engine ko CEO ke last explicit choice (on/off) par restore karta hai — kill switch active ho to off hi rehta hai.

telegram_bot.py

- `load_settings() / save_settings(fields)`: Telegram settings persist karte hain (bot token, channel id, auto-send config, silent hours, etc.), env-var seeding aur cloud/local persistence dono handle karte hain.
- `public_settings()`: Safe-to-display settings view (kabhi raw token expose nahi karta).
- `channel_for_strategy(strategy_id) / set_strategy_channel_override(strategy_id, channel_id)`: Per-strategy alag Telegram channel routing (Multi-Channel Support).
- `is_within_silent_hours(now)`: Do-Not-Disturb window check karta hai (sirf notification sound mute karta hai, message send hota rehta hai).
- `_master_enabled()`: Master ON/OFF switch check karta hai — ye sab sends ka single point of control hai.
- `_rate_limited()`: Per-hour rate limit check karta hai.
- `_build_proxies(settings)`: Configured SOCKS5/HTTP proxy dict banata hai (kuch countries mein Telegram network-blocked hai).
- `_raw_send(text, channel_id_override)`: Real Telegram HTTP API call karta hai retries ke sath (transient network failures par retry, real API errors par nahi).
- `test_proxy_connectivity()`: Proxy ka basic internet-reachability test karta hai.
- `send_test_message()`: Connection-confirmation test message bhejta hai.
- `send_private_message(text)`: CEO ke private chat par message bhejta hai (public channel se alag).
- `send_private_document(file_path, caption)`: CEO ko private chat mein file (PDF report) bhejta hai.
- `_format_price(value)`: Display ke liye price ko sahi decimals ke sath format karta hai (low-price coins ke liye zyada decimals).
- `_direction_emoji / _confidence_icon`: Direction aur confidence ke liye emoji/icon return karte hain.
- `_signal_age_text(entry_time_ms)`: Signal ki age ko human-readable form mein deta hai.
- `_fetch_live_price(exchange, symbol)`: Live current price fetch karta hai (best-effort, fail-safe).

- `_recent_price_sparkline(exchange, symbol)`: Recent 24h price ka text sparkline banata hai.
- `signal_age_minutes(position, now_ms) / is_signal_stale(...)`: Signal ki age check karke stale hai ya nahi decide karte hain (Freshness Gate).
- `price_has_moved_away(position, live_price, max_drift_pct)`: Live price entry se bohot zyada drift kar gaya hai ya nahi check karta hai.
- `freshness_check(position, now_ms)`: Age aur price-drift dono ko combine karke signal bhejne layak hai ya nahi decide karta hai.
- `format_signal_message(...)`: Poora Telegram signal message text banata hai — high-confidence marker, challenge tags, trailing-stop status, profitability label, entry/SL/TP, live price, sparkline, statistical confidence, confluence factors, explanation, reason, timestamp, disclaimer sab include karke.
- `_profitability_label(strategy_id, lang)`: Strategy "Profitable" hai ya "under evaluation" ye label deta hai (25-trade real live PnL gate).
- `_challenge_mode_tag(position, lang) / _multi_challenge_tags(position, lang)`: Agar signal kisi active challenge se match kare to tag add karta hai.
- `_pattern_reliability_for(strategy_id, symbol, market_state, session)`: Wilson-gated pattern reliability nikalta hai (strategy+coin level grouping).
- `send_signal_for_position(position_id, trigger_type, high_confidence, retry_id)`: Asal Telegram send ka single entry point — kill-switch, master-switch, rate-limit, freshness sab check karke message bhejta hai aur poora audit trail log karta hai.
- `evaluate_auto_send(position_id)`: HIGH-tier auto-send rule — confluence 100%, 25-trade Wilson-confirmed pattern, strategy not paused, live PnL non-negative sab zaroori.
- `evaluate_auto_send_low_tier(position_id)`: LOW-tier auto-send rule — same base checks lekin Wilson gate ke bina, signal flow maintain karne ke liye.
- `evaluate_auto_send_tier(position_id)`: High tier try karta hai, phir low tier — "High Confidence Only" setting hone par low-tier signals ko silently drop kar deta hai (near-miss log karke).
- `_record_near_miss(position_id, reason)`: Har signal jo High Confidence tak nahi pahoncha uska permanent near-miss record banata hai.
- `send_close_followup(closed_position)`: Trade close hone par result message bhejta hai (sirf agar original signal bheja gaya tha).
- `send_breakeven_notification(position)`: Jab stop-loss break-even par move ho jaye to informational update bhejta hai.
- `sweep_unsent_qualifying_signals()`: Har hourly sweep mein open positions ko re-check karta hai jo pehle qualify nahi hui thi (naya data aane se qualify ho sakti hain).
- `sweep_pending_telegram_retries()`: Failed Telegram sends ko retry karta hai (max 5 attempts, phir abandon karke alert deta hai).
- `_humanize_hours(hours, en)`: Raw hours ko readable form ("about 2 weeks") mein badalta hai.
- `no_signal_alert_status(now_iso, lang)`: 24+ ghante se koi signal na bheja gaya ho to dashboard alert deta hai.

sindhu_web (dashboard + API layer)

sindhu_web (root)

__init__.py

- Khaali file hai, koi package-level code nahi (sirf package marker).

api_monitor.py

- record_request(status_code, duration_seconds) — Har API request ka status code aur time record karta hai (in-memory counters), taake total/failed requests aur average response time track ho sake.
- get_stats() — Total requests, failed requests, aur average response time (seconds) return karta hai. Yeh sirf is app ke apne API ki health hai, exchange ke API ki nahi. Process restart hone par sab zero ho jata hai (deliberately, kyunke yeh permanent audit record nahi, sirf live gauge hai).

auth.py

- has_credentials() — Check karta hai ke kya CEO ne kabhi login account (username/password) banaya hai ya nahi.
- set_credentials(username, password) — Naya account banata ya password update karta hai; password PBKDF2-HMAC-SHA256 (200,000 iterations) se hash hokar store hota hai, kabhi plaintext nahi. Local machine par JSON file mein, cloud (Postgres) par database table mein save hota hai.
- verify_password(username, password) — Login credentials check karta hai, hashed comparison ke zariye (secrets.compare_digest se timing-attack safe).
- change_password(current_password, new_password) — Purana password verify karke naya set karta hai.
- create_session() / is_valid_session(token) / invalidate_session(token) — Login session tokens banata, verify karta, aur logout par cancel karta hai. Session 30 din tak valid rehta hai. Yeh single-user login gate hai jo cloud deployment ke liye zaroori hai (LAN-only check ke upar ek extra layer).

broadcast.py

- publish(message) / get_queue() — Background job threads se WebSocket clients tak messages bhejne ka thread-safe pub/sub bridge. Job thread kisi bhi waqt publish() call kar sakta hai, aur FastAPI app queue ko drain karke har connected client ko forward karta hai.

cache.py

- cached(key, ttl_seconds, compute_fn) — Stale-while-revalidate in-memory cache: agar purana cached value expire ho chuka hai to background thread mein naya compute hota hai jab tak purana value serve hota rehta hai (request kabhi block nahi hota, sirf pehli dafa jab koi cache hi nahi hota).
- cached_nonblocking(key, ttl_seconds, compute_fn, default) — Same jaisa cached() lekin pehli call bhi kabhi block nahi karti — turant default value deti hai aur background mein compute shuru karti hai. Bohot slow computations (jaise disk scan) ke liye.
- invalidate(key) / clear_all() — Ek specific cache entry ya poora cache saaf karta hai (jab data change ho to fresh data dikhane ke liye).

devices.py

- register(ws, ip, user_agent) / unregister(ws) / list_devices() — Control Center ke "Connected Devices" list ke liye — har live WebSocket connection ek "device" hai. Tab band hone par entry automatically remove ho jati hai.

manager_chat.py

- ask(question, lang) — Ek deterministic (bina AI ke) keyword-matched Q&A system, jo real database numbers se jawab deta hai — kabhi guess nahi karta. Urdu ya English mein jawab de sakta hai.
- _answer_best_strategy — "Sabse acchi strategy kaunsi hai" jaisa sawal ka jawab, kam se kam 5 trades wali strategies mein se sabse zyada profit wali dhoondta hai.

- `_answer_today` — "Aaj kya hua" — aaj band hui trades, jeeti hui trades, total PnL, aur Telegram par bheje gaye signals ki ginti.
- `_answer_signals_recent` — Pichle 24 ghanton mein kitne Telegram signals bheje gaye aur aakhri signal kab bheja gaya.
- `_answer_balance_pnl` — Combined balance aur PnL (percentage ke sath) sab strategies mila kar.
- `_answer_locked_strategies` — Kaunsi strategies abhi "locked"/incomplete hain (extraction issues ki wajah se).
- `_answer_maturity` — System ka current maturity level (1-5) batata hai.
- Intent-matching keyword list, jo unrecognized sawal par honest "samajh nahi aaya" jawab deta hai (kabhi guess nahi karta).

module_health.py

- `score_paper_trading()` — Paper Trading module ka 0-100 health score: agar engine chal nahi raha (-50), kill switch active hai (-30), ya drawdown circuit-breaker active hai (-20) to points katey jate hain.
- `score_telegram()` — Telegram module ka score: bot token/channel configure na hone par (-60), sending off hone par (-30), proxy misconfigured hone par (-10).
- `score_database()` — Database health: agar file reachable nahi to 0, warna recent error lines ki tadaad ke hisaab se deduction.
- `score_evolution()` — Evolution Engine health (cloud par None hota hai, sirf local-only feature): running na hona, zyada rollback rate, ya Governor CPU limit exceed hone par points katey jate hain.
- `score_self_learning()` — Self-Learning Engine health (cloud par None): koi discovery cycle na hua ho ya latest cycle fail hua ho.
- `compute_all_module_scores()` — Sab modules ka score ek sath return karta hai, transparent reasons ke sath (kabhi black-box nahi).

network.py

- `get_local_ip()` — LAN IP address dhoondta hai jisse phone/tablet same WiFi par dashboard access kar sake.
- `make_qr_svg(data)` — Dashboard URL ka QR code SVG banata hai taake mobile se scan karke connect kiya ja sake.

security.py

- `token_guard_middleware` — Har HTTP request ke liye main security gate: GET requests hamesha khule hain (LAN par), lekin state-changing (POST/PUT/DELETE) requests ke liye X-Sindhu-Token header zaroori hai.
- `_is_lan_client(host)` — Check karta hai ke request wahi WiFi network se aa rahi hai ya nahi (loopback ya private IP range) — cloud mode mein yeh check bypass hota hai.
- `get_or_create_token()` — Ek per-installation secret token generate/store karta hai jo state-changing requests ko authorize karta hai (local file ya Postgres mein).
- Login-exempt paths list — kaunse paths bina login session ke bhi accessible hain (login page, health check, machine-to-machine sync endpoints).
- `CLOUD_MODE` flag — jab `SINDHU_CLOUD_MODE` set ho, LAN-check bypass hoti hai lekin login-session gate hamesha lagu rehta hai.

server.py

- `create_app()` — Poori FastAPI application banata hai: CORS, middleware (token guard + API monitor), sab routers include karta hai, aur exception handler set karta hai.
- `_broadcast_loop()` — Background thread jo job/log messages ko WebSocket clients tak forward karta hai.
- `_warm_caches()` — Server start hote hi expensive endpoints (market, data, home, correlation, portfolio, strategy summary) ko parallel threads mein pre-compute karta hai, taake pehla user slow response na dekhe.

- `_lifespan()` — App startup par: database init, automation pipeline crash-recovery resume, submission queue restart, Evolution Engine resume, SINDHU Strategy daily scheduler, backup/weekly-snapshot/self-learning/weekly-report/monthly-report/private-report/challenge-snapshot/evolution-weekly-review/strategy-lab/daily-report/telegram-command-polling/plausibility-sweep sab scheduler threads start karta hai, Paper Trading engine ka pehla state restore karta hai, aur startup record karta hai.
- `/api/token` endpoint — CSRF-jaisa X-Sindhu-Token deta hai frontend ko.
- `/login` page — Agar already login hai to dashboard par redirect, warna login form dikhata hai.
- `/` (root) — Session check karke ya to dashboard HTML serve karta hai (cache-busted JS/CSS ke sath) ya `/login` par bhejta hai.
- `_dual_stack_socket(port)` — IPv4 aur IPv6 dono par ek sath listen karne wala socket (Windows par localhost slowness fix karta hai).
- `run(host, port, open_browser)` — Server ko start karta hai, browser khud kholta hai (local use ke liye).

session_guard.py

- `claim(device_id, ws)` / `release(device_id, ws)` — Ek device (browser profile) par sirf ek active WebSocket session allow karta hai — naya tab khulte hi purana tab band ho jata hai, taake do tabs ek dusre se conflict na karein. Alag device (phone) is se unaffected rehta hai.

strategy_aggregate.py

- `compute_strategy_summary()` — Har strategy ka latest completed backtest batch le kar total trades, win rate, net PnL, profit factor, aur worst drawdown ka aggregate summary banata hai — Home dashboard aur Strategy Lifecycle page dono ke liye shared.
- `compute_backtest_metrics_for_strategy(strategy_id)` — Ek strategy ke 7 named comparison metrics (Win Rate, Profit Factor, Max Drawdown, Avg Win, Avg Loss, Total Trades, Net PnL) nikalta hai — Strategy Comparison view ke liye.

sync.py

- `notify(entity, action, message, extra)` — Har mutating API endpoint (strategy/lesson/settings/paper-trading changes) is function ko call karta hai jo Activity Feed mein record karta hai, permanent Audit Trail mein bhi save karta hai, aur WebSocket ke zariye live har connected device ko update bhejta hai — yeh poori app ka central "kuch important hua" signal hai.

sync` root file (`sync.py`) — already covered above.

jobs/ __init__.py

- Khaali file, sirf package marker.

jobs/job_manager.py

- `create_job(kind, target, control, args, kwargs, job_id)` — Background thread mein koi bhi lambi running task (backtest, download, pipeline) start karta hai aur turant `job_id` return karta hai — API kabhi block nahi hoti.
- `get_job(job_id)` / `list_jobs()` / `get_running_job_of_kind(kind)` — Job status check karne ke liye; ek waqt mein sirf ek backtest chalne diya jata hai.
- `update_progress(job_id, fields)` — Job ki progress update karta hai aur WebSocket par broadcast karta hai.
- `make_log_fn(job_id)` — Job-specific log function banata hai jo file aur WebSocket dono mein likhta hai.
- `_prune_finished_locked()` — Purani finished jobs ko memory se hata deta hai (max 200 retain), lekin chal rahi jobs kabhi nahi hataita.
- `Job._estimated_completion()` — Job ki current progress se ETA (kitni der aur lagegi) calculate karta hai, linear-rate projection ke zariye.

sindhu_web/api

__init__.py

- Khaali file, sirf package marker.

activity.py

- GET /api/activity — Recent activity feed (limit=50 default) return karta hai — dashboard par live activity dikhane ke liye.
- GET /api/audit-trail — Permanent, kabhi-na-delete-hone-wala audit trail — entity/date filter ke sath, plus total count.

ai_integration.py

- GET /api/ai/providers — Sab AI providers (OpenAI, Claude, etc.) ki settings dikhata hai (API key masked).
- POST /api/ai/providers/{name}/config — Provider ki settings (model, temperature, quota, cost) update karta hai.
- POST /api/ai/providers/{name}/enable, /disable, /activate — Provider ko on/off ya active karta hai.
- POST /api/ai/providers/deactivate — AI assistance poori tarah band karta hai (rule-based fallback).
- POST /api/ai/providers/{name}/test — Real connection test karta hai provider ke sath.
- POST /api/ai/import — Text se strategy/lesson import karta hai, AI-assisted ya rule-based.
- POST /api/ai/import/upload — File upload se text extract karke import karta hai.
- POST /api/ai/import/youtube — YouTube URL se transcript le kar import karta hai.
- GET /api/ai/usage — Har provider ka daily/monthly usage, quota, aur estimated cost dikhata hai.
- GET /api/ai/logs — AI call logs history.
- POST /api/ai/cache/clear — AI cache saaf karta hai.
- POST /api/ai/import/queue, GET /api/ai/import/queue — Batch import queue mein documents daalta/list karta hai.
- POST /api/ai/import/queue/{item_id}/retry, /retry-failed — Fail hui import ko dobara try karta hai.
- GET /api/ai/knowledge-score/{doc_id} — Compiled document ka quality score.
- GET /api/ai/dictionary — AI-learned terminology dictionary.
- GET /api/ai/dashboard — CEO ke liye AI Center ka poora summary (strategies, lessons, patterns, success rate, hidden rules, learned correction patterns).

auth.py

- GET /api/auth/status — Kya account configure hai aur user login hai ya nahi.
- POST /api/auth/setup — Pehli dafa account banata hai (username+password), turant login session bhi de deta hai.
- POST /api/auth/login — Normal login, session cookie set karta hai.
- POST /api/auth/logout — Session invalidate karke cookie delete karta hai.
- POST /api/auth/change-password — Password change karta hai (purana password verify karke).

automation_pipeline.py

- GET /api/automation/optimizations — Optimizer ne jo comparison records banaye hain unki list.
- POST /api/automation/trigger — Manually ek strategy ke liye poora pipeline (backtest -> optimize -> compare -> paper trading) start karta hai.
- POST /api/automation/submit-batch — Ek sath kai strategies ko queue mein daal deta hai — ek waqt mein ek chalti hai, order mein.
- GET /api/automation/submission-queue — Queue ki current state (kitni pending, kaunsi chal rahi hai).
- GET /api/automation/pipeline-history — Har pipeline run ka permanent record.
- GET /api/automation/pipeline-history/{job_id} — Ek specific run ka stage-by-stage detail.

backtesting.py

- POST /api/backtesting/parse — Free-text strategy description ko structured config mein convert karta hai.

- POST /api/backtesting/strategies — Naya strategy save karta hai ya existing ko update karta hai (name se duplicate detect karke).
- POST /api/backtesting/strategies/similarity-check — Save karne se pehle warn karta hai agar strategy pehle se maujood strategy se multi-julti hai.
- GET /api/backtesting/strategies — Sab strategies ki list (cached), har ek ki status, safety check, last backtest result, extraction lock status, performance verdict sath.
- GET /api/backtesting/strategies/{id}/versions — Strategy ki version history.
- GET /api/backtesting/strategies/{id}/claim-check — Document mein jo win-rate claim ki gayi thi usay real backtest result se compare karta hai.
- GET .../versions/{a}/diff/{b} — Do versions ke darmiyan farq dikhata hai.
- POST .../restore-version — Purani version ko naye version ke roop mein restore karta hai.
- GET /api/backtesting/duplicates — DNA-fingerprint se duplicate strategies ke groups dhoondta hai.
- POST .../archive, /unarchive — Duplicate strategy ko archive (reversible) karta hai — aakhri active copy archive nahi hone deta.
- GET .../extraction-verification — Original text vs samjhi gayi rules ka side-by-side comparison, plain-language mein.
- POST .../extraction-override — "Test anyway" override — locked strategy ko force chalane ki ijazat deta hai (warning ke sath).
- POST .../extraction-audit — Purani strategy ka dobara AI-extraction audit chalata hai.
- GET /api/backtesting/strategies/{id} — Ek strategy ka full detail (config, validity, safety, performance).
- DELETE /api/backtesting/strategies/{id} — Strategy delete karta hai.
- POST .../favourite, /tags, /comment, /duplicate — Strategy ko favourite mark karna, tags set karna, comment add karna, ya duplicate banana.
- GET /api/backtesting/coins — Available trading coins ki list.
- GET /api/backtesting/monte-carlo/{batch_id} — Monte Carlo simulation ek backtest batch par.
- GET /api/backtesting/plausibility/{batch_id} — Batch ka sanity/plausibility check.
- GET /api/backtesting/slippage-sensitivity/{batch_id} — Slippage sensitivity test.
- GET /api/backtesting/duration-stats — Backtest ki duration statistics.
- POST /api/backtesting/what-if — Ek ya zyada parameters change karke fast re-simulation (real config kabhi touch nahi hota).
- POST /api/backtesting/feature-importance — Kaunsi condition strategy ki performance mein sabse zyada asar dalti hai (leave-one-out ablation).
- GET /api/backtesting/cross-coin-validation/{batch_id} — Low/medium/high volatility groups ke across performance validate karta hai.
- POST /api/backtesting/strategy-variants — Strategy ke sibling variants banata aur test karta hai (temporary, real config save nahi hota).
- GET /api/backtesting/stress-test/{id}/{symbol} — Stress test.
- GET /api/backtesting/genetic-optimize/{id}/{symbol} — Grid-search aur genetic optimizer dono side by side.
- GET /api/backtesting/trade-audit/{batch}/{symbol}/{tf}/{num} — Ek specific trade ka manual-verification detail, real candles ke sath.
- GET /api/backtesting/replay/{batch}/{symbol} — Puri backtest run ka bar-by-bar replay.
- POST /api/backtesting/run — Backtest job start karta hai — extraction lock, manual review, aur validation checks pehle karta hai.

backup.py

- create_backup() — Live database ki safe hot-copy backup banata hai (SQLite backup API se, corruption-safe).
- POST /api/backup/create — Manual backup trigger.
- GET /api/backup/list — Available backups ki list.
- POST /api/backup/restore — Backup se database restore karta hai (safety snapshot pehle khud banata hai).

- `start_auto_backup_thread()` — Har kuch ghanton mein automatic backup chalane wala background thread, feature-toggle se control hota hai.
- `_prune_old_backups()` — Purani backups delete karta hai (keep_last limit ke hisaab se).

clarification.py

- `GET .../clarification` — Ek strategy ke saare unclear/missing items structured "issues" ke roop mein dikhata hai (why-asking, why-matters, example ke sath).
- `POST .../clarify` — CEO ke jawab apply karta hai (redescribe, suggested value pick, ya condition reject karna), naya version save karta hai, aur agar ab ready hai to automation pipeline trigger karta hai.
- `build_issues(cfg, hidden_rules)` — Validator errors ko structured, actionable issues mein convert karta hai (raw_condition, missing_conditions, invalid_indicator, missing_field, contradiction, quality_filter_threshold types).
- `_find_impossible_combination_issues` — Do contradictory rules (jaise $RSI < 30$ aur $RSI > 70$ dono zaroori) dhoondta hai jo kabhi ek sath sach nahi ho saktin.
- `_find_unspecified_quality_filter_issues` — "Strong candles" jaisi vague quality checks jinka exact number nahi diya gaya.
- `ambiguity_overview` — Confidence, uncertain aur unresolved issues ka ek deterministic breakdown.
- `POST .../reextract-field` — Ek single field ko dobara original text se extract karta hai (baaki sab fields untouched).
- `POST .../clarify/preview` — Free-text jawab save karne se pehle dikhata hai ke parser ne usay kaise samjha — safety confirmation ke liye.
- `GET /api/backtesting/clarification/all` — Sab strategies ke pending clarification issues ek grouped list mein.
- `GET .../read-mode` — Poori strategy ka plain-language Roman Urdu summary, section-wise.

concepts_usage.py

- `GET /api/concepts/usage` — Har Concepts Library entry (Doji, Hammer, FVG, Order Block, etc.) ko kaunsi active strategies use kar rahi hain.
- `GET /api/concepts/family-tree` — Strategies ko shared concept ke hisaab se groups mein dikhata hai — kaunse strategies "family" bana rahe hain aur kaunse akele hain.

dashboard_scores.py

- `GET /api/module-health` — Har module (paper trading, telegram, database, evolution, self-learning) ka transparent health score.
- `GET /api/project-score` — Ek overall number, reliability/stability/performance/risk categories mein tod kar.
- `GET /api/decision-center` — "Biggest Problem / Recommended Action / Impact / Priority" — real gates se computed, kabhi placeholder nahi (kill switch, drawdown, paused strategies, incomplete locks, low module health).

data.py

- `_default_exchange()` — Configured default exchange return karta hai (Binance cloud par geo-block hone ki wajah se automatically Bybit par switch karta hai).
- `GET /api/data` — Har coin ka candle count, date range, aur status dikhata hai — non-blocking cached (kyunke COUNT query lakhon rows par slow ho sakti hai).
- `GET /api/data/quality` — Har symbol ka data quality score (live, cached 5 min ke liye).
- `POST /api/data/download` — Exchange se historical candle data download karne wala background job.

evolution.py

- `GET /api/evolution/status` — Evolution Engine ki running state, current job, aur resource governor ki details.
- `POST /api/evolution/start, /stop` — Evolution Engine on/off karta hai.
- `POST /api/evolution/run-tick` — Manually ek Analyze->Mutate->Archive->Rank pass chalata hai.
- `GET /api/evolution/strategies` — Evolution-generated (BOT) strategies ki list.
- `GET/POST /api/evolution/settings` — Evolution Mode (conservative/aggressive) aur uski knobs.

- GET .../lineage — Ek strategy ki generation-by-generation lineage history.
- GET .../explain — Lineage ka plain-language narrative explanation.
- GET /api/evolution/ranking — Strategies ki ranking.
- GET /api/evolution/research/dna-correlations — DNA correlations research.
- GET /api/evolution/lessons — Evolution se generated lessons.
- GET /api/evolution/champions — Current champion strategies.
- GET /api/evolution/knowledge-versions — Knowledge versions history.
- GET /api/evolution/comparisons — Har evolution event ka before/after audit trail, confidence score ke sath.
- GET/POST /api/evolution/weekly-reviews[/generate-now] — Weekly tuning/evolution activity ka summary report.
- GET /api/evolution/history[/compare] — Time-window (week/month/etc.) ke hisaab se evolution activity ki history aur period-over-period comparison.

external_signals.py

- GET/POST /api/external-signals/channels — External Telegram channels manage karta hai (add/list) jo CEO follow karta hai.
- POST .../enable, /disable, /rename, DELETE — Channel ki settings badalta hai.
- GET .../messages, POST .../messages/manual — Channel ke raw messages dekhta/manually add karta hai.
- POST /api/external-signals/process-pending — Pending messages ko process (parse) karta hai.
- GET .../signals — Parsed signals ki list.
- POST .../signals/{id}/open-position — External signal se ek paper position khol deta hai.
- POST /api/external-signals/check-price-updates — Live price check karke position updates lagata hai.
- GET .../positions — External channel ki paper positions.
- GET .../report, /comparison — Ek channel ka performance report, ya sab channels ka comparison.
- GET .../eligibility — Kya yeh channel forwarding ke liye eligible hai.
- GET/POST /api/external-signals/settings — Telegram session/API keys aur forwarding settings (secrets kabhi frontend ko show nahi hote).

feature_control.py

- GET /api/feature-control/state — Har automated background feature (Pattern Auto-Avoid, Drawdown Protection, Dynamic Risk Sizing, Lesson Auto-Apply, Capital Allocation, Daily Strategy Generator, AI Trade Review, Telegram Auto-Send, Backup, Weekly/Monthly Reports, etc.) ki on/off state aur live status.
- POST /api/feature-control/toggle — Ek specific feature ko on/off karta hai.
- POST /api/feature-control/master-pause — Sab automated actions ko ek sath pause/resume karta hai (Paper Trading khud chalti rehti hai, sirf extra automation silent hoti hai).

home.py

- NAV_PAGES / NAV_GROUPS / NAV_ICONS — Poori app ki navigation structure, 10 departments mein group ki hui (CEO, Research, Data, Strategy, Trading, Risk, Learning, Reporting, Memory Core, Operations).
- GET /api/nav — Navigation pages aur groups return karta hai (dashboard sidebar ke liye).
- GET /api/strategy-summary — Cross-strategy aggregate summary (cached).
- GET /api/home — Dashboard ka main overview: database status, coin count, candle count, CPU/RAM, running jobs, knowledge score, task summary, module status, disk usage, maturity level, account balance snapshot.
- _account_snapshot() — Combined balance/PnL/win-rate/trades sab strategies mila kar — live Paper Trading account prefer karta hai, warna latest backtest par fallback.

incidents.py

- POST /api/incidents — Naya incident record banata hai (problem -> detection -> root cause -> fix workflow).
- GET /api/incidents, GET /api/incidents/{id} — Incidents list aur detail.
- POST .../update — Root cause, fix description, test description, ya status update karta hai.

- POST .../resolve — Incident ko resolved mark karta hai.
- Har action Activity Feed aur permanent Audit Trail mein bhi log hoti hai.

infra_weekly_digest.py

- generate_infra_weekly_digest() — Hafte ka infrastructure health summary banata hai (backups, incidents opened/resolved, database/disk size) — trading ya evolution reports se alag, sirf system health.
- maybe_generate_infra_weekly_digest() — 7-din gate ke sath auto-trigger, Telegram par bhi bhej deta hai agar enabled ho.
- start_infra_weekly_digest_scheduler_thread() — Har 6 ghante check karne wala background thread.
- GET /api/infra-weekly-digest — Digests ki history.
- POST .../generate-now — Manual trigger (7-din gate bypass karke).

jobs.py

- GET /api/jobs, GET /api/jobs/{id} — Sab running/finished jobs ki list ya ek job ka detail.
- POST /api/jobs/{id}/pause, /resume, /stop — Ek chalti hui job ko control karta hai.

knowledge.py

- GET /api/knowledge/categories — Lesson categories ki list.
- GET /api/knowledge/report — Poore Knowledge system ka aggregate report (cached), knowledge score aur recent lessons ke sath.
- GET /api/knowledge/lessons — Lessons search/filter karta hai.
- GET/PUT/POST/DELETE /api/knowledge/lessons/{id} — Lesson CRUD (create, read, update, delete).
- POST .../duplicate — Lesson duplicate karta hai (disabled state mein).
- POST .../status — Lesson ka status (active/disabled/draft) badalta hai.

knowledge_compiler.py

- POST /api/knowledge-compiler/compile — Raw text se strategy/lesson compile karta hai (structured document mein).
- GET .../documents, GET .../documents/{id} — Compiled documents ki list/detail.
- GET .../concepts — Extracted knowledge concepts.

manager_chat.py` (api wrapper)

- POST /api/manager-chat/ask — sindhu_web.manager_chat.ask() ko call karta hai — question-answer endpoint jo real database data se jawab deta hai.

market.py

- _signal_and_volatility(exchange, symbol) — Har coin ka price vs 20-EMA signal (Bullish/Bearish/Neutral) aur volatility % nikalta hai, cached (5-min bucket).
- GET /api/market — Har coin ka live price, change%, volume, trend, signal, aur volatility dikhata hai — non-blocking cached (kyunke live exchange ticker call 60-130s tak le sakti hai).

memory_core.py

- GET /api/memory-core/checkpoints — Project ke checkpoint JSON files ka readable index — kya task, status, phase, aur next-item-to-resume tha, file size/modified-date ke sath. Read-only.

network.py` (api wrapper)

- GET /api/network — LAN IP, port, connection URL, QR code SVG, aur connected devices ki list — mobile se connect karne ke liye.

paper_trading.py` (biggest file — 2593 lines)

- GET /api/paper-trading/status — Engine ki running state.

- GET .../health-check — Full health check.
- POST .../start, /stop — Paper Trading engine ko chalu/band karta hai (restart ke baad bhi state persist).
- GET/POST /api/paper-trading/kill-switch/status, /activate, /deactivate — Emergency Kill Switch — sab trading foran rok deta hai (positions bhi close kar sakta hai).
- GET .../account-drawdown-status, POST .../account-drawdown-resume — Account-wide drawdown circuit-breaker.
- POST .../run-tick-now, /run-coin-scan-now/{symbol} — Manual single-tick ya single-coin scan trigger.
- GET/POST /api/paper-trading/settings — Dry-run, risk %, max open trades, cooldown, ensemble voting, time-filter, profit-lock jaisi settings.
- GET .../reset-balance/preview, POST .../reset-balance — Balance ko fresh start par reset karta hai (trade history save rehti hai).
- GET .../positions, /trades — Open positions aur closed trades ki list.
- POST .../positions/{id}/close — Ek position manually band karta hai.
- POST .../positions/{id}/note — Trade par note/annotation lagata hai.
- GET .../decisions, /strategy-performance, /lesson-performance — Decision log, per-strategy aur per-lesson performance.
- GET/POST .../strategy-config/{id} — Ek strategy ka Paper Trading config (enabled, priority, supported coins).
- POST .../strategy-config/{id}/pause, /resume — Manual pause/resume, drawdown auto-pause se alag.
- POST .../strategy-config/{id}/overrides — Risk% aur max-open-trades ka per-strategy override.
- POST .../strategy-config/{id}/signal-filters — HTF Confluence Filter, Volume Filter, Trailing Stop opt-in.
- GET/POST .../strategy-config/{id}/reset-stats[/preview] — Ek strategy ke stats reset karta hai (archive karke, delete nahi).
- GET .../strategy-config/{id}/reset-history — Reset history archive.
- GET .../strategy-configs, /strategy-overview — Sab strategies ki paper-trading configuration aur live performance ek sath.
- GET .../periods, /strategy-periods/{id} — Time-period vocabulary (today/yesterday/7d/15d/30d/all) aur per-strategy period stats.
- GET .../strategy-comparison/export — Excel export (.xlsx) strategy comparison ka.
- GET/POST .../cloud-sync/status, /run-now, /download — Cloud se local ko 24h backup snapshot sync.
- GET/POST .../strategy-sync/secret, /target, /trigger/{id}, /log, /status, /push — Local machine se cloud ko strategy config push karne ka poora mechanism (secret-token gated, machine-to-machine).
- GET .../cloud-status-for-auto-stop — Local ko batata hai ke cloud kya kar raha hai (auto-stop decision ke liye).
- GET/POST .../cloud-aware-auto-stop/state, /resume — Agar cloud already dono (paper trading + telegram) on hain to local khud ko pause kar leta hai; resume sirf manual click se.
- GET/POST .../status-ping/state, /send-now — Private Telegram status ping.
- GET .../dashboard-badge-status — Har page ke top par badge (engine, live-candles, telegram status).
- GET .../analytics — Poora analytics dashboard (period-wise summary, per-coin, per-strategy, best/worst, alerts).
- GET/POST .../override/{id}, GET .../overrides — Manual Telegram alert override.
- GET .../session-stats, /hour-of-day-stats — Trading session aur hour-of-day ke hisaab se performance split.
- GET .../coin-stats/{id}, /coin-heatmap, /coin-deep-dive/{symbol} — Coin-level performance analysis.
- GET .../alerts — Auto-generated alerts.
- GET/POST/DELETE .../custom-alert-rules[/]{id}/enabled] — Custom alert rules banana/manage karna.
- GET .../pattern-memory, /balance-history/{id}, /confluence-history/{id} — Coin-pattern memory, virtual balance history over time, confluence score history.
- GET .../pattern-reliability — Wilson 95% confidence interval based statistical pattern reliability.
- GET .../lesson-candidates — Auto-detected potential lessons (review ke liye, kabhi auto-apply nahi).
- GET .../streak/{id}, /genealogy/{id} — Win/loss streak, aur strategy ki poori history timeline (versions + auto-avoid + auto-lessons events milakar).

- GET .../readiness/{id} — Kya strategy real trading ke liye ready hai.
- GET/POST .../auto-avoid-rules/{id}/deactivate] — Pattern Auto-Avoid rules (5 consecutive losses par auto-pause).
- GET/POST .../auto-lessons/{id}/deactivate] — Auto-applied lessons.
- GET/POST/DELETE /api/paper-trading/goals/{id}] — Goal System (target metric set karke progress track karna).
- GET/POST .../maintenance-mode/enter, /exit — Maintenance Mode (trading + telegram dono pause).
- GET/POST .../auto-downgrade/state, /check-now — Auto-Downgrade Rule check.
- GET .../paused-strategies, POST .../resume/{id} — Drawdown Protection se paused strategies ko resume karta hai.
- GET .../risk-metrics/{id}, /risk-metrics-all — Risk analytics per strategy.
- GET/POST .../ai-trade-review/settings, GET .../ai-trade-review/{id} — AI-written one-sentence trade review.
- GET .../capital-allocations, POST .../recompute-now — Sharpe-Ratio-based dynamic capital allocation.
- GET .../risk-pct-recommendations — Optimal risk % suggestion per strategy.
- GET .../regime, /regime/{symbol} — Market regime (trending/ranging) classification.
- GET .../correlation-warnings, /strategy-correlation-matrix — Price aur PnL correlation warnings.
- GET .../portfolio, /portfolio-risk-score, /coin-exposure, /duplicate-exposure-warnings, /strategy-exposure, /direction-exposure — Portfolio-level risk/exposure analytics.
- GET .../strategy-profile/{id} — Ek strategy ka poora profile (health score, Sharpe, MAE/MFE, backtest metrics).
- GET/POST .../telegram/settings, /channel-override/{id}, /test, /test-proxy, /log, /alert-status, /send/{id} — Telegram bot poori configuration aur manual controls.
- GET .../telegram/signals, /analytics, /delivery-log, /near-misses, /connection-status, /preview/{id} — Telegram Dashboard page ka data — sent signals, analytics, delivery status, near-miss log, connection health, message preview.
- GET .../signal-tracker/feed, /match-table — Live Signal Tracker feed aur backtest-vs-paper-vs-telegram match table.
- GET/POST /api/paper-trading/challenge[/clear] — Single Challenge Mode (start amount -> target amount tracking).
- GET .../challenge/breakdown, /best-portfolio — Per-strategy/coin breakdown aur best portfolio suggestion.
- POST .../challenge/recommend — Real historical-data-based recommended paths to target.
- GET/POST /api/paper-trading/challenges/{id}/archive, /extend] — Multi-Challenge system (kai challenges parallel track karna).
- GET .../challenges/{id}/full-analysis — Consolidated analysis (difficulty, risk suggestion, give-up-point, loss-streak impact, best historical period, AI explanation).
- POST .../challenges/replay — Historical challenge replay.
- GET .../challenges/rotation-suggestion — Do complementary strategies suggest karta hai.
- GET .../confluence/{id}, /signal-detail/{id} — Retroactive confluence score aur poora signal detail (grade, explanation, freshness check).
- GET .../graveyard, POST .../graveyard/check-similarity, GET .../retirement-suggestions — Retired/archived strategies aur auto-retirement suggestions.
- GET .../signal-history, /active-signals — Poori signal history (running/win/loss/cancelled) aur live open positions ke sath current price/unrealized PnL.
- GET .../export-center/trades.csv, /trades.json, /trade-journal/export-pdf — Trades ko CSV/JSON/PDF format mein export karta hai.
- GET/POST/DELETE .../coin-blacklist — Coins ko permanently exclude karna.
- GET/POST/DELETE .../coin-priority[/pin, /demote] — Coins ko priority dena ya demote karna.
- POST .../position-size-calculator — Position size calculate karta hai (pure calculation, koi real trade nahi).
- GET .../weekly-reports, POST .../weekly-reports/generate-now — Weekly performance report.

- POST .../private-report/send-now, GET .../private-report/status — Private (personal chat) PDF performance report.
- GET .../trade-audit/{id} — Ek position ka poora manual-verification detail, real candles ke sath.

project_meta.py

- MODULE_DEPENDENCY_MAP — Har module (Data Engine, Backtest Engine, Strategy Library, Paper Trading, Telegram, Evolution, Self-Learning, Database) ka dependency aur "agar yeh down ho jaye to kya hoga" documentation.
- GET /api/project-meta/dependency-map — Yeh map return karta hai.
- GET /api/project-meta/readiness — Har feature category (Risk & Safety, Signals, Self-Learning, Other) ka readiness label (Stable/Production/Testing) — honest heuristic, deep telemetry se nahi.

project_status.py

- GET /api/compare-strategies — Sab strategies side-by-side, optimizer se pehle/baad ke comparison ke sath.
- GET /api/compare-strategies/dual-tp — 16 original strategies vs unke Fixed-1:2-TP variants ka comparison.
- GET /api/live-logs — Running jobs, queued jobs (hamesha empty by design), recently completed jobs, aur recent activity — ek jagah.
- GET /api/what-changed — Audit trail se ek period ka genuine, automatic activity diff.
- GET /api/session-handoff — Narrative summary jo session end par copy-paste karke handoff note ban sake.
- GET /api/project-status — Changelog, pending items, engine-gap tracker stats, concepts library completion — poora project overview.
- POST/GET /api/feedback/{id}/status — User feedback box (Suggest/Add/Fix/Wrong types).
- POST/GET/DELETE /api/quick-notes/{id} — Instant scratchpad notes (no workflow, koi status nahi).

report_builder.py

- POST /api/report-builder — Custom date-range report banata hai, jisme CEO khud modules choose kar sakta hai (paper_trading, telegram, evolution) — real aggregate functions reuse karta hai.

reports.py

- GET /api/reports — Recent backtest batches ki list.
- GET /api/backtest-history — Har completed batch ka summary, optimization comparison ke sath.
- POST .../rename — Batch ka display name badalta hai (sirf label, data untouched).
- GET .../comparison — Original vs Optimized comparison (winner aur reason ke sath).
- GET /api/reports/best-worst/strategies — Har strategy ke sirf LATEST batch se ranking (purane batches se poison nahi hota).
- GET /api/reports/{batch_id} — Poora backtest report.
- GET .../condition-reports — 0-trade results ke liye har condition kitni baar true hui, diagnosis ke sath.
- GET .../trades — Batch ki trades list (Equity Curve chart ke liye).
- GET .../export/{fmt} — CSV/Excel/PDF export.

research.py

- _check_rate_limit() — Autonomous research ki daily rate limit check karta hai.
- POST /api/research/search — Web search karke matching articles ko import queue mein daalta hai (rate-limited, on-demand only).
- POST /api/research/queue-url — Ek specific URL ko direct queue karta hai.
- GET .../trusted-sources — Trusted domains ki list.
- GET/POST .../settings — Daily research run limit.
- GET .../runs — Research run history.
- GET .../web-sourced-strategies — Sab strategies jo web research se aayi hain, source URL ke sath.

risk_department.py

- `get_risk_department_summary()` — Har safety gate (Kill Switch, Account Drawdown, Paused Strategies, Coin Blacklist, Confluence Threshold, Signal Freshness, Wilson Statistical Gate, Position Cap, Incomplete Lock, Evolution/Rollback Gate) ki current live state — ek jagah, read-only.
- `GET /api/risk-department` — Yeh summary endpoint se serve hota hai — koi naya risk logic nahi, sirf existing gates ka consolidated view.

search.py

- `GET /api/search` — Global search — coins, strategies, lessons, backtest reports, aur trades ek sath dhoondta hai.

self_learning.py

- `GET /api/self-learning/status` — Discovery cycle chal raha hai ya nahi, aur naya cycle chalega ya nahi.
- `GET .../cycles, /attempts` — Cycle history aur attempts.
- `GET .../combination-scores` — Live evidence jisse engine combinations ko score karta hai.
- `POST .../run-now` — Manual discovery cycle trigger (fire-and-forget, do cycles ek sath nahi chalne dete).
- `start_self_learning_scheduler_thread()` — Weekly automatic discovery cycle scheduler.

settings.py

- `GET/POST /api/settings` — Global app settings: exchange, quote asset, num_coins, theme, refresh speed, default risk%. Postgres free-tier expiry notice bhi dikhata hai.

sindhu_strategy.py

- `GET /api/sindhu-strategy/today` — Aaj generate hui candidate strategies.
- `GET .../daily-log` — Ek din ka generation log.
- `GET .../candidates` — Sab SINDHU-generated candidates (AI ya deterministic origin).
- `GET .../candidates/{id}` — Ek candidate ka detail.
- `POST .../generate` — Manually daily generation cycle trigger karta hai (safe no-op agar aaj ke 11 already ban chuke hon).
- `POST .../candidates/{id}/backtest` — Candidate ko poore backtest pipeline se guzaarta hai.

strategy_lab.py

- `GET /api/strategy-lab/latest` — Sabse haal ka honest finding — sirf genuinely profitable strategy, kabhi weak strategy ko "best" dikha kar nahi.
- `GET .../history` — Purane scans ki history.
- `POST .../scan-now` — Manual scan trigger.
- `POST .../approve` — Ek finding ko approve karna — YEH HI EKLOTA raasta hai jisse Strategy Lab ka result live Paper Trading/Telegram mein feed hota hai (kabhi automatic nahi).

strategy_lifecycle.py

- `GET /api/strategy-lifecycle` — Har active strategy ka ek row: backtest result, why-win/why-loss summary, optimizer variants (Medium/Strict), paper-trading config, discrete lifecycle stage (Created -> Paper Trading -> Performance Analysis -> Promoted/Evolution-Escalated/Needs-Optimization -> Paused/Archived), evolution summary.
- `_compute_lifecycle_stage` — Purely derived stage naam, koi new state machine nahi — hamesha real data se hi banta hai.
- `compute_failure_reasons(id)` — Ek losing strategy ke liye real breakdown — kaunse coins par aur kis exit reason se loss hua.
- `GET .../{id}/failure-reasons` — Yeh breakdown endpoint se serve hota hai.

system.py

- POST /api/system/restart-services — Sab in-memory caches clear karta hai (server process restart nahi karta, sirf soft reset).
- POST /api/system/client-diagnostics — Browser se viewport/device info le kar log karta hai (mobile CSS debugging ke liye).
- _recent_errors() — Log file ke tail se error lines nikalta hai.
- _active_background_processes() — Kaunse background jobs/engines abhi chal rahe hain.
- GET /api/system/api-monitor — API request/failure/latency stats.
- GET /api/system/health — Live system health (uptime, CPU, RAM, database size, active processes, recent errors, last restart time).
- record_startup() — Har server start par ek record likhta hai aur ek private Telegram restart-notification bhejta hai.
- GET /api/system/restart-analytics — Server ke restart history aur gaps (honestly note karta hai ke yeh crash detection nahi hai).

time_machine.py

- GET /api/time-machine/{date} — Ek din ke real audit-trail events dikhata hai, plus us date ke sabse qareeb weekly snapshot — honestly clarify karta hai ke yeh poor point-in-time state reconstruct nahi karta.

timeline_compare.py

- GET /api/timeline-compare — Current period ko previous period se compare karta hai (today/week/month), delta ke sath.

weekly_snapshot.py

- create_weekly_snapshot() — Har hafte database ki ek genuinely-weekly-cadence snapshot (rolling 6-hourly backup se alag retention policy).
- maybe_create_weekly_snapshot() / start_weekly_snapshot_scheduler_thread() — 7-din gate ke sath automatic scheduling.
- GET /api/weekly-snapshot/list, POST ../create-now — Snapshots dekhna aur manual trigger.
- GET ../compare — Do snapshots ka comparison (closed trades, realized PnL, open positions ka diff).

wizard.py

- GET /api/wizard/concept-library — Wizard dropdowns ke liye live concept/indicator vocabulary (validator.py se, kabhi out-of-sync nahi hota).
- POST /api/wizard/classify-other — Optional AI call: free-text description ko kisi known concept se match karta hai — agar AI available nahi to gracefully manual-review par fallback.
- POST /api/wizard/save — Wizard-built strategy save karta hai — NEVER REJECT policy: Manual Review items ke sath bhi save hoti hai, sirf execution se exclude hoti hain jab tak resolve na ho.

ws.py

- WS /ws/logs — Ek WebSocket channel jisme live log lines, job events, progress updates, aur sync events sab aate hain. LAN-check aur login-session check dono yahan explicitly hain (kyunke HTTP middleware WebSocket par nahi chalta). Device-based single-session guard bhi lagu hota hai (naya tab purane ko replace kar deta hai).

evolution_engine / automation_pipeline / dashboard / cloud_runtime / scripts

evolution_engine (17 files)

evolution_engine/ __init__.py

- Package docstring only — Ye sirf documentation hai, koi code nahi. Evolution Engine ka purpose bata raha hai: sirf deterministic logic use karta hai, koi AI/ML call nahi karta, aur BOT-owned strategies/lessons ko user ke apne strategies se physically alag rakhta hai.

champion.py

- `_best_bucket(history, key_fn, min_sample)` — Trades ko kisi bhi dimension (coin/session/timeframe) ke hisaab se group karke best performing bucket nikalta hai, win-rate aur PnL ka weighted composite score use karke.
- `recompute_champions(now_iso)` — Har category (strategy, generation, lesson, coin, session, timeframe, market_condition) ka current "best performer" recompute karta hai aur naya champion_records row save karta hai — read-only hai, kabhi purane champion ko delete nahi karta.
- `current_champions()` — Dashboard ke liye har category ka latest champion return karta hai.

config.py

- `load() / save() / update()` — Evolution Engine ke settings (evolution_mode) ko JSON file ya Postgres cloud_settings mein load/save karta hai, dashboard se editable.
- `current_mode_params()` — Teen modes (conservative/balanced/aggressive) ke parameters return karta hai — conservative default hai jisme sirf achi performing strategies ko hi auto-generations milte hain (max 3), balanced top-25 lineages ko allow karta hai (max 5), aggressive unrestricted hai (max 25, purana behavior).

dna.py

- `extract_dna(config)` — Kisi strategy ke indicators/concepts/filters dekh kar usko DNA categories (trend, momentum, liquidity, volume, breakout, session, risk) mein tag karta hai — pure lookup logic hai, AI nahi.
- `concepts_for_dna(tag)` — Kisi DNA category ke saare known concepts/indicators ki list deta hai, jo naya candidate banate waqt use hoti hai.
- `dna_overlap(dna_a, dna_b)` — Do strategies ke shared DNA tags nikalta hai, taake mutator duplicate lineages avoid kar sake.

engine.py

- `EvolutionEngine.start()/stop()/is_running()` — Background thread mein chalne wala continuous evolution loop start/stop karta hai, crash ke baad resume bhi kar sakta hai.
- `_checkpoint()` — Har tick ka progress durable DB row mein save karta hai taake server crash hone par resume ho sake.
- `_loop()` — Har ~5 minute (jitter ke sath, taake dusre background systems se collide na ho) tick chalata hai.
- `_tick()` — Ek full Analyze→Mutate→Archive→Rank cycle — Governor ke resource checks (CPU/RAM) pass hone par hi kaam karta hai, evolution mode ke hisaab se eligible lineages choose karta hai, unhe mutate karta hai, underperformers archive karta hai, aur champions recompute karta hai.
- `_backtest_untested_candidates()` — Naye generate hue BOT candidates ko real backtest dene ke liye chota sa batch chalata hai (1 per tick), taake unke paas evolve hone ke liye trade data ho.
- `status()` — Engine ki current running state aur Governor stats return karta hai.
- `resume_evolution_jobs_on_startup()` — Server restart hone par jo evolution job "running" reh gaya tha usko wapas resume karta hai.

evolution_confidence.py

- `compute_confidence(comparison)` — Ek evolution comparison (before/after generation) kitna trustworthy hai uska score (0-100) nikalta hai — sample size (kitne trades), improvement margin (kitna farq aaya), aur metric coverage (kitne metrics compare ho paye) ke weighted sum se.

failed_hypothesis_memory.py

- `regressed_dna_history(limit)` — Pichle saare rolled-back (regressed) generations ke DNA tags collect karta hai — matlab jo mutations fail ho chuke hain unki memory.
- `matches_a_known_failure(candidate_dna, history)` — Naye candidate ka DNA check karta hai ke kahin ye pehle fail ho chuki kisi lineage se bohat milta-julta to nahi — agar 3+ DNA tags match karein to flag karta hai, taake wahi galti dobara na ho.

generation_manager.py

- `next_strategy_base_id() / next_lesson_base_id()` — Naye BOT strategy/lesson lineage ke liye agla sequential ID (BOT_S001, BOT_L001, etc) nikalta hai.
- `create_new_strategy_lineage()` — Bilkul nayi strategy lineage ki Generation 1 banata hai.
- `create_next_strategy_generation()` — Kisi existing lineage ki agli generation banata hai, max generation cap check karke.
- `create_new_lesson_lineage() / create_next_lesson_generation()` — Same pattern lessons ke liye.
- `lineage_history(base_id)` — Kisi strategy lineage ki poori generation history (audit trail) return karta hai.

governor.py

- `Governor.check_resources() / resource_ok()` — Real CPU/RAM usage psutil se check karta hai, agar limit se zyada ho to naya kaam start nahi hone deta.
- `wait_for_resources()` — Resources free hone tak wait karta hai (max timeout ke sath).
- `try_start_experiment() / reset_run_budget()` — Har tick mein kitne mutation experiments chal sakte hain uska hard cap enforce karta hai (max 5).
- `enqueue()/try_enqueue()/dequeue()/clear_queue()` — Priority queue jisme lineages ko unke score ke hisaab se process karne ke liye rakha jata hai — bounded size (20), full hone par sirf better item worst ko replace karta hai.
- `stats()` — Dashboard ke liye Governor ki current state (CPU%, RAM%, queue size, experiment count) return karta hai.

history.py

- `compute_evolution_history(window)` — Kisi time window (week/15d/month/longer) mein evolution activity ka summary banata hai: kitni strategies evolve hui, kitne rollbacks hue, before/after metrics ka average.
- `compare_periods(window)` — Do consecutive periods ka comparison deta hai (jaise ye mahina vs pichla mahina) — pure read-only reporting.

lesson_generator.py

- `analyze_and_generate_lessons(closed_position, now_iso)` — Har closed paper-trade ke baad strategy ki history dekh kar automatically lessons banata hai: coin performance ("X coin par ye strategy accha/bura karti hai"), session performance, aur market-regime performance — pure statistical analysis hai, koi story-telling nahi.

lineage_explainer.py

- `explain_lineage(base_id)` — Kisi strategy lineage ki poori history ko plain-language narrative mein convert karta hai — kaunsi generation kab bani, kyun bani, improve hui ya rollback hui — sab already-existing data se assemble karta hai.

market_regime.py

- `detect_regime(exchange, symbol, timeframe)` — OHLCV data se market ka current regime detect karta hai: trending_up, trending_down, ranging, ya volatile — ATR aur % price change ke fixed rules se.

- `adapt_params_for_regime(config_dict, regime)` — Detected regime ke hisaab se strategy ke `risk_pct/risk_reward/breakeven/stop_loss` parameters ko chote se fixed factor se adjust karta hai — sirf wahi fields jo already configured hain.

mutator.py

- `is_matured(base_id, max_generations)` — Check karta hai ke kisi lineage ne apni max generation cap chhu li hai ya last 2 generations mein improvement plateau ho gaya — agar haan to auto-mutation rok deta hai.
- `is_eligible_for_mode(backtest_summary, mode, mode_params)` — Conservative mode ka minimum bar (Profit Factor ≥ 0.7 YA ≥ 25 trades) check karta hai.
- `rank_strategies(base_id)` — Saari scored BOT strategies ko best-first order mein rank karta hai.
- `archive_underperformers(now_iso)` — Jo strategies enough trades ke baad bhi kam score kar rahi hain unhe archive (delete nahi) kar deta hai.
- `research_dna_correlations()` — Kaunsa DNA tag combination (jaise Liquidity+Session) higher score se correlate karta hai, ye analysis karta hai — deterministic builder ke liye guidance.
- `compare_generations(base_id)` — Ek lineage ki saari generations side-by-side compare karta hai.
- `regime_context_for(base_id)` — Lineage ke latest real backtest se exchange/symbol/timeframe nikalta hai taake regime-aware mutation ho sake.
- `mutate_strategy(base_id, governor, now_iso)` — Ek strategy lineage ki agli generation banata hai — weakest score component ya market regime ke hisaab se sirf EK parameter change karta hai (one-change-at-a-time rule), 100-trade gate pass karne ke baad hi chalta hai.

rollback.py

- `effective_generation(base_id)` — Kaunsi generation abhi "active" hai (rollback ke baad pehle wali bhi ho sakti hai) ye nikalta hai.
- `should_evolve(base_id, generation_row)` — Check karta hai ke kya lineage ne naya 100-trade threshold cross kiya hai jo abhi evolve nahi hua.
- `record_evolution_event()` — Nayi generation banne ke turant baad before-metrics record karta hai aur nayi generation ko active mark karta hai.
- `_is_regression(before, after)` — 4 core metrics (win_rate, PnL, profit factor, drawdown) mein se majority (3/4) worse ho to "regression" declare karta hai.
- `try_finalize_comparison(bot_strategy_id)` — Naye generation ke enough trades hone ke baad verdict deta hai (improved/regressed) aur agar regress hui to automatically pichli generation par rollback kar deta hai.

scoring.py

- `compute_evolution_score(stats)` — Har BOT strategy ka 0-100 evolution score compute karta hai — 11 alag components (win_rate, profit_factor, net_profit, avg_rr, drawdown, trade_count, stability, consistency, session/coin/market performance) ka fixed-weight sum, poori formula transparent/auditable hai.
- `time_decay_weights(n, half_life)` — Purane data ko kam weight, recent data ko zyada weight deta hai (exponential decay) — purana data delete nahi hota, sirf kam important treat hota hai.
- `weighted_average(values, weights)` — Simple weighted average helper.

weekly_review.py

- `generate_weekly_review()` — Har hafte Evolution/Tuning activity ka plain-language summary banata hai (kitni strategies evolve hui, kitne rollback hue, before/after win-rate/PnL).
- `maybe_generate_weekly_review()` — Har 7 din mein sirf ek baar review generate karta hai aur Telegram par bhi bhej deta hai.
- `start_evolution_weekly_review_scheduler_thread()` — Server startup par background thread start karta hai jo har 6 ghante check karta hai ke naya weekly review due hai ya nahi.

automation_pipeline (6 files)

automation_pipeline/__init__.py

- Empty file — Koi code nahi, sirf package marker hai.

genetic_optimizer.py

- `_random_genome(dims, rng) / _apply_genome()` — Random parameter combination (genome) banata hai aur strategy config par apply karta hai.
- `run_genetic_optimization()` — Grid-search optimizer ka genetic-algorithm alternative — population of parameter combinations lekar, generations ke through crossover+mutation se best combination dhoondta hai, baseline (current) values ko hamesha population mein rakhta hai taake result kabhi baseline se worse na ho.

optimizer.py

- `_run_in_memory(config, exchange, symbol, ...)` — Ek symbol par, database mein kuch save kiye bagair, fast in-memory backtest chalata hai — quick screening ke liye.
- `_run_in_memory_bounded()` — Same as upar, lekin timeout (90 sec) ke sath — agar koi candidate atak jaye to skip kar deta hai, poora pipeline hang nahi hota.
- `_score(metrics)` — Candidate ka fitness score (profit %) nikalta hai, minimum trade count floor ke sath.
- `tunable_dimensions(config)` — Strategy ke already-configured numeric fields (indicator periods, stop-loss %, risk-reward, risk%, session filter, confirmation lookback) ko tunable dimensions ki list mein convert karta hai, candidate values ke sath.
- `_numeric_ordered_directions(dim)` — Baseline se upar/neeche dono directions mein candidates ko sort karta hai, taake early-elimination outward search ho sake.
- `optimize(config, exchange, screen_symbol, ...)` — Pure math-based coordinate-search optimizer — ek-ek dimension change karke best combination dhoondta hai, early elimination se worthless candidates skip karta hai, progress callback aur stop control ke sath.

pipeline.py

- `trigger_pipeline_for_strategy()` — Jab bhi koi strategy import "Backtesting Ready" ho jati hai, automatically backtest→optimize→compare→paper-trading pipeline start kar deta hai, background job ke roop mein.
- `resume_pipeline_jobs_on_startup()` — Server restart ke baad jo pipeline job "running" reh gaya tha usko resume karta hai.
- `_resume_one_pipeline_job()` — Ek specific interrupted job ko wapas start karta hai.
- `run_pipeline(job_id, strategy_id, ...)` — Poora 4-stage pipeline chalata hai: (1) Backtesting original strategy, (2) Optimizer se better settings dhoondhna aur validate karna, (3) Comparison save karna aur winner ko strategy library mein save karna, (3.5) Walk-Forward Test se overfitting check karna, (4) Paper Trading mein winning version handoff karna — har stage checkpoint hoti hai taake crash ke baad resume ho sake, plain-language live logs ke sath.

submission_queue.py

- `enqueue() / enqueue_batch()` — Ek ya multiple strategies ko pipeline queue mein daalta hai taake koi bhi strategy pipeline busy hone ki wajah se skip/lost na ho.
- `queue_status()` — Dashboard ke liye queue ki current state (kitne pending, kaunsa abhi process ho raha hai) deta hai.
- `_wait_for_job(job_id)` — Ek pipeline job ke complete hone tak poll karke wait karta hai.
- `_process(item)` — Queue ke ek item ka pipeline trigger karta hai aur uska result wait karta hai.
- `_worker_loop() / ensure_worker_running()` — Background daemon thread jo continuously queue se items claim karke process karta hai, ek time par sirf ek.

walk_forward.py

- `compute_split(exchange, symbol, train_fraction)` — Symbol ki पूरी history ko chronologically 70% training / 30% testing mein split karta hai (shuffle nahi karta, kyunki future data check karna hai).
- `_verdict(training, testing)` — PASS/FAIL/INCONCLUSIVE verdict deta hai — agar testing period training se profit ka 50%+ retain kare to PASS, warna overfitting ka signal.

- `run_walk_forward_test(config, exchange, symbol, ...)` — Poora walk-forward test chalata hai: training period par optimize karke, wahi parameters testing period (jo optimizer ne kabhi nahi dekha) par test karta hai — ye check karta hai ke strategy genuinely acchi hai ya sirf purane data par fit ki gayi hai (overfitting detect karta hai).

dashboard (14 files)

dashboard/__init__.py

- Empty file — Sirf package marker.

backtest_tab.py

- `QuickBacktestTab` — Original simple single-timeframe backtest UI — strategy select karo, coins/timeframes check karo, backtest chalao, live progress dekho.
- `_load_last_batch_id()` / `_save_last_batch_id()` — Interrupted backtest ko resume karne ke liye last batch ID JSON file mein save/load karta hai.
- `BacktestingTab` — Poora Backtesting page — 5 sub-tabs ko combine karta hai: Quick Backtest, Strategy Builder, Strategy Library, Trade History, Rankings & Queue, tab change hone par relevant data refresh karta hai.

backtest_worker.py

- `BacktestWorker(QThread)` — GUI ko block kiye bagair background thread mein poora backtest batch chalata hai, progress/trade updates Qt signals ke through UI ko bhejta hai.

candlestick_item.py

- `CandlestickItem` — pyqtgraph ke liye custom candlestick chart drawing widget — green/red candles draw karta hai based on open vs close.

main_window.py

- `MainWindow` — App ki main window — Data Engine tab (download control, session logging, disk usage) aur Backtesting tab ko host karta hai.
- `on_start/on_pause/on_resume/on_stop` — Coin data download worker ko control karta hai.
- `_refresh_stats()` — Har 5 second mein database size, disk usage refresh karta hai.
- `_record_session()` — Session start/stop events ko JSON log file mein append karta hai.

mtf_backtest_worker.py

- `MTFBacktestWorker(QThread)` — Multi-timeframe strategy config ko multiple coins par background thread mein backtest karta hai, `run_mtf_batch` use karke.

queue_worker.py

- `QueueWorker(QThread)` — Multiple library strategies ko ek ke baad ek queue mein chalata hai (`backtest_engine.queue_runner` use karke).

rankings_tab.py

- `RankingsTab` — Coin/Timeframe/Session ranking tables dikhata hai kisi batch ke liye.
- `_on_export_csv/_on_export_excel/_on_export_pdf` — Ek click mein results ko CSV/Excel/PDF format mein export karta hai.
- Backtest Queue section — Library se multiple strategies check karke ek queue banate hain jo back-to-back chalti hai (`QueueWorker` use karke), start/stop control ke sath.

settings_dialog.py

- `SettingsDialog` — CEO ko exchange, coin count, quote asset, history depth, aur enabled timeframes edit karne deta hai — JSON config files mein directly save karta hai, next restart par apply hota hai.

strategy_builder_tab.py

- StrategyBuilderTab — Plain-English/Roman-Urdu mixed strategy text paste karo → parse karo → preview + validate karo → backtest chalao.
- on_parse() — Strategy text ko parse_strategy_text se parse karke config banata hai aur validate karta hai.
- _render_preview(cfg, errors) — Parsed strategy ka human-readable preview dikhata hai (indicators, entry/exit rules, SL/TP, errors/warnings ke sath).
- on_save_library() — Parsed strategy ko library mein save karta hai.
- load_from_library(strategy_id) — Library se ek saved strategy load karke text box mein wapas dikhata hai.
- on_start() — Validated strategy ko multi-timeframe backtest ke liye run karta hai (MTFBacktestWorker use karke), full history ya custom date range ke sath.

strategy_library_tab.py

- StrategyLibraryTab — Saved strategies ko browse/search/favourite/rename/duplicate/delete karne ka UI, version history ke sath.
- refresh() — Search query ke hisaab se list refresh karta hai.
- _on_favourite/_on_rename/_on_duplicate/_on_delete — Har button ka respective library action call karta hai.

trade_history_tab.py

- TradeHistoryTab — Kisi batch ke saare trades ko sortable table mein dikhata hai — columns mein entry/exit price, SL/TP, PnL, reasons.
- _on_row_double_clicked() — Row par double-click karne se Trade Replay Dialog khulta hai.

trade_replay_dialog.py

- TradeReplayDialog — Ek specific trade ke around candlestick chart dikhata hai (entry se pehle aur exit ke baad ke candles), SL/TP levels ke horizontal lines ke sath, aur plain-English explanation ke sath ke trade kyun khula aur kyun band hua.

cloud_runtime (2 files)

cloud_runtime/_init_.py

- Package docstring only — Bata raha hai ke ye SINDHU ka lightweight, cloud-deployable hissa hai jo apna alag FastAPI app banata hai, local app ka heavy code import kiye bagair.

app.py

- create_app() — Cloud ke liye ek chota, minimal FastAPI app banata hai — sirf Paper Trading engine, Telegram signals, aur login-gated dashboard include karta hai, poora local app (backtesting/evolution/AI) nahi laata.
- _lifespan() — App startup par: database initialize karta hai, Paper Trading engine resume karta hai (agar pehle chal raha tha), Telegram bot status log karta hai, cloud-sync scheduler, Telegram command polling, Challenge Mode snapshots, status ping, aur auto-downgrade scheduler — sab background threads start karta hai.
- _broadcast_loop() — WebSocket clients ko real-time messages broadcast karta hai.
- _token_guard middleware — Login session check karta hai har request par.
- _api_monitor_middleware — Har API request ka response time aur status code track karta hai.
- health_check() endpoint (/health) — External uptime pinger (jaise cron-job.org) ke liye — cloud_mode, db_backend type, aur process start time return karta hai taake pata chale ke server sach mein zinda hai ya nahi.
- get_nav() — Cloud dashboard ki navigation pages list return karta hai — sirf wahi pages jo is lightweight runner par actually kaam karte hain (Paper Trading, Strategies, Telegram Signals, Signal Tracker, Challenge Mode, Self-Learning, Risk, Memory Core).
- login_page() / index() — Login page aur main dashboard page serve karta hai, session cookie check karke.

scripts (49 files — one-off / utility scripts)

Master 15-Item task / New Batch 5 strategy builders & runners

- `_tmp_activate_new_batch_5.py` — New Batch 5 ke saare 23 strategy variants ko validator+safety-check se pass karake Paper Trading mein activate karta hai, sirf fixed list ke IDs ke liye (baaki library ko touch nahi karta).
- `_tmp_build_strategy1.py` — Strategy 1 (Liquidity Sweep + Engulfing Candle) ke Loose aur Strict variants manually build karke library mein save karta hai.
- `_tmp_build_strategy2.py` — Strategy 2 (FRVP Market Shape) ka ek variant manually build karta hai.
- `_tmp_build_strategy3.py` — Strategy 3 (S/R + Liquidity Sweep) ke Structure aur Fixed-RR variants build karta hai.
- `_tmp_build_strategy4.py` — Strategy 4 (FVG Momentum Pullback) ke 3 take-profit variants (Structure, Fixed 1:2, Fixed 1:3) build karta hai.
- `_tmp_build_strategy5.py` — Strategy 5 (FVG Pure + Inverse) ke 2 variants build karta hai.
- `_tmp_build_strategy6.py` — Strategy 6 (Order Block Trading) ke Loose/Strict variants build karta hai.
- `_tmp_build_strategy7.py` — Strategy 7 (Candle Range Theory) ke Loose/Strict variants build karta hai.
- `_tmp_build_strategy8.py` — Strategy 8 (BOS/CHoCH Structure Break + Retest) ka single variant build karta hai.
- `_tmp_build_strategy9.py` — Strategy 9 (Ichimoku Cloud System) ke saare 8 combinations (4 timeframes × 2 exit modes) build karta hai.
- `_tmp_check_s9_status.py` — Database se Ichimoku Indicator-Exit batches ki recent status query karta hai.
- `_tmp_diag_strategy2.py` — Strategy 2 (FRVP) ke 0-trades bug ko diagnose karta hai — column values aur concept matches inspect karke.
- `_tmp_item2_item3_smoke_verify.py` — Item 2 (null take-profit bug) aur Item 3 (continuation-strategy fix) ka combined single-coin smoke test verification chalata hai.
- `_tmp_item2_rerun.py` — Item 2 fix se pehle/baad ka comparison — fixed list of strategies ko poore 50-coin batch mein re-run karta hai.
- `_tmp_item3_smoke_verify.py` — Item 3 fix ke liye representative strategies ka single-coin (BTCUSDT) smoke test chalata hai.
- `_tmp_resume_s9_remaining.py` — Interrupt hue Strategy 9 ke bache hue 3 indicator-exit variants ko resume karke poora backtest run karta hai (Windows spawn-multiprocessing guard ke sath).
- `_tmp_run_batch_strategy1.py` to `_tmp_run_batch_strategy9.py` (9 files) — Har New Batch 5 strategy ke saare variants ka poora 50-coin backtest chalata hai aur `batch_id` JSON checkpoint file mein save karta hai.
- `_tmp_smoke_strategy1.py` to `_tmp_smoke_strategy9.py` (9 files) — Har strategy ke liye BTCUSDT par quick single-symbol sanity check (Engine Health Report) chalata hai, poore 50-coin run se pehle.

Maintenance / one-off operational scripts

- `archive_and_reset_paper_trading.py` — Purane Paper Trading data (positions, performance) ko timestamped JSON file mein archive karta hai, open positions ko neutral close karta hai, aur rolling aggregates (PnL totals) ko zero par reset karta hai — history kabhi delete nahi hoti.
- `backfill_condition_roles.py` — Purani saved strategies mein missing `Condition.role` field ko unke `raw_text` se deterministically re-derive karke backfill karta hai (dry-run default, `--apply` se save hota hai).
- `backfill_strategy_safety_check.py` — Library ki saari strategies par Automatic Safety Check dobara chalata hai aur unke `meta.json` `safety_status` ko refresh karta hai.
- `deploy_ready_strategies_to_paper_trading.py` — Jo bhi strategies "Ready" status mein hain (validator + safety check pass) unhe Paper Trading ke liye enable kar deta hai.
- `migrate_to_postgres.py` — Local SQLite database ke sirf curated tables (paper trading + Telegram related) ko Postgres (cloud) database mein copy karta hai, idempotent (dobara chalane se duplicate nahi hota) aur read-only source ke sath.
- `reextract_clarification_strategies.py` — Jo strategies "NEEDS_CLARIFICATION" mein atki hain unko dobara genuine AI extraction se process karta hai (cache clear karke), self-correction aur verification ke sath.
- `reextract_library_sentence_level.py` — Poori library ko naye sentence-level AI extraction pipeline se dobara process karta hai, DNA fingerprint compare karke sirf genuinely changed strategies ko naya version save karta hai.
- `run_all_ready_backtests.py` — Library ki saari "Ready" strategies par BTCUSDT ka poora Engine Health Report backtest chalata hai, progress ko JSON file mein continuously save karta hai.

- `run_engine_health_report.py` — Kuch fixed target strategies par Engine Health Report (Strategy/Data/Execution/PnL/Trade/Statistics verification) chalata hai aur print karta hai.
- `run_verification.py` — Fixed target strategies par Verification Engine chalata hai — rule coverage aur trade validation check karta hai.
- `run_walk_forward_demo.py` — Ek real saved strategy par Walk-Forward Test ka live demonstration chalata hai aur result ko strategy library mein save karta hai.
- `self_correct_library.py` — Library ki saari failing strategies ko 3-level Self-Correcting Pipeline (deterministic repair → targeted AI call → user flag) se guzarta hai, dry-run default ke sath.
- `strategy_batch_report_helper.py` — Ek completed backtest batch ke saare per-symbol results ko pool karke ek combined report banata hai, aur Why-Win/Why-Loss classifier lagata hai har trade par.
- `uptime_watchdog.py` — Cloud deployment ka /health endpoint periodically ping karta hai (external watchdog, scheduled task se), agar down ho to CEO ko Telegram DM alert bhejta hai (3 consecutive failures ke baad), aur recovery par bhi inform karta hai.
- `verify_mtf_enforcement.py` — Multi-timeframe enforcement fix ko verify karta hai — role-stripped (before) vs role-backfilled (after) versions ka comparison backtest chalata hai, aur real trades trace karke dikhata hai ke higher-timeframe data actually decision mein use ho raha hai.

SECTION 2 -- Data Storage / Database Schema

Ye SQLite database (data/database/sindhu.db) mein actually maujood tables hain -- seedha live schema se nikale gaye hain, kisi documentation summary se nahi. Har table ke real columns aur is waqt uske andar kitni rows hain, dono diye gaye hain.

Total tables in live database: 80

Table Name	Rows	Key Fields / Columns
_lock_probe	0	id
account_drawdown_state	1	id, peak_balance, paused, paused_reason, paused_at, updated_at
activity_log	500	id, entity, action, message, created_at
ai_dictionary_entries	52	canonical_name, definition, keywords_json, category, source_doc_id, created_at, updated_at, aliases_json, examples_json, related_concepts_json, usage_notes
ai_import_cache	5	content_hash, ai_result_json, provider, created_at
ai_import_queue	0	id, title, source_hint, filename, raw_text, use_ai, status, result_doc_id, ai_assisted, ai_provider, error_message, processing_time_ms, created_at, started_at, finished_at, input_kind, content_type
ai_usage_log	2519	id, provider, model, endpoint, status, tokens_in, tokens_out, latency_ms, error_message, created_at
audit_trail_log	222	id, entity, action, message, created_at
backtest_batches	397	batch_id, strategy_name, exchange, settings_json, status, created_at, updated_at, display_name, extraction_override_warning
backtest_condition_reports	1522	batch_id, symbol, timeframe, report_json, created_at
backtest_results	10898	batch_id, symbol, timeframe, status, metrics_json, completed_at
backtest_trades	3413625	batch_id, symbol, timeframe, trade_num, side, entry_time, entry_price, exit_time, exit_price, size, pnl, pnl_pct, exit_reason, stop_loss, take_profit, risk_amount, reward_amount, entry_reason, entry_type, gross_pnl, commission_cost, slippage_cost, spread_cost, is_partial
bot_lessons	144	id, base_id, generation, parent_id, title, category, description, derived_from_json, conditions_json, status, confidence, created_at, updated_at
bot_strategies	263	id, base_id, generation, parent_id, name, config_json, dna_json, origin, made_with_ai, status, evolution_score, score_breakdown_json, backtest_summary_json, mutation_reason, created_at, updated_at
challenge_achievability_snapshots	0	id, challenge_id, achievability_score, recorded_at
challenges	2	id, label, start_amount, target_amount, timeframe_type, days, started_at, scope_strategy_id, scope_symbol, baseline_win_rate_pct, telegram_report_enabled, compounding, archived, created_at, updated_at
champion_records	1238	id, category, value, score, details_json, computed_at
compiled_documents	20	id, title, source_type, doc_type, classification_confidence, status, raw_text, sections_json, strategy_ids_json, lesson_ids_json, concepts_json, unresolved_json, clarification_notes_json, tags_json, created_at, ai_assisted, ai_provider, hidden_rules_json, psychology_notes_json, deep_knowledge_json
confluence_score_log	985	id, strategy_id, position_id, confluence_ratio, created_at
custom_alert_rules	0	id, name, metric, strategy_id, comparison, threshold, enabled, created_at, last_triggered_at
daily_generation_log	22	date, ai_calls_used, candidates_generated, updated_at
download_progress	50	exchange, symbol, last_open_time, status, updated_at
evolution_comparisons	19	id, base_id, parent_id, child_id, trade_threshold, before_json, after_json, verdict, rolled_back, created_at, updated_at
evolution_jobs	2	job_id, status, stage, checkpoint_json, error, created_at, updated_at

Table Name	Rows	Key Fields / Columns
evolution_trade_gates	19	base_id, last_threshold_evolved, active_generation_id, updated_at
evolution_weekly_reports	1	id, period_start, period_end, report_json, report_text, created_at
external_channel_performance	0	channel_id, balance, trades, wins, losses, total_pnl, total_rr_sum, updated_at
external_channels	0	id, name, telegram_identifier, enabled, forwarding_source_label, created_at, updated_at
external_messages	2	id, channel_id, telegram_message_id, content_type, raw_text, raw_media_path, received_at, processed, process_error, created_at
external_positions	1	id, channel_id, signal_id, symbol, direction, entries_json, avg_entry_price, filled_size_pct, stop_loss, take_profit_json, exit_price, pnl, pnl_pct, exit_reason, status, opened_at, closed_at, created_at
external_signals	2	id, message_id, channel_id, is_signal, reject_reason, symbol, direction, entries_json, stop_loss, take_profit_json, leverage, parsed_by, parsed_at
extraction_fidelity_reports	4	id, content_hash, strategy_id, expected_rule_count, captured_rule_count, call_count, retry_count, provider, rules_json, created_at, updated_at, overridden
incidents	0	id, title, problem, detected_by, severity, status, root_cause, fix_description, fix_reference, test_description, detected_at, resolved_at, created_at, updated_at
infra_weekly_digests	1	id, period_start, period_end, report_json, report_text, created_at
kill_switch_state	0	id, active, reason, close_positions, activated_at, activated_by, deactivated_at, deactivated_by, activation_count
klines_1m	29124439	exchange, symbol, open_time, open, high, low, close, volume, close_time, quote_volume, trades
knowledge_concepts	43	canonical_name, category, aliases_json, usage_count, first_seen_at, last_seen_at
knowledge_relationships	764	id, from_type, from_id, to_type, to_id, relation, created_at
knowledge_versions	380	version, reason, snapshot_json, created_at
lesson_applications	22488596	id, lesson_id, batch_id, symbol, timeframe, applied_at, outcome
lesson_stats_summary	10	lesson_id, times_used, trades_approved, trades_rejected
lessons	29	id, title, category, description, priority, status, notes, apply_backtesting, apply_paper_trading, apply_evolution, rule_type, direction, conditions_json, created_at, updated_at, version, tags_json, supported_market_types_json, supported_timeframes_json
near_miss_log	440	id, position_id, strategy_id, strategy_name, symbol, confluence_ratio, confluence_passed, confluence_total, confluence_required_ratio, confluence_required_count, pattern_status, pattern_trades, pattern_required, pattern_win_rate_pct, live_pnl, reason, created_at
paper_account_state	55	strategy_id, realized_pnl_total, closed_count, win_count, updated_at
paper_alerts	102	id, alert_type, strategy_id, strategy_name, message, severity, created_at
paper_auto_avoid_rules	0	id, strategy_id, strategy_name, symbol, market_state, session, consecutive_losses, reason, active, triggered_at, deactivated_at
paper_auto_lessons	0	id, strategy_id, strategy_name, symbol, market_state, session, influence, sample_size, win_rate, explanation, active, applied_at, deactivated_at
paper_balance_resets	0	id, strategy_id, reset_at, previous_realized_pnl_total, open_positions_left_running
paper_coin_blacklist	0	symbol, reason, added_at
paper_coin_priority	0	symbol, priority, reason, added_at
paper_decision_log	2000	id, exchange, symbol, direction, decision, reason, strategy_id, strategy_name, lesson_ids_json, confidence, market_state, session, timeframe, position_id, market_snapshot_json, created_at
paper_downgrade_state	0	strategy_id, downgraded, profit_factor, sample_size, reason, updated_at
paper_lesson_candidates	0	id, strategy_id, strategy_name, symbol, market_state, session, pattern_description, sample_size, win_rate, total_pnl, status, created_at

Table Name	Rows	Key Fields / Columns
paper_lesson_performance	9	lesson_id, lesson_title, usage_count, wins, losses, total_pnl, confidence_avg, score, updated_at
paper_monthly_reports	1	id, period_start, period_end, report_json, report_text, created_at
paper_positions	985	id, exchange, symbol, direction, entry_price, exit_price, stop_loss, take_profit, size, risk_amount, entry_time, exit_time, pnl, pnl_pct, exit_reason, entry_reason, strategy_id, strategy_name, strategy_version, lesson_ids_json, confidence, market_snapshot_json, tags_json, session, timeframe, market_state, lifecycle_json, reflection_json, status, created_at, closed_at, lowest_price_seen, highest_price_seen, user_note
paper_strategy_config	63	strategy_id, enabled, priority, supported_coins_json, supported_market_types_json, updated_at, paused, paused_reason, paused_at, capital_multiplier, capital_multiplier_reason, risk_pct_override, max_open_trades_override, htf_confluence_filter_enabled, volume_spike_filter_enabled, trailing_stop_enabled
paper_strategy_overrides	0	strategy_id, manual_alert, note, updated_at
paper_strategy_performance	55	strategy_id, strategy_name, trades, wins, losses, total_pnl, avg_rr, score, updated_at
paper_strategy_stat_archives	0	id, strategy_id, archived_at, previous_realized_pnl_total, previous_closed_count, previous_win_count, open_positions_left_running
paper_trade_reviews	0	id, position_id, review_text, provider, created_at
paper_weekly_reports	3	id, period_start, period_end, report_json, report_text, created_at
pipeline_jobs	7	job_id, strategy_id, strategy_name, symbols_json, status, stage, checkpoint_json, error, created_at, updated_at
pipeline_submission_queue	7	id, strategy_id, strategy_name, symbols_json, status, job_id, error, created_at, started_at, finished_at
quick_notes	0	id, content, created_at
research_run_log	2	id, kind, query_or_url, queued_count, run_at
self_learning_attempts	0	id, dna_combo_json, concepts_drawn_json, variant, strategy_id, outcome, reason, discovery_metrics_json, validation_metrics_json, created_at
self_learning_cycles	1	id, started_at, completed_at, status, report_json, created_at
server_restart_log	2	id, deployment, started_at
sqlite_sequence	30	name, seq
strategies	90	name, file_path, added_at
strategy_extraction_corrections	0	id, strategy_id, corrected_at_version, previous_expected_count, previous_captured_count, new_expected_count, new_captured_count, reason, corrected_at
strategy_graveyard	1	id, strategy_id, strategy_name, reason_category, reason_detail, concepts_used_json, buried_at
strategy_lab_scans	3	id, scanned_at, strategies_checked, qualifying_strategy_id, qualifying_strategy_name, qualifying_win_rate, qualifying_pnl, qualifying_trade_count, approved, approved_at
strategy_optimizations	5	id, strategy_id, original_batch_id, optimized_batch_id, winner, params_changed_json, candidates_tried_json, created_at
symbols	50	exchange, symbol, added_at
telegram_message_log	50	id, position_id, strategy_id, strategy_name, trigger_type, message_text, success, error, sent_at, explanation_text, quality_grade, grade_reason
telegram_retry_queue	0	id, position_id, trigger_type, high_confidence, status, attempts, last_error, created_at, last_attempt_at
user_feedback	0	id, type, text, status, created_at
user_goals	1	id, metric, target_value, comparison, scope_strategy_id, label, archived, created_at, achieved_at

SECTION 3 -- Strategy Inventory

Ye strategies/library/ (real strategy library, jahan har save hui strategy ek folder ke andar meta.json + version files ke sath store hoti hai) se li gayi hai -- Status column wahi live logic use karke calculate kiya gaya hai jo Strategies page khud use karta hai (validator + safety check + paper-trading enabled/paused state).

Total strategies: 154 (Active: 53 | Inactive: 79 | Ready: 22)

Strategy Name	Added On	Status	Concepts Used
Supply/Demand Zone Strategy [Manual Build]	2026-08-23	Active	valid_structure_trend, demand_zone, supply_zone, resistance
Candle Range Theory (CRT) Strategy 2.0 [Manual Build]	2026-08-23	Ready	liquidity_sweep, sweep_invalidation_state, support, resistance, fvg, valid_structure_trend
Richard Dennis Turtle Trader Strategy [Manual Build]	2026-08-23	Active	(none detected)
Kotegawa Bear Market Reversal Strategy [Manual Build]	2026-08-23	Active	support, resistance
Double Confirmation CHoCH with Liquidity Trap [Manual Build]	2026-08-23	Ready	support, resistance, choch, liquidity_sweep, double_choch_confirmation, order_block
Daily Liquidity Scalping Strategy [Manual Build]	2026-08-23	Active	pdh, pdl, candle_break
Lower Time Frame Liquidity Reversal Strategy [Manual Build]	2026-08-23	Ready	support, resistance, liquidity_sweep, volume, rejection_block, engulfing
Support/Resistance Breakout (Swing/Crypto Futures) [Manual Build]	2026-08-24	Ready	support, resistance, valid_structure_trend
4-Hour Range Breakout-Retest [Manual Build]	2026-08-24	Active	four_hour_range_reentry
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) [Manual Build]	2026-08-24	Active	(none detected)
9-20 EMA SMC Hybrid [Manual Build]	2026-08-24	Active	choch, bos, volume, support, resistance
Asian Range London Sweep [Manual Build]	2026-08-24	Active	asian_range_sweep_reclaim, support, resistance, valid_structure_trend
Candlestick Pattern Reversal Strategy [Manual Build]	2026-08-24	Active	candlestick_patterns, engulfing, support, resistance, valid_structure_trend
Liquidity Sweep Reversal Strategy [Manual Build]	2026-08-24	Active	liquidity_sweep_reclaim, valid_structure_trend, support, resistance
Laxman Rekha 5-EMA Trigger-Candle Strategy [Manual Build]	2026-08-25	Active	laxman_trigger
Dumb Money Concepts -- Confirmation Entry [Manual Build]	2026-08-25	Active	dmc_confirmation
Richard Dennis Turtle Trader Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	(none detected)
Liquidity Sweep Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	liquidity_sweep_reclaim, valid_structure_trend, support, resistance
Candlestick Pattern Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	candlestick_patterns, engulfing, support, resistance, valid_structure_trend
Support/Resistance Breakout (Swing/Crypto Futures) -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	support, resistance, valid_structure_trend

Strategy Name	Added On	Status	Concepts Used
Dumb Money Concepts -- Confirmation Entry -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	dmc_confirmation
Double Confirmation CHoCH with Liquidity Trap -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	support, resistance, choch, liquidity_sweep, double_choch_confirmation, order_block
Supply/Demand Zone Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	valid_structure_trend, demand_zone, supply_zone, resistance
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	(none detected)
Asian Range London Sweep -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	asian_range_sweep_reclaim, support, resistance, valid_structure_trend
9-20 EMA SMC Hybrid -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	choch, bos, volume, support, resistance
4-Hour Range Breakout-Retest -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	four_hour_range_reentry
Laxman Rekha 5-EMA Trigger-Candle Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	laxman_trigger
Kotegawa Bear Market Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	support, resistance
Daily Liquidity Scalping Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	pdh, pdl, candle_break
Lower Time Frame Liquidity Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	support, resistance, liquidity_sweep, volume, rejection_block, engulfing
Candle Range Theory (CRT) Strategy 2.0 -- Fixed 1:2 TP variant [Manual Build]	2026-08-25	Inactive	liquidity_sweep, sweep_invalid_state, support, resistance, fvg, valid_structure_trend
Dumb Money Concepts -- Blind Entry [Manual Build]	2026-08-25	Active	dmc_blind_entry
Dumb Money Concepts -- Combined Confirmation [Manual Build]	2026-08-25	Active	dmc_combined
Fair Value Gap Reversal [Manual Build]	2026-08-26	Active	fvg, fvg_reversal, valid_structure_trend, support, resistance
Order Block Reversal [Manual Build]	2026-08-26	Ready	order_block, order_block_reversal, support, resistance
Equal Highs/Lows Reversal [Manual Build]	2026-08-26	Active	equal_highs_low, eqhl_reversal, support, resistance
Previous High/Low Reversal [Manual Build]	2026-08-26	Active	pdh, pdl, pdhl_reversal, support, resistance
Market Structure Shift Reversal [Manual Build]	2026-08-26	Active	valid_structure_trend, mss_reversal, support, resistance
4-Hour Range Breakout-Retest -- Confirmation Medium variant [Manual Build]	2026-08-26	Inactive	four_hour_range_reentry
4-Hour Range Breakout-Retest -- Confirmation Strict variant [Manual Build]	2026-08-26	Inactive	four_hour_range_reentry
Candlestick Pattern Reversal Strategy -- Confirmation Medium variant [Manual Build]	2026-08-26	Inactive	candlestick_patterns, engulfing, support, resistance, valid_structure_trend
Candlestick Pattern Reversal Strategy -- Confirmation Strict variant [Manual Build]	2026-08-26	Inactive	candlestick_patterns, engulfing, support, resistance, valid_structure_trend
Laxman Rekha 5-EMA Trigger-Candle Strategy -- Confirmation Medium variant [Manual Build]	2026-08-26	Inactive	laxman_trigger

Strategy Name	Added On	Status	Concepts Used
Laxman Rekha 5-EMA Trigger-Candle Strategy -- Confirmation Strict variant [Manual Build]	2026-08-26	Inactive	laxman_trigger
Dumb Money Concepts -- Confirmation Entry -- Confirmation Medium variant [Manual Build]	2026-08-26	Inactive	dmc_confirmation
Dumb Money Concepts -- Confirmation Entry -- Confirmation Strict variant [Manual Build]	2026-08-26	Inactive	dmc_confirmation
Supply/Demand Zone Strategy -- Confirmation Medium variant [Manual Build]	2026-08-26	Inactive	valid_structure_trend, demand_zone, supply_zone, resistance
Asian Range London Sweep -- Confirmation Medium variant [Manual Build]	2026-08-26	Inactive	asian_range_sweep_reclaim, support, resistance, valid_structure_trend
Asian Range London Sweep -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	asian_range_sweep_reclaim, support, resistance, valid_structure_trend
Liquidity Sweep Reversal Strategy -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	liquidity_sweep_reclaim, valid_structure_trend, support, resistance
Liquidity Sweep Reversal Strategy -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	liquidity_sweep_reclaim, valid_structure_trend, support, resistance
Previous High/Low Reversal -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	pdh, pdl, pdhl_reversal, support, resistance
Previous High/Low Reversal -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	pdh, pdl, pdhl_reversal, support, resistance
Kotegawa Bear Market Reversal Strategy -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	support, resistance
Kotegawa Bear Market Reversal Strategy -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	support, resistance
Equal Highs/Lows Reversal -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	equal_highs_lows, eqhl_reversal, support, resistance
Equal Highs/Lows Reversal -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	equal_highs_lows, eqhl_reversal, support, resistance
Fair Value Gap Reversal -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	fvg, fvg_reversal, valid_structure_trend, support, resistance
Fair Value Gap Reversal -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	fvg, fvg_reversal, valid_structure_trend, support, resistance
Dumb Money Concepts -- Blind Entry -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	dmc_blind_entry
Dumb Money Concepts -- Blind Entry -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	dmc_blind_entry
Richard Dennis Turtle Trader Strategy -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	(none detected)
Richard Dennis Turtle Trader Strategy -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	(none detected)
9-20 EMA SMC Hybrid -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	choch, bos, volume, support, resistance
Market Structure Shift Reversal -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	valid_structure_trend, mss_reversal, support, resistance
Market Structure Shift Reversal -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	valid_structure_trend, mss_reversal, support, resistance
Order Block Reversal -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	order_block, order_block_reversal, support, resistance
Order Block Reversal -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	order_block, order_block_reversal, support, resistance

Strategy Name	Added On	Status	Concepts Used
Candle Range Theory (CRT) Strategy 2.0 -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	liquidity_sweep, sweep_invalidation_state, support, resistance, fvg, valid_structure_trend
Candle Range Theory (CRT) Strategy 2.0 -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	liquidity_sweep, sweep_invalidation_state, support, resistance, fvg, valid_structure_trend
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	(none detected)
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	(none detected)
Daily Liquidity Scalping Strategy -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	pdh, pdl, candle_break
Daily Liquidity Scalping Strategy -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	pdh, pdl, candle_break
Support/Resistance Breakout (Swing/Crypto Futures) -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	support, resistance, valid_structure_trend
Support/Resistance Breakout (Swing/Crypto Futures) -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	support, resistance, valid_structure_trend
Dumb Money Concepts -- Combined Confirmation -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	dmc_combined
Dumb Money Concepts -- Combined Confirmation -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	dmc_combined
Double Confirmation CHoCH with Liquidity Trap -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	support, resistance, choch, liquidity_sweep, double_choch_confirmation, order_block
Double Confirmation CHoCH with Liquidity Trap -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	support, resistance, choch, liquidity_sweep, double_choch_confirmation, order_block
Lower Time Frame Liquidity Reversal Strategy -- Confirmation Medium variant [Manual Build]	2026-08-27	Inactive	support, resistance, liquidity_sweep, volume, rejection_block, engulfing
Lower Time Frame Liquidity Reversal Strategy -- Confirmation Strict variant [Manual Build]	2026-08-27	Inactive	support, resistance, liquidity_sweep, volume, rejection_block, engulfing
HTF Key Level Engulfing Reversal [Manual Build]	2026-08-27	Ready	htf_key_level_engulfing, engulfing
HTF Key Level Engulfing Reversal -- Confirmation Medium variant [Manual Build]	2026-08-28	Inactive	htf_key_level_engulfing, engulfing
Sniper Headshot Entry [Manual Build]	2026-08-28	Active	sniper_headshot_entry, liquidity_sweep, demand_zone, supply_zone, pin_bar, valid_structure_trend
Sniper Headshot Entry -- Confirmation Medium variant [Manual Build]	2026-08-28	Inactive	sniper_headshot_entry, liquidity_sweep, demand_zone, supply_zone, pin_bar, valid_structure_trend
Sniper Headshot Entry -- Confirmation Strict variant [Manual Build]	2026-08-28	Inactive	sniper_headshot_entry, liquidity_sweep, demand_zone, supply_zone, pin_bar, valid_structure_trend

Strategy Name	Added On	Status	Concepts Used
PDH/PDL Multi-Timeframe Reversal [Manual Build]	2026-08-28	Active	pdhl_mtf_reversal, engulfing, pin_bar
SAR + SMC (CISD Entry) [Manual Build]	2026-08-28	Active	cisd_entry, demand_zone, supply_zone, support, resistance
PDH/PDL Multi-Timeframe Reversal -- Confirmation Medium variant [Manual Build]	2026-08-28	Inactive	pdhl_mtf_reversal, engulfing, pin_bar
4H Fractal Sweep Reversal [Manual Build]	2026-08-28	Ready	fractal_sweep_reversal
SAR + SMC (CISD Entry) -- Confirmation Medium variant [Manual Build]	2026-08-28	Inactive	cisd_entry, demand_zone, supply_zone, support, resistance
SAR + SMC (CISD Entry) -- Confirmation Strict variant [Manual Build]	2026-08-28	Inactive	cisd_entry, demand_zone, supply_zone, support, resistance
4H Fractal Sweep Reversal -- Confirmation Medium variant [Manual Build]	2026-08-28	Inactive	fractal_sweep_reversal
Liquidity Sweep Multi-Confirmation [Manual Build]	2026-08-28	Active	liquidity_sweep_multi_confirm, support, resistance
Liquidity Sweep + CISD (Pure Swing Variant) [Manual Build]	2026-08-28	Active	liquidity_sweep_cisd_swing, support, resistance
OTE Liquidity Sweep Reversal [Manual Build]	2026-08-28	Active	ote_liquidity_sweep_reversal
Liquidity Sweep Multi-Confirmation [Medium Variant]	2026-08-28	Inactive	liquidity_sweep_multi_confirm, support, resistance
Liquidity Sweep Multi-Confirmation [Strict Variant]	2026-08-28	Inactive	liquidity_sweep_multi_confirm, support, resistance
Liquidity Sweep + CISD (Pure Swing Variant) [Medium Variant]	2026-08-28	Inactive	liquidity_sweep_cisd_swing, support, resistance
Liquidity Sweep + CISD (Pure Swing Variant) [Strict Variant]	2026-08-28	Inactive	liquidity_sweep_cisd_swing, support, resistance
OTE Liquidity Sweep Reversal [Medium Variant]	2026-08-28	Inactive	ote_liquidity_sweep_reversal
OTE Liquidity Sweep Reversal [Strict Variant]	2026-08-28	Inactive	ote_liquidity_sweep_reversal
HTF Trend Trendline Breakout [Manual Build]	2026-08-31	Active	trendline_breakout, support, resistance
Range Breakout Volume Confirmation [Manual Build]	2026-08-31	Active	range_breakout_volume_confirm, support, resistance
HTF-LTF FVG/OB Confluence Entry [Manual Build]	2026-08-31	Active	htf_ltf_fvg_ob_confluence, order_block, fvg, support, resistance
Range Breakout Volume Confirmation -- Medium Variant [Manual Build]	2026-08-31	Inactive	range_breakout_volume_confirm, support, resistance
Range Breakout Volume Confirmation -- Strict Variant [Manual Build]	2026-08-31	Inactive	range_breakout_volume_confirm, support, resistance
HTF-LTF FVG/OB Confluence Entry -- Medium Variant [Manual Build]	2026-08-31	Inactive	htf_ltf_fvg_ob_confluence, order_block, fvg, support, resistance
HTF-LTF FVG/OB Confluence Entry -- Strict Variant [Manual Build]	2026-08-31	Inactive	htf_ltf_fvg_ob_confluence, order_block, fvg, support, resistance
HTF Trend Trendline Breakout -- Medium Variant [Manual Build]	2026-08-31	Inactive	trendline_breakout, support, resistance
HTF Trend Trendline Breakout -- Strict Variant [Manual Build]	2026-08-31	Inactive	trendline_breakout, support, resistance
Heikin Ashi Trend Reversal [Manual Build]	2026-08-31	Active	heikin_ashi_reversal, support, resistance
Fibonacci Golden Zone Retracement [Manual Build]	2026-08-31	Active	fibonacci_golden_zone, support, resistance
FRVP POC Reversal [Manual Build]	2026-08-31	Active	frvp_poc_reversal, engulfing, pin_bar, support, resistance

Strategy Name	Added On	Status	Concepts Used
FVG 50% Equilibrium Entry [Manual Build]	2026-08-31	Active	fvg_equilibrium_entry, fvg, support, resistance
Donchian LWTI Volume Confluence [Manual Build]	2026-08-31	Active	donchian_lwti_volume_confluence, support, resistance
Heikin Ashi Trend Reversal [Manual Build] [Medium]	2026-08-31	Ready	heikin_ashi_reversal, support, resistance
Heikin Ashi Trend Reversal [Manual Build] [Strict]	2026-08-31	Ready	heikin_ashi_reversal, support, resistance
Fibonacci Golden Zone Retracement [Manual Build] [Medium]	2026-08-31	Ready	fibonacci_golden_zone, support, resistance
Fibonacci Golden Zone Retracement [Manual Build] [Strict]	2026-08-31	Ready	fibonacci_golden_zone, support, resistance
FRVP POC Reversal [Manual Build] [Medium]	2026-08-31	Ready	frvp_poc_reversal, engulfing, pin_bar, support, resistance
FRVP POC Reversal [Manual Build] [Strict]	2026-08-31	Ready	frvp_poc_reversal, engulfing, pin_bar, support, resistance
FVG 50% Equilibrium Entry [Manual Build] [Medium]	2026-08-31	Ready	fvg_equilibrium_entry, fvg, support, resistance
FVG 50% Equilibrium Entry [Manual Build] [Strict]	2026-08-31	Ready	fvg_equilibrium_entry, fvg, support, resistance
Donchian LWTI Volume Confluence [Manual Build] [Medium]	2026-08-31	Ready	donchian_lwti_volume_confluence, support, resistance
Donchian LWTI Volume Confluence [Manual Build] [Strict]	2026-08-31	Ready	donchian_lwti_volume_confluence, support, resistance
Liquidity Sweep + Engulfing Candle (Loose) [Manual Build]	2026-09-01	Ready	liquidity_sweep_engulfing_loose, engulfing
Liquidity Sweep + Engulfing Candle (Loose) [Manual Build]	2026-09-01	Active	liquidity_sweep_engulfing_loose, engulfing
Liquidity Sweep + Engulfing Candle (Strict) [Manual Build]	2026-09-01	Active	liquidity_sweep_engulfing_strict, engulfing
Fixed Range Volume Profile -- Market Shape [Manual Build]	2026-09-01	Active	frvp_hvn_reaction, frvp_lvn_breakout
Support/Resistance + Liquidity Sweep (Structure) [Manual Build]	2026-09-01	Ready	sr_liquidity_sweep_sideways
Support/Resistance + Liquidity Sweep (Fixed-RR) [Manual Build]	2026-09-01	Active	sr_liquidity_sweep_sideways, sr_sweep_tp_fixed_rr
FVG Momentum Pullback (Structure) [Manual Build]	2026-09-01	Active	fvg_momentum_pullback
FVG Momentum Pullback (Fixed 1:2) [Manual Build]	2026-09-01	Active	fvg_momentum_pullback
FVG Momentum Pullback (Fixed 1:3) [Manual Build]	2026-09-01	Active	fvg_momentum_pullback
FVG Pure + Inverse FVG (Structure) [Manual Build]	2026-09-01	Active	fvg_pure_inverse
FVG Pure + Inverse FVG (Fixed 1:2) [Manual Build]	2026-09-01	Active	fvg_pure_inverse
Order Block Trading (Loose) [Manual Build]	2026-09-01	Ready	order_block_trading_loose
Order Block Trading (Strict) [Manual Build]	2026-09-01	Ready	order_block_trading_strict
Order Block Trading (Loose) [Manual Build]	2026-09-01	Ready	order_block_trading_loose, support, resistance
Order Block Trading (Strict) [Manual Build]	2026-09-01	Active	order_block_trading_strict, support, resistance
Candle Range Theory (Loose) [Manual Build]	2026-09-02	Active	crt_loose

Strategy Name	Added On	Status	Concepts Used
Candle Range Theory (Strict) [Manual Build]	2026-09-02	Active	crt_strict
BOS/CHoCH Structure Break + Strong Level Retest [Manual Build]	2026-09-02	Active	bos_choch_retest
Ichimoku 5m Indicator-Exit [Manual Build]	2026-09-02	Active	ichimoku_system
Ichimoku 5m Trailing-SL [Manual Build]	2026-09-02	Active	ichimoku_system
Ichimoku 15m Indicator-Exit [Manual Build]	2026-09-02	Active	ichimoku_system
Ichimoku 15m Trailing-SL [Manual Build]	2026-09-02	Active	ichimoku_system
Ichimoku 1h Indicator-Exit [Manual Build]	2026-09-02	Active	ichimoku_system
Ichimoku 1h Trailing-SL [Manual Build]	2026-09-02	Active	ichimoku_system
Ichimoku 1d Indicator-Exit [Manual Build]	2026-09-02	Active	ichimoku_system
Ichimoku 1d Trailing-SL [Manual Build]	2026-09-02	Active	ichimoku_system

SECTION 4 -- Backtest Results

Har strategy ka SABSE RECENT completed backtest batch (multi-symbol -- jitne bhi symbols us batch mein test hue, sab ke trades/PnL yahan sum kiye gaye hain), best-to-worst Net PnL ke hisaab se sorted.

Total strategies jinka kam se kam ek completed backtest maujood hai: 154 / 154

Strategy Name	Total Trades	Win Rate %	Net PnL	Profit Factor	Last Tested
HTF-LTF FVG/OB Confluence Entry [Manual Build]	2798	26.34	61443.51	1.369	2026-08-31
HTF-LTF FVG/OB Confluence Entry -- Medium Variant [Manual Build]	2798	26.34	61443.51	1.369	2026-08-31
Richard Dennis Turtle Trader Strategy [Manual Build]	1376	6.83	57909.27	1.260	2026-08-23
Liquidity Sweep Reversal Strategy [Manual Build]	4159	39.72	39901.06	1.194	2026-08-24
Candlestick Pattern Reversal Strategy [Manual Build]	3755	31.19	28415.73	1.143	2026-08-24
Liquidity Sweep + CISD (Pure Swing Variant) [Strict Variant]	12000	48.60	23019.79	1.041	2026-08-28
Liquidity Sweep Multi-Confirmation [Manual Build]	14761	39.65	18300.02	1.024	2026-08-28
Liquidity Sweep Multi-Confirmation [Medium Variant]	14761	39.65	18300.02	1.024	2026-08-28
Support/Resistance Breakout (Swing/Crypto Futures) [Manual Build]	4479	39.03	14333.28	1.055	2026-08-24
Range Breakout Volume Confirmation [Manual Build]	1636	25.00	14073.54	1.117	2026-08-31
Range Breakout Volume Confirmation -- Medium Variant [Manual Build]	1636	25.00	14073.54	1.117	2026-08-31
OTE Liquidity Sweep Reversal [Manual Build]	20253	40.43	12449.02	1.012	2026-08-28
OTE Liquidity Sweep Reversal [Medium Variant]	20253	40.43	12449.02	1.012	2026-08-28
OTE Liquidity Sweep Reversal [Strict Variant]	19882	40.82	10205.78	1.010	2026-08-28
Ichimoku 1h Trailing-SL [Manual Build]	517	8.70	6596.55	2.296	2026-09-02
Fair Value Gap Reversal -- Confirmation Medium variant [Manual Build]	1728	19.85	4830.76	1.373	2026-08-27
Fair Value Gap Reversal [Manual Build]	1728	19.85	4830.76	1.373	2026-08-26
Liquidity Sweep Reversal Strategy -- Confirmation Medium variant [Manual Build]	4159	39.72	3990.11	1.194	2026-08-27
Ichimoku 5m Trailing-SL [Manual Build]	743	6.33	3489.17	1.460	2026-09-02
Liquidity Sweep Multi-Confirmation [Strict Variant]	19277	25.81	3426.16	1.003	2026-08-28
Richard Dennis Turtle Trader Strategy -- Confirmation Strict variant [Manual Build]	1357	6.85	3119.36	1.226	2026-08-27
Richard Dennis Turtle Trader Strategy -- Confirmation Medium variant [Manual Build]	1357	6.85	3119.36	1.226	2026-08-27
Candlestick Pattern Reversal Strategy -- Confirmation Medium variant [Manual Build]	3755	31.19	2841.57	1.143	2026-08-26
Order Block Reversal [Manual Build]	1449	15.94	2096.10	1.169	2026-08-26

Strategy Name	Total Trades	Win Rate %	Net PnL	Profit Factor	Last Tested
Order Block Reversal -- Confirmation Medium variant [Manual Build]	1449	15.94	2096.10	1.169	2026-08-27
Support/Resistance Breakout (Swing/Crypto Futures) -- Confirmation Medium variant [Manual Build]	4479	39.03	1433.33	1.055	2026-08-27
Ichimoku 15m Trailing-SL [Manual Build]	572	8.04	1066.27	1.187	2026-09-02
Ichimoku 1d Trailing-SL [Manual Build]	126	33.33	932.94	2.064	2026-09-02
FVG Momentum Pullback (Structure) [Manual Build]	4417	24.95	415.71	1.016	2026-09-01
Supply/Demand Zone Strategy -- Fixed 1:2 TP variant [Manual Build]	70	2.86	12.53	1.015	2026-08-25
Supply/Demand Zone Strategy -- Confirmation Medium variant [Manual Build]	193	37.31	-187.91	0.863	2026-08-26
FRVP POC Reversal [Manual Build]	7921	36.45	-281.60	0.994	2026-08-31
FRVP POC Reversal [Manual Build] [Medium]	7921	36.45	-281.60	0.994	2026-08-31
Equal Highs/Lows Reversal -- Confirmation Medium variant [Manual Build]	2474	25.38	-300.86	0.982	2026-08-27
Equal Highs/Lows Reversal [Manual Build]	2474	25.38	-300.86	0.982	2026-08-26
Ichimoku 1d Indicator-Exit [Manual Build]	902	28.60	-359.89	0.936	2026-09-02
Dumb Money Concepts -- Confirmation Entry -- Confirmation Medium variant [Manual Build]	1602	29.15	-619.33	0.950	2026-08-26
Dumb Money Concepts -- Confirmation Entry [Manual Build]	1602	29.15	-619.33	0.950	2026-08-25
Donchian LWTI Volume Confluence [Manual Build]	2387	32.89	-1000.14	0.931	2026-08-31
Donchian LWTI Volume Confluence [Manual Build] [Medium]	2387	32.89	-1000.14	0.931	2026-08-31
9-20 EMA SMC Hybrid -- Confirmation Medium variant [Manual Build]	771	36.19	-1074.98	0.817	2026-08-27
Dumb Money Concepts -- Combined Confirmation [Manual Build]	1867	28.76	-1807.51	0.876	2026-08-25
Dumb Money Concepts -- Combined Confirmation -- Confirmation Medium variant [Manual Build]	1867	28.76	-1807.51	0.876	2026-08-27
Supply/Demand Zone Strategy [Manual Build]	193	37.31	-1879.05	0.863	2026-08-25
SAR + SMC (CISD Entry) -- Confirmation Strict variant [Manual Build]	570	31.05	-2079.86	0.190	2026-08-28
SAR + SMC (CISD Entry) [Manual Build]	571	31.00	-2090.42	0.189	2026-08-28
SAR + SMC (CISD Entry) -- Confirmation Medium variant [Manual Build]	571	31.00	-2090.42	0.189	2026-08-28
Dumb Money Concepts -- Confirmation Entry -- Fixed 1:2 TP variant [Manual Build]	1707	34.21	-2244.59	0.822	2026-09-06
Candlestick Pattern Reversal Strategy -- Confirmation Strict variant [Manual Build]	1570	33.50	-2779.96	0.771	2026-08-26
Market Structure Shift Reversal -- Confirmation Medium variant [Manual Build]	2445	22.13	-3239.98	0.823	2026-08-27
Market Structure Shift Reversal [Manual Build]	2445	22.13	-3239.98	0.823	2026-08-26

Strategy Name	Total Trades	Win Rate %	Net PnL	Profit Factor	Last Tested
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) -- Fixed 1:2 TP variant [Manual Build]	3287	37.91	-3255.74	0.839	2026-08-25
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) -- Confirmation Medium variant [Manual Build]	3287	37.91	-3255.74	0.839	2026-08-27
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) -- Confirmation Strict variant [Manual Build]	3287	37.91	-3255.74	0.839	2026-08-27
Double Confirmation CHoCH with Liquidity Trap -- Confirmation Medium variant [Manual Build]	8011	26.26	-3452.69	0.873	2026-08-27
Donchian LWTI Volume Confluence [Manual Build] [Strict]	1658	27.38	-3976.66	0.697	2026-08-31
Dumb Money Concepts -- Blind Entry [Manual Build]	2632	23.67	-4153.17	0.806	2026-08-25
Dumb Money Concepts -- Blind Entry -- Confirmation Medium variant [Manual Build]	2632	23.67	-4153.17	0.806	2026-08-27
Asian Range London Sweep -- Confirmation Strict variant [Manual Build]	3688	34.25	-4622.02	0.832	2026-08-27
Asian Range London Sweep -- Confirmation Medium variant [Manual Build]	3688	34.25	-4622.02	0.832	2026-08-27
Asian Range London Sweep -- Fixed 1:2 TP variant [Manual Build]	3688	34.25	-4622.02	0.832	2026-08-25
Heikin Ashi Trend Reversal [Manual Build] [Strict]	2094	40.69	-4715.11	0.660	2026-08-31
Heikin Ashi Trend Reversal [Manual Build] [Medium]	2232	40.14	-4774.15	0.680	2026-08-31
Heikin Ashi Trend Reversal [Manual Build]	2232	40.14	-4774.15	0.680	2026-08-31
Fixed Range Volume Profile -- Market Shape [Manual Build]	4374	32.60	-4917.54	0.819	2026-09-01
Dumb Money Concepts -- Combined Confirmation -- Confirmation Strict variant [Manual Build]	2961	16.04	-5055.48	0.820	2026-08-27
Support/Resistance Breakout (Swing/Crypto Futures) -- Confirmation Strict variant [Manual Build]	3507	33.48	-5454.48	0.800	2026-08-27
Liquidity Sweep Reversal Strategy -- Confirmation Strict variant [Manual Build]	2849	31.98	-5465.57	0.751	2026-08-27
Dumb Money Concepts -- Confirmation Entry -- Confirmation Strict variant [Manual Build]	3593	16.50	-6815.05	0.798	2026-08-26
Sniper Headshot Entry [Manual Build]	9082	29.30	-7491.65	0.839	2026-08-28
Sniper Headshot Entry -- Confirmation Medium variant [Manual Build]	9082	29.30	-7491.65	0.839	2026-08-28
Fibonacci Golden Zone Retracement [Manual Build] [Medium]	5151	30.50	-7507.91	0.747	2026-08-31
Fibonacci Golden Zone Retracement [Manual Build]	5146	30.51	-7743.99	0.738	2026-08-31
FVG Pure + Inverse FVG (Structure) [Manual Build]	10226	34.45	-7858.03	0.806	2026-09-01
Kotegawa Bear Market Reversal Strategy -- Confirmation Strict variant [Manual Build]	5581	23.13	-9206.90	0.799	2026-08-27

Strategy Name	Total Trades	Win Rate %	Net PnL	Profit Factor	Last Tested
Kotegawa Bear Market Reversal Strategy -- Confirmation Medium variant [Manual Build]	5581	23.13	-9206.90	0.799	2026-08-27
Double Confirmation CHoCH with Liquidity Trap -- Confirmation Strict variant [Manual Build]	4254	29.92	-10140.90	0.622	2026-08-27
Candle Range Theory (Strict) [Manual Build]	4438	33.15	-10330.37	0.674	2026-09-06
9-20 EMA SMC Hybrid [Manual Build]	771	36.19	-10749.76	0.817	2026-08-25
Kotegawa Bear Market Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	7074	30.93	-11007.77	0.789	2026-08-25
Dumb Money Concepts -- Blind Entry -- Confirmation Strict variant [Manual Build]	5047	14.17	-11278.27	0.758	2026-08-27
Sniper Headshot Entry -- Confirmation Strict variant [Manual Build]	12764	31.13	-11420.41	0.845	2026-08-28
Richard Dennis Turtle Trader Strategy -- Fixed 1:2 TP variant [Manual Build]	10694	33.37	-13307.38	0.827	2026-08-25
FRVP POC Reversal [Manual Build] [Strict]	9186	32.72	-13537.48	0.792	2026-08-31
Support/Resistance Breakout (Swing/Crypto Futures) -- Fixed 1:2 TP variant [Manual Build]	12525	34.71	-16239.40	0.820	2026-08-25
BOS/CHoCH Structure Break + Strong Level Retest [Manual Build]	10358	23.31	-17847.29	0.708	2026-09-02
Support/Resistance + Liquidity Sweep (Structure) [Manual Build]	12052	40.47	-18105.19	0.673	2026-09-01
PDH/PDL Multi-Timeframe Reversal -- Confirmation Medium variant [Manual Build]	10701	25.50	-19944.30	0.739	2026-08-28
PDH/PDL Multi-Timeframe Reversal [Manual Build]	10701	25.50	-19944.30	0.739	2026-08-28
Order Block Reversal -- Confirmation Strict variant [Manual Build]	26242	46.20	-20728.24	0.707	2026-08-27
4-Hour Range Breakout-Retest -- Fixed 1:2 TP variant [Manual Build]	15361	33.33	-21078.20	0.810	2026-08-25
4-Hour Range Breakout-Retest -- Confirmation Strict variant [Manual Build]	15361	33.33	-21078.20	0.810	2026-08-26
4-Hour Range Breakout-Retest -- Confirmation Medium variant [Manual Build]	15361	33.33	-21078.20	0.810	2026-08-26
Previous High/Low Reversal -- Confirmation Strict variant [Manual Build]	19470	31.63	-23070.36	0.792	2026-08-27
Lower Time Frame Liquidity Reversal Strategy -- Confirmation Strict variant [Manual Build]	14248	32.74	-23129.40	0.769	2026-08-27
Lower Time Frame Liquidity Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	14248	32.74	-23129.40	0.769	2026-08-25
Lower Time Frame Liquidity Reversal Strategy -- Confirmation Medium variant [Manual Build]	14248	32.74	-23129.40	0.769	2026-08-27
FVG Momentum Pullback (Fixed 1:3) [Manual Build]	12712	25.17	-23658.72	0.726	2026-09-06
Order Block Trading (Strict) [Manual Build]	11615	36.25	-23930.74	0.611	2026-09-02
Order Block Trading (Strict) [Manual Build]	11615	36.25	-23930.74	0.611	2026-09-02
Ichimoku 1h Indicator-Exit [Manual Build]	18318	27.18	-24645.61	0.711	2026-09-02

Strategy Name	Total Trades	Win Rate %	Net PnL	Profit Factor	Last Tested
FVG Momentum Pullback (Fixed 1:2) [Manual Build]	13468	33.15	-24947.70	0.682	2026-09-06
Laxman Rekha 5-EMA Trigger-Candle Strategy [Manual Build]	16674	24.77	-25066.18	0.796	2026-09-06
Laxman Rekha 5-EMA Trigger-Candle Strategy -- Confirmation Strict variant [Manual Build]	16674	24.77	-25066.18	0.796	2026-09-06
Laxman Rekha 5-EMA Trigger-Candle Strategy -- Confirmation Medium variant [Manual Build]	16674	24.77	-25066.18	0.796	2026-09-06
Support/Resistance + Liquidity Sweep (Fixed-RR) [Manual Build]	14369	44.55	-25541.59	0.669	2026-09-01
Liquidity Sweep + CUSD (Pure Swing Variant) [Medium Variant]	14993	58.55	-25666.85	0.948	2026-08-28
Liquidity Sweep + CUSD (Pure Swing Variant) [Manual Build]	14993	58.55	-25666.85	0.948	2026-08-28
Order Block Trading (Loose) [Manual Build]	13962	29.80	-26056.65	0.667	2026-09-02
Order Block Trading (Loose) [Manual Build]	13962	29.80	-26056.65	0.667	2026-09-02
Market Structure Shift Reversal -- Confirmation Strict variant [Manual Build]	21562	41.40	-27330.84	0.554	2026-08-27
9-20 EMA SMC Hybrid -- Fixed 1:2 TP variant [Manual Build]	17722	34.47	-27610.52	0.756	2026-08-25
Previous High/Low Reversal [Manual Build]	18925	39.68	-28196.50	0.669	2026-08-26
Previous High/Low Reversal -- Confirmation Medium variant [Manual Build]	18925	39.68	-28196.50	0.669	2026-08-27
Liquidity Sweep Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	27749	33.51	-31885.94	0.821	2026-08-25
Fibonacci Golden Zone Retracement [Manual Build] [Strict]	19523	35.89	-32516.49	0.612	2026-08-31
SMA-Alignment Strategy (Approximate - ZIP Indicator Excluded) [Manual Build]	3287	37.91	-32557.38	0.839	2026-08-25
Fair Value Gap Reversal -- Confirmation Strict variant [Manual Build]	29153	48.03	-32601.40	0.538	2026-08-27
Laxman Rekha 5-EMA Trigger-Candle Strategy -- Fixed 1:2 TP variant [Manual Build]	26450	33.15	-33335.00	0.785	2026-09-06
Liquidity Sweep + Engulfing Candle (Strict) [Manual Build]	24258	32.43	-34204.29	0.721	2026-09-06
Double Confirmation CHoCH with Liquidity Trap [Manual Build]	8011	26.26	-34526.92	0.873	2026-08-25
Equal Highs/Lows Reversal -- Confirmation Strict variant [Manual Build]	31238	45.78	-34666.34	0.534	2026-08-27
FVG 50% Equilibrium Entry [Manual Build] [Strict]	27033	39.30	-34721.56	0.632	2026-08-31
FVG 50% Equilibrium Entry [Manual Build]	27543	39.01	-35017.81	0.635	2026-08-31
FVG 50% Equilibrium Entry [Manual Build] [Medium]	27543	39.01	-35017.81	0.635	2026-08-31
Daily Liquidity Scalping Strategy -- Confirmation Medium variant [Manual Build]	48947	46.07	-35020.05	0.785	2026-08-27
Daily Liquidity Scalping Strategy -- Confirmation Strict variant [Manual Build]	48947	46.07	-35020.05	0.785	2026-08-27

Strategy Name	Total Trades	Win Rate %	Net PnL	Profit Factor	Last Tested
Candle Range Theory (Loose) [Manual Build]	24911	33.52	-35283.23	0.702	2026-09-06
Candlestick Pattern Reversal Strategy -- Fixed 1:2 TP variant [Manual Build]	25766	32.83	-35851.54	0.749	2026-08-25
FVG Pure + Inverse FVG (Fixed 1:2) [Manual Build]	25904	32.88	-37195.08	0.670	2026-09-06
Daily Liquidity Scalping Strategy -- Fixed 1:2 TP variant [Manual Build]	67866	51.72	-39863.57	0.769	2026-08-25
4H Fractal Sweep Reversal -- Confirmation Medium variant [Manual Build]	34221	32.82	-41154.65	0.661	2026-08-28
4H Fractal Sweep Reversal [Manual Build]	34221	32.82	-41154.65	0.661	2026-08-28
Candle Range Theory (CRT) Strategy 2.0 -- Confirmation Medium variant [Manual Build]	42013	33.74	-42192.20	0.733	2026-08-27
Candle Range Theory (CRT) Strategy 2.0 -- Fixed 1:2 TP variant [Manual Build]	42013	33.74	-42192.20	0.733	2026-08-25
Candle Range Theory (CRT) Strategy 2.0 -- Confirmation Strict variant [Manual Build]	42013	33.74	-42192.20	0.733	2026-08-27
Liquidity Sweep + Engulfing Candle (Loose) [Manual Build]	43428	32.75	-43030.32	0.711	2026-09-06
Liquidity Sweep + Engulfing Candle (Loose) [Manual Build]	43428	32.75	-43030.32	0.711	2026-09-06
Asian Range London Sweep [Manual Build]	3688	34.25	-46220.23	0.832	2026-08-25
Double Confirmation CHOCH with Liquidity Trap -- Fixed 1:2 TP variant [Manual Build]	73443	26.71	-46806.54	0.642	2026-08-25
HTF Key Level Engulfing Reversal -- Confirmation Medium variant [Manual Build]	98556	33.27	-47318.47	0.683	2026-08-28
HTF Key Level Engulfing Reversal [Manual Build]	98556	33.27	-47318.47	0.683	2026-08-28
Ichimoku 15m Indicator-Exit [Manual Build]	66896	24.34	-47694.13	0.597	2026-09-02
Ichimoku 5m Indicator-Exit [Manual Build]	190055	18.92	-49761.42	0.412	2026-09-06
Range Breakout Volume Confirmation -- Strict Variant [Manual Build]	2504	32.71	-64966.32	0.630	2026-08-31
Kotegawa Bear Market Reversal Strategy [Manual Build]	5581	23.13	-92069.01	0.799	2026-08-25
4-Hour Range Breakout-Retest [Manual Build]	15361	33.33	-210781.99	0.810	2026-08-25
HTF-LTF FVG/OB Confluence Entry -- Strict Variant [Manual Build]	10862	31.11	-228605.51	0.624	2026-08-31
Lower Time Frame Liquidity Reversal Strategy [Manual Build]	14248	32.74	-231294.03	0.769	2026-08-25
Daily Liquidity Scalping Strategy [Manual Build]	48947	46.07	-350200.55	0.785	2026-08-25
Candle Range Theory (CRT) Strategy 2.0 [Manual Build]	42013	33.74	-421921.95	0.733	2026-08-25
HTF Trend Trendline Breakout -- Strict Variant [Manual Build]	36816	15.47	-463625.04	0.230	2026-08-31
HTF Trend Trendline Breakout -- Medium Variant [Manual Build]	50447	14.32	-482971.64	0.261	2026-08-31
HTF Trend Trendline Breakout [Manual Build]	50447	14.32	-482971.64	0.261	2026-08-31

SECTION 5 -- Paper Trading Summary

Ye paper_strategy_performance table se hai -- jo bhi strategy kabhi bhi live paper-trading loop mein trade kar chuki hai uska All-Time record (Local desktop bot ka data hai; is report mein cloud/deployment ka koi zikar nahi kiya gaya jaisa maanga gaya tha).

Total strategies jo paper trading mein kabhi active hui: 55 | Total combined trades: 769

Strategy Name	Trades	Wins	Losses	Win Rate %	All-Time PnL	Avg RR
Richard Dennis Turtle Trader Strategy [Manual Build]	8	1	7	12.50	8.74	88.14
Laxman Rekha 5-EMA Trigger-Candle Strategy [Manual Build]	18	9	9	50.00	8.00	5.21
Sniper Headshot Entry [Manual Build]	40	23	17	57.50	3.47	0.84
PDH/PDL Multi-Timeframe Reversal [Manual Build]	5	3	2	60.00	3.23	2.68
Kotegawa Bear Market Reversal Strategy [Manual Build]	2	2	0	100.00	2.97	10.39
BOS/CHoCH Structure Break + Strong Level Retest [Manual Build]	7	1	6	14.29	2.10	14.31
OTE Liquidity Sweep Reversal [Manual Build]	3	1	2	33.33	1.18	4.95
Heikin Ashi Trend Reversal [Manual Build]	2	1	1	50.00	0.94	2.52
Daily Liquidity Scalping Strategy [Manual Build]	26	8	18	30.77	0.90	4.46
9-20 EMA SMC Hybrid [Manual Build]	1	1	0	100.00	0.42	1.87
Liquidity Sweep + Engulfing Candle (Loose) [Manual Build]	6	2	4	33.33	-0.34	2.58
FRVP POC Reversal [Manual Build]	27	16	11	59.26	-0.36	1.20
FVG Pure + Inverse FVG (Fixed 1:2) [Manual Build]	1	0	1	0.00	-0.57	N/A
Donchian LWTI Volume Confluence [Manual Build]	12	5	7	41.67	-0.68	1.51
Candlestick Pattern Reversal Strategy [Manual Build]	32	16	16	50.00	-0.83	1.16
Candle Range Theory (Strict) [Manual Build]	5	1	4	20.00	-1.26	1.61
Range Breakout Volume Confirmation [Manual Build]	22	11	11	50.00	-1.44	0.69
Dumb Money Concepts -- Confirmation Entry [Manual Build]	4	1	3	25.00	-1.45	7.00
FVG Momentum Pullback (Structure) [Manual Build]	5	1	4	20.00	-1.55	0.20
Liquidity Sweep Multi-Confirmation [Manual Build]	31	13	18	41.94	-1.59	0.89
Liquidity Sweep + Engulfing Candle (Strict) [Manual Build]	3	0	3	0.00	-1.60	N/A
Ichimoku 5m Indicator-Exit [Manual Build]	3	0	3	0.00	-1.61	N/A
Ichimoku 5m Trailing-SL [Manual Build]	3	0	3	0.00	-1.61	N/A

Strategy Name	Trades	Wins	Losses	Win Rate %	All-Time PnL	Avg RR
Ichimoku 15m Indicator-Exit [Manual Build]	3	0	3	0.00	-1.63	N/A
Ichimoku 15m Trailing-SL [Manual Build]	3	0	3	0.00	-1.63	N/A
Fair Value Gap Reversal [Manual Build]	21	15	6	71.43	-1.65	0.48
FVG Momentum Pullback (Fixed 1:3) [Manual Build]	3	0	3	0.00	-1.68	N/A
FVG Momentum Pullback (Fixed 1:2) [Manual Build]	3	0	3	0.00	-1.68	N/A
Asian Range London Sweep [Manual Build]	6	1	5	16.67	-1.70	3.53
Liquidity Sweep Reversal Strategy [Manual Build]	26	15	11	57.69	-1.76	0.92
Order Block Trading (Strict) [Manual Build]	7	2	5	28.57	-2.00	0.85
4-Hour Range Breakout-Retest [Manual Build]	12	3	9	25.00	-2.02	3.76
Fixed Range Volume Profile -- Market Shape [Manual Build]	10	3	7	30.00	-2.32	1.37
Previous High/Low Reversal [Manual Build]	33	11	22	33.33	-2.44	2.00
Dumb Money Concepts -- Combined Confirmation [Manual Build]	6	1	5	16.67	-2.46	7.00
FVG 50% Equilibrium Entry [Manual Build]	16	3	13	18.75	-2.53	2.79
4H Fractal Sweep Reversal [Manual Build]	10	2	8	20.00	-2.54	1.36
Dumb Money Concepts -- Blind Entry [Manual Build]	8	2	6	25.00	-2.58	3.00
FVG Pure + Inverse FVG (Structure) [Manual Build]	18	9	9	50.00	-2.73	0.21
HTF Trend Trendline Breakout [Manual Build]	5	0	5	0.00	-2.78	N/A
Support/Resistance + Liquidity Sweep (Fixed-RR) [Manual Build]	18	8	10	44.44	-2.82	0.76
Double Confirmation CHoCH with Liquidity Trap [Manual Build]	18	9	9	50.00	-3.08	0.28
Lower Time Frame Liquidity Reversal Strategy [Manual Build]	16	3	13	18.75	-3.94	2.99
Support/Resistance Breakout (Swing/Crypto Futures) [Manual Build]	11	2	9	18.18	-4.02	0.73
Support/Resistance + Liquidity Sweep (Structure) [Manual Build]	10	1	9	10.00	-4.05	1.02
Order Block Trading (Loose) [Manual Build]	8	0	8	0.00	-4.12	N/A
Candle Range Theory (Loose) [Manual Build]	14	2	12	14.29	-4.13	2.16
Order Block Reversal [Manual Build]	15	4	11	26.67	-4.39	0.08
HTF-LTF FVG/OB Confluence Entry [Manual Build]	28	14	14	50.00	-4.45	0.35
Equal Highs/Lows Reversal [Manual Build]	21	9	12	42.86	-4.46	0.24

Strategy Name	Trades	Wins	Losses	Win Rate %	All-Time PnL	Avg RR
Fibonacci Golden Zone Retracement [Manual Build]	24	10	14	41.67	-4.75	0.46
Market Structure Shift Reversal [Manual Build]	26	10	16	38.46	-5.16	0.43
Candle Range Theory (CRT) Strategy 2.0 [Manual Build]	14	1	13	7.14	-5.72	1.90
HTF Key Level Engulfing Reversal [Manual Build]	53	16	37	30.19	-5.78	1.44
Liquidity Sweep + CISD (Pure Swing Variant) [Manual Build]	37	17	20	45.95	-5.85	0.50